Slackware Linux Essentials - Version Beta



Alan Hicks Chris Lumens David Cantrell Logan Johnson

Copyright © 1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005 Slackware L

Slackware Linux is a registered trademark of Patrick Volkerding and Sla

Linux is a registered trademark of Linus Torvalds.

America Online and AOL are registered trademarks of America Online, Inc
States and/or other countries.

Apple, FireWire, Mac, Macintosh, Mac OS, Quicktime, and TrueType are tr
Apple Computer, Inc., registered in the United States and other countri

IBM, AIX, EtherJet, Netfinity, OS/2, PowerPC, PS/2, S/390, and ThinkPad
of International Business Machines Corporation in the United States, ot
or both.

IEEE, POSIX, and 802 are registered trademarks of Institute of Electric
Electronics Engineers, Inc. in the United States.

Intel, Celeron, EtherExpress, i386, i486, Itanium, Pentium, and Xeon ar
registered trademarks of Intel Corporation or its subsidiaries in the U
other countries.

Microsoft, IntelliMouse, MS-DOS, Outlook, Windows, Windows Media and Wi
either registered trademarks or trademarks of Microsoft Corporation in
and/or other countries.

Netscape and the Netscape Navigator are registered trademarks of Netsca
Corporation in the U.S. and other countries.

Red Hat, RPM, are trademarks or registered trademarks of Red Hat, Inc.
States and other countries.

XFree86 is a trademark of The XFree86 Project, Inc.

Many of the designations used by manufacturers and sellers to distingui
products are claimed as trademarks. Where those designations appear in
and Slackware Linux, Inc. was aware of the trademark claim, the designa
followed by the “™” or the “®” symbol.

1-57176-338-4

Índice

- Slackware Linux Essentials - Version Beta
- Foreward
  - Intended Audience
  - Why A New Slackware Book?
  - Conventions Used in this Book
  - Acknowledgements
- 1. Introduction to Slackware
  - 1.1 Why Use Slackware?
  - 1.2 Differences Compared to Other Linux Distributions

- 1.3 Licensing
- 2. Installation
  - 2.1 Booting the Installer
  - 2.2 Partitioning
  - 2.3 The setup Program
    - 2.3.1 Help
    - 2.3.2 Keymap
    - 2.3.3 Addswap
    - 2.3.4 Target
    - 2.3.5 Source
    - 2.3.6 Select
    - 2.3.7 Install
    - 2.3.8 Configure
- 3. Booting
  - 3.1 mkinitrd
  - 3.2 LILO
  - 3.3 Dual Booting
    - 3.3.1 Dual Booting with Partitions
    - 3.3.2 Dual Booting from Hard Drives
- 4. Basic Shell Commands
  - 4.1 System Documentation
  - 4.2 Dealing with Files and Directories
    - 4.2.1 Listing Files and Directory Contents
    - 4.2.2 Moving Around the Filesystem
    - 4.2.3 File and Directory Creation and Deletion
  - 4.3 Archive and Compression
    - 4.3.1 zip and unzip
    - 4.3.2 gzip
    - 4.3.3 bzip2
    - 4.3.4 XZ / LZMA
    - 4.3.5 tar

- 4.4 Reading Documents
  - 4.4.1 more
  - 4.4.2 less
- 4.5 Linking

- 5. The Bourne Again Shell
  - 5.1 What Is A Shell?
  - 5.2 Environment Variables
  - 5.3 Wildcards
  - 5.4 Tab Completion
  - 5.5 Input and Output Redirection
  - 5.6 Task Management
  - 5.7 Terminals
  - 5.8 Customization
- 6. Process Control
  - 6.1 ps
  - 6.2 kill and killall
  - 6.3 top
  - 6.4 cron
- 7. The X Window System
  - 7.1 What Is (And Isn't) X
  - 7.2 Configuring the X Server
  - 7.3 Choosing a Window Manager
  - 7.4 Setting Up A Graphical Login
- 8. Printing
  - 8.1 Choosing A Printer
  - 8.2 Getting the driver
  - 8.3 Setting Up a Printer in CUPS
  - 8.4 Printing from the Command Line
    - 8.4.1 Formatting for Print
- 9. Users and Groups
  - 9.1 What Are Users and Groups?
  - 9.2 Managing Users and Groups

- 9.3 Other User and Group Tools
- 9.4 Managing Users and Groups Manually

• 10. Filesystem Permissions

- 10.1 Permissions Overview
- 10.2 chmod, chown, and chgrp
- 10.3 SUID, SGID, and the "Sticky" Bit

• 11. Working with Filesystems

- 11.1 The Filesystem Hierarchy
- 11.2 Local Filesystem Types
  - 11.2.1 ext2
  - 11.2.2 ext3
  - 11.2.3 ext4
  - 11.2.4 reiserfs
  - 11.2.5 XFS
  - 11.2.6 JFS
  - 11.2.7 iso9660
  - 11.2.8 vfat
  - 11.2.9 swap
- 11.3 Using mount
- 11.4 Network Filesystems
  - 11.4.1 NFS
  - 11.4.2 SMB

• 12. vi

- 12.1 What is vi?
- 12.2 The Different Modes of vi
- 12.3 Opening, Saving, and Quitting
- 12.4 Moving Around
- 12.5 Editing A Document
- 12.6 vi Cheat Sheet

• 13. Emacs

- 13.1 What is Emacs?
- 13.2 Starting Out

- 13.3 How to Move Around
- 13.4 Getting Help
- 13.5 Calling Functions
- 13.6 Emacs Cheat Sheet

- 14. Networking
  - 14.1 netconfig
  - 14.2 Manual Configuration
  - 14.3 Automatic Configuration with rc.inet1.conf
- 15. Wireless Networking
  - 15.1 iwconfig
  - 15.2 Wired Equivilant Protection (or Lack Thereof)
  - 15.3 Wifi Protected Access
    - 15.3.1 rc.inet1.conf revisited
  - 15.4 wicd
- 16. Basic Networking Utilities
  - 16.1 Network Diagnostic Tools
    - 16.1.1 ping
    - 16.1.2 traceroute
    - 16.1.3 telnet
    - 16.1.4 ssh
    - 16.1.5 tcpdump
    - 16.1.6 nmap
    - 16.1.7 host
    - 16.1.8 dig
    - 16.1.9 finger
  - 16.2 Web Browsers
    - 16.2.1 lynx
    - 16.2.2 links
    - 16.2.3 wget
  - 16.3 Mail Clients
    - 16.3.1 pine
    - 16.3.2 mutt

        - 16.3.3 mailx
    - 16.4 FTP Clients
        - 16.4.1 ftp
        - 16.4.2 ncftp
        - 16.4.3 lftp
    - 16.5 rsync
- 17. Package Management
    - 17.1 pkgtool
    - 17.2 Installing, Removing, and Upgrading Packages
    - 17.3 Package Compression Formats
        - 17.3.1 slackpkg
        - 17.3.2 rpm2tgz
- 18. Keeping Track of Updates
    - 18.1 The -stable Branch
    - 18.2 Security Update Mailing List
    - 18.3 Upgrading Slackware Versions
- 19. The Linux Kernel
    - 19.1 What Does the Kernel Do?
    - 19.2 Working with Modules
    - 19.3 Compiling A Kernel and Why to do So

Foreward

Intended Audience

Why A New Slackware Book?

Slackware Linux may be one of the oldest surviving Linux distributions but it's still regularly updated and includes the latest releases of many of the most popular free

software programs. While Slackware does aim to maintain its traditional UNIX roots and values, there is no escaping "progress". Subsystems change, window managers come and go and new ways are devised to manage the complexities of a modern operating system. While we do resist change for change's sake, it's inevitable that as things evolve documentation becomes stale — books are no exception.

Conventions Used in this Book

Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

1. Introduction to Slackware

1.1 Why Use Slackware?

Slackware has a long tradition of excellence. Started in 1992 and first released in 1993, Slackware is the oldest surviving commercial Linux distribution. Slackware's focus on making a clean, simple Linux distribution that is as UNIX-like as possible makes it a natural choice for those people who really want to learn about Linux and

other UNIX-like operating systems. In a 2012 interview, Slackware founder and benevolent dictator for life, Patrick Volkerding, put it thusly.

"I try not to let things get juggled around simply for the sake of making them different. People who come back to Slackware after a time tend to be pleasantly surprised that they don't need to relearn how to do everything. This has given us quite a loyal following, for which I am grateful."

Slackware's simplicity makes it ideal for those users who want to create their own custom systems. Of course, Slackware is great in its own right as a desktop, workstation, or server as well.

1.2 Differences Compared to Other Linux Distributions

There are a great number of differences between Slackware and other mainstream distributions such as Red Hat, Debian, and Ubuntu. Perhaps the greatest difference is the lack of "hand-holding" that Slackware will do for the administrator. Many of those other distributions ship with custom graphical configuration tools for all manner of services. In many cases, these configuration tools are the preferred method of setting up applications on these systems and will overwrite any changes you make to the configuration files via other means. These tools often make it easy (or at least possible) for a rookie with no in-depth understanding of his system to setup basic services; however, they also make it difficult to do anything too out of the ordinary. In contrast, Slackware expects you, the system administrator, to do these tasks on your own. Slackware provides no general purpose setup tools beyond those included with the source code published by upstream developers. This means there is often a somewhat steeper learning curve associated with Slackware, even for those users familiar with other Linux distributions, but also makes it much easier to do whatever you want with your operating system.

Also, you may hear users of other distributions say that Slackware has no package management system. This is completely and obviously false. Slackware has always had package management (see for more information). What it does not have is automatic dependency resolution - Slackware's package tools trade dependency management for simplicity, ease-of-use, and reliability.

1.3 Licensing

Each piece of Slackware (this is true of all Linux distributions) is developed by different people (or teams of people), and each group has their own ideas about what it means to be "free". Because of this, there are literally dozens and dozens of different licenses granting you different permissions regarding their use or distribution. Fortunately dealing with free software licenses isn't as difficult as it may first appear. Most things are licensed with either the Gnu General Public License or the BSD license. Sometimes you'll encounter a piece of software with a different license, but in almost all cases they are remarkably similar to either the GPL or the BSD license.

Probably the most popular license in use within the Free Software community is the GNU General Public License. The GPL was created by the Free Software Foundation , which actively works to create and distribute software that guarantees the freedoms which they believe are basic rights. In fact, this is the very group that coined the term "Free Software." The GPL imposes no restrictions on the use of software. In fact, you don't even have to accept the terms of the license in order to use the software, but you are not allowed to redistribute the software or any changes to it without abiding by the terms of the license agreement. A large number of software projects shipped with Slackware, from the Linux kernel itself to the Samba project, are released under the terms of the GPL.

Another very common license is the BSD license, which is arguably "more free" than the GPL because it imposes virtually no restrictions on derivative works. The BSD license simply requires that the copyright remain intact along with a simple disclaimer. Many of the utilities specific to Slackware are licensed with a BSD-style license, and this is the preferred license for many smaller projects and tools.

2. Installation

Slackware's installation is a bit more simplistic than that of most other Linux distributions and is very reminiscent of installing one of the varieties of BSD

operating systems. If you're familiar with those, you should feel right at home. If you've never installed Slackware or have only used distributions that make use of graphical installers, you may feel a bit overwhelmed at first. Don't panic! The installation is very easy once you understand it, and it works on just about any x86 or x86_64 platform.

The latest versions of Slackware Linux are distributed on DVD or CD media, but Slackware can be installed in a variety of other ways. We're only going to focus on the most common method - booting from a DVD - in this book. If you don't have a CD or DVD drive, you might wish to take a look at the various README files inside the `usb-and-pxe-installers` directory at your favorite Slackware mirror. This directory includes the necessary files and instructions for booting the Slackware installer from a USB flash drive or from a network card that support PXE. The files there are the best source of information available for such boot methods.

2.1 Booting the Installer

Booting the installer is simply a process of inserting the Slackware install disk into your CD or DVD drive and rebooting. You may have to enter your computer's BIOS and alter the boot order to place the optical drive at a higher boot priority than your hard drives. Some computers allow you to change the boot order on the fly by pressing a specific function key during system boot-up. Since every computer is different, we can't offer instructions on how to do this, but the method is simple on nearly all machines.

Once your computer boots from the CD you'll be taken to a screen that allows you to enter any special kernel parameters. This is here primarily to allow you to use the installer as a sort of rescue disk. Some systems may need special kernel parameters in order to boot, but these are very rare exceptions to the norm. Most users can simply press enter to let the kernel boot.

```
Welcome to Slackware version 14.0 (Linux kernel 3.2.27)!

If you need to pass extra parameters to the kernel, enter them at the p
below after the name of the kernel to boot (huge.s etc).
```

```
In a pinch, you can boot your system from here with a command like:

boot: huge.s root=/dev/sda1 rdinit= ro

In the example above, /dev/sda1 is the / Linux partition.

To test your memory with memtest86+, enter memtest on the boot line bel

This prompt is just for entering extra parameters. If you don't need to
any parameters, hit ENTER to boot the default kernel "huge.s" or press
for a listing of more kernel choices.
```

After pressing ENTER you should see a lot of text go flying across your screen. Don't be alarmed, this is all perfectly normal. The text you see is generated by the kernel during boot-up as it discovers your hardware and prepares to load the operating system (in this case, the installer). You can later read these messages with the `dmesg` (1) command if you're interested. Often these messages are very important for troubleshooting any hardware problems you may have. Once the kernel has completed its hardware discovery, the messages should stop and you'll be given an option to load support for non-us keyboards.

```
<OPTION TO LOAD SUPPORT FOR NON-US KEYBOARD>

If you are not using a US keyboard, you may need to load a different
keyboard map. To select a different keyboard map, please enter 1
now. To continue using the US map, just hit enter.

Enter 1 to select a keyboard map: _
```

Entering 1 and pressing ENTER will give you a list of keyboard mappings. Simply select the mapping that matches your keyboard type and continue on.

```
Welcome to the Slackware Linux installation disk! (version 14.0)

'###### IMPORTANT! READ THE INFORMATION BELOW CAREFULLY. ######

- You will need one or more partitions of type 'Linux' prepared. It is
  recommended that you create a swap partition (type 'Linux swap') pric
  to installation. For more information, run 'setup' and read the help
- If you're having problems that you think might be related to low memo
  can try activating a swap partition before you run setup. After makir
  swap partition (type 82) with cfdisk or fdisk, activate it like this
    mkswap /dev/<partition> ; swapon /dev/<partition>
- Once you have prepared the disk partitions for Linux, type 'setup' to
  the installation process.
- If you do not have a color monitor, type: TERM=vt100
  before you start 'setup'.

You may now login as 'root'.

slackware login: root
```

Unlike other Linux distributions which boot you directly into a dedicated installer program, Slackware's installer places you in a limited Linux distribution loaded into your system's RAM. This limited distribution is then used to run all the installation programs manually, or can be used in emergencies to fix a broken system that fails to boot. Now that you're logged in as root (there is no password within the installer) it's time to start setting up your disks. At this point, you may setup software RAID or LVM support if you wish or even an encrypted root partition, but those topics are outside of the scope of this book. I encourage you to refer to the excellent `README_RAID.TXT`, `README_LVM.TXT`, and `README_CRYPT.TXT` files on your CD if you desire to setup your system with these advanced tools. Most users won't have any need to do so and should proceed directly to partitioning.

2.2 Partitioning

Unlike many other Linux distributions, Slackware does not make use of a dedicated graphical disk partitioning tool in its installer. Rather, Slackware makes use of the traditional Linux partitioning tools, the very same tools that you will have available once you've installed Slackware. Traditionally, partitioning is performed with either `fdisk` (8) or `cfdisk` (8), both of which are console tools. `cfdisk` is preferred by many people because it is curses menu-based, but either works well. Additionally, Slackware includes `sfdisk` (8) and `gdisk` (8). These are more powerful command-line partitioning tools. `gdisk` is required to alter GUID partition tables found on some of today's larger hard drives. In this book, we're going to focus on using `fdisk` , but the other tools are similar. You can find additional instructions for using these other tools online or in their man pages.

In order to partition your hard drive, you'll first need to know how to identify it. In Linux, all hardware is identified by a special file called a device file. These are (typically) located in the `/dev` directory. Nearly all hard drives today, are identified as SCSI hard drives by the kernel, and as such, they'll be assigned a device node such as `/dev/sda` . (Once upon a time each hard drive type had its own unique identifier such as /dev/hda for the first IDE drive. Over the years the kernel's SCSI subsystem morphed into a generic drive access system and came to be used for all hard disks and optical drives no matter how they are connected to your computer. If you think this is confusing, imagine what it would be like if you had a system with a SCSI hard drive, a SATA CD-ROM, and a USB memory stick, all with unique subsystem indentifiers. The current system is not only cleaner, but performs better as well.)

If you don't know which device node is assigned to your hard drive, `fdisk` can help you find out.

```
root@slackware:/# fdisk -l

Disk /dev/sda: 72.7 GB, 72725037056 bytes
255 heads, 63 sectors/track, 8841 cylinders
```

```
Units = cylinders of 16065 * 512 = 8225280 bytes
```

Here, you can see that my system has a hard drive located at `/dev/sda` that is 72.7 GB in size. You can also see some additional information about this hard drive. The `-l` argument to `fdisk` tells it to display the hard drives and all the partitions it finds on those drives, but it won't make any changes to the disks. In order to actually partition our drives, we'll have to tell `fdisk` the drive on which to operate.

```
root@slackware:/# fdisk /dev/sda

The number of cylinders for this disk is set to 8841.
There is nothing wrong with that, but this is larger than 1024,
and could in certain setups cause problems with:
1) software that runs at boot time (e.g., old versions of LILO)
2) booting and partitioning software from other OSs (e.g., DOS FDISK, O

Command (m for help):
```

Now we've told fdisk what disk we wish to partition, and it has dropped us into command mode after printing an annoying warning message. The 1024 cylinder limit has not been a problem for quite some time, and Slackware's boot loader will have no trouble booting disks larger than this. Typing `m` and pressing ENTER will print out a helpful message telling you what to do with `fdisk` .

```
Command (m for help): m

Command action

  a toggle a bootable flag
  b edit bsd disklabel
  c toggle the dos compatibility flag
```

```
  d delete a partition
  l list known partition types
  m print this menu
  n add a new partition
  o create a new empty DOS partition table
  p print the partition table
  q quit without saving changes
  s create a new empty Sun disklabel
  t change a partition's system id
  u change display/entry units
  v verify the partition table
  w write table to disk and exit
  x extra functionality (experts only)
```

Now that we know what commands will do what, it's time to begin partitioning our drive. At a minimum, you will need a single `/` partition, and you should also create a swap partition. You might also want to make a separate `/home` partition for storing user files (this will make it easier to upgrade later or to install a different Linux operating system by keeping all of your users' files on a separate partition). Therefore, let's go ahead and make three partitions. The command to create a new partition is `n` (which you noticed when you read the help).

```
Command: (m for help): n

Command action
  e extended
  p primary partition (1-4)
p

Partition number (1-4): 1
First cylinder (1-8841, default 1): 1
Last cylinder or +size or +sizeM or +sizeK (1-8841, default 8841): +8G

Command (m for help): n
```

```
Command action
  e extended
  p primary partition (1-4)
p
Partition number (1-4): 2
First cylinder (975-8841, default 975): 975
Last cylinder or +size or +sizeM or +sizeK (975-8841, default 8841): +1
```

Here we have created two partitions. The first is 8GB in size, and the second is only 1GB. We can view our existing partitions with the `p` command.

```
Command (m for help): p

Disk /dev/sda: 72.7 GB, 72725037056 bytes
255 heads, 63 sectors/track, 8841 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes

  Device Boot      Start         End      Blocks   Id  System
  /dev/sda1            1         974     7823623+  83  Linux
  /dev/sda2          975        1097      987997+  83  Linux
```

Both of these partitions are of type "83" which is the standard Linux filesystem. We will need to change `/dev/sda2` to type "82" in order to make this a swap partition. We will do this with the `t` argument to `fdisk` .

```
Command (m for help): t
Partition number (1-4): 2
Hex code (type L to list codes): 82

Command (me for help): p

Disk /dev/sda: 72.7 GB, 72725037056 bytes
```

```
255 heads, 63 sectors/track, 8841 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes

  Device Boot      Start         End      Blocks   Id  System
  /dev/sda1            1         974     7823623+  83  Linux
  /dev/sda2          975        1097      987997+  82  Linux s
```

The swap partition is a special partition that is used for virtual memory by the Linux kernel. If for some reason you run out of RAM, the kernel will move the contents of some of the RAM to swap in order to prevent a crash. The size of your swap partition is up to you. A great many people have participated in a great many flamewars on the size of swap partitions, but a good rule of thumb is to make your swap partition about twice the size of your system's RAM. Since my machine has only 512MB of RAM, I decided to make my swap partition 1GB. You may wish to experiment with your swap partition's size and see what works best for you, but generally there is no harm in having "too much" swap. If you plan to use hibernation (suspend to disk), you will need to have at least as much swap space as you have physical memory (RAM), so keep that in mind.

At this point we can stop, write these changes to the disk, and continue on, but I'm going to go ahead and make a third partition which will be mounted at `/home` .

```
Command: (me for help): n
Command action
  e extended
  p primary partition (1-4)
p
Partition number (1-4): 3
First cylinder (1098-8841, default 1098): 1098
Last cylinder or +size or +sizeM or +sizeK (1098-8841, default 8841): 8
```

Now it's time to finish up and write these changes to disk.

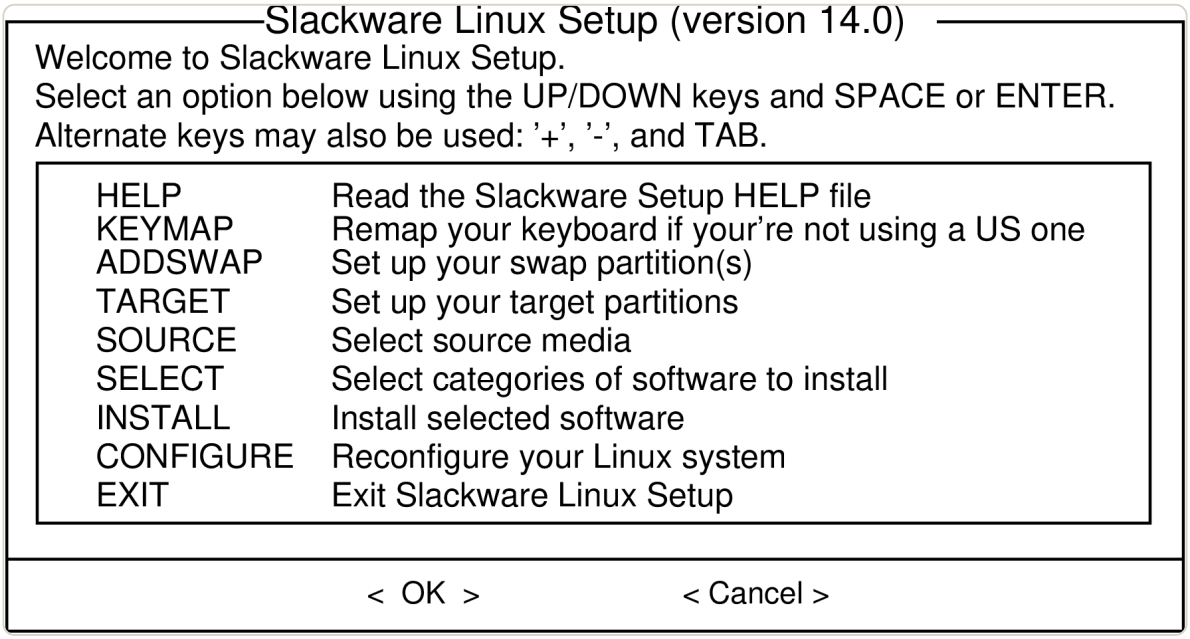
```
Command: (me for help): w
The partition table has been altered!

Calling ioctl() to re-read partition table.
Syncing disks.
root@slackware:/#
```

At this point, we are done partitioning our disks and are ready to begin the `setup` program. However, if you have created any extended partitions, you may wish to reboot once to ensure that they are properly read by the kernel.

2.3 The setup Program

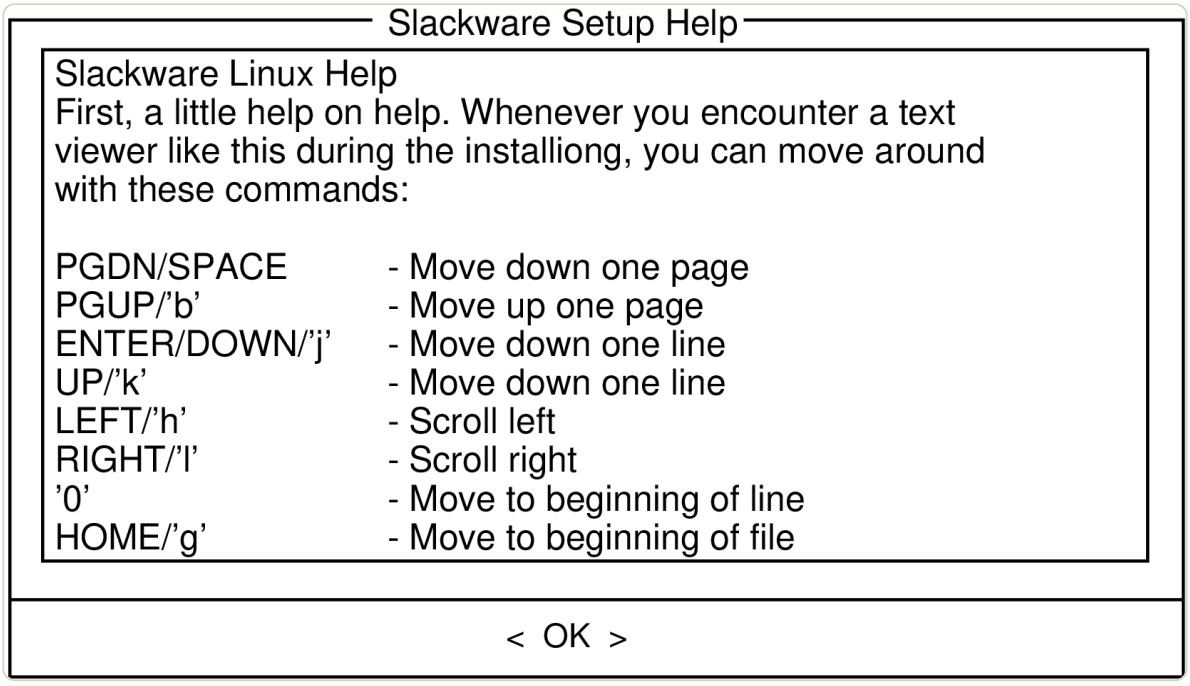
Now that you've created your partitions it's time to run the `setup` program to install Slackware. `setup` will handle formatting partitions, installing packages, and running basic configuration scripts step-by-step. In order to do so, just type `setup` at your shell prompt.



2.3.1 Help

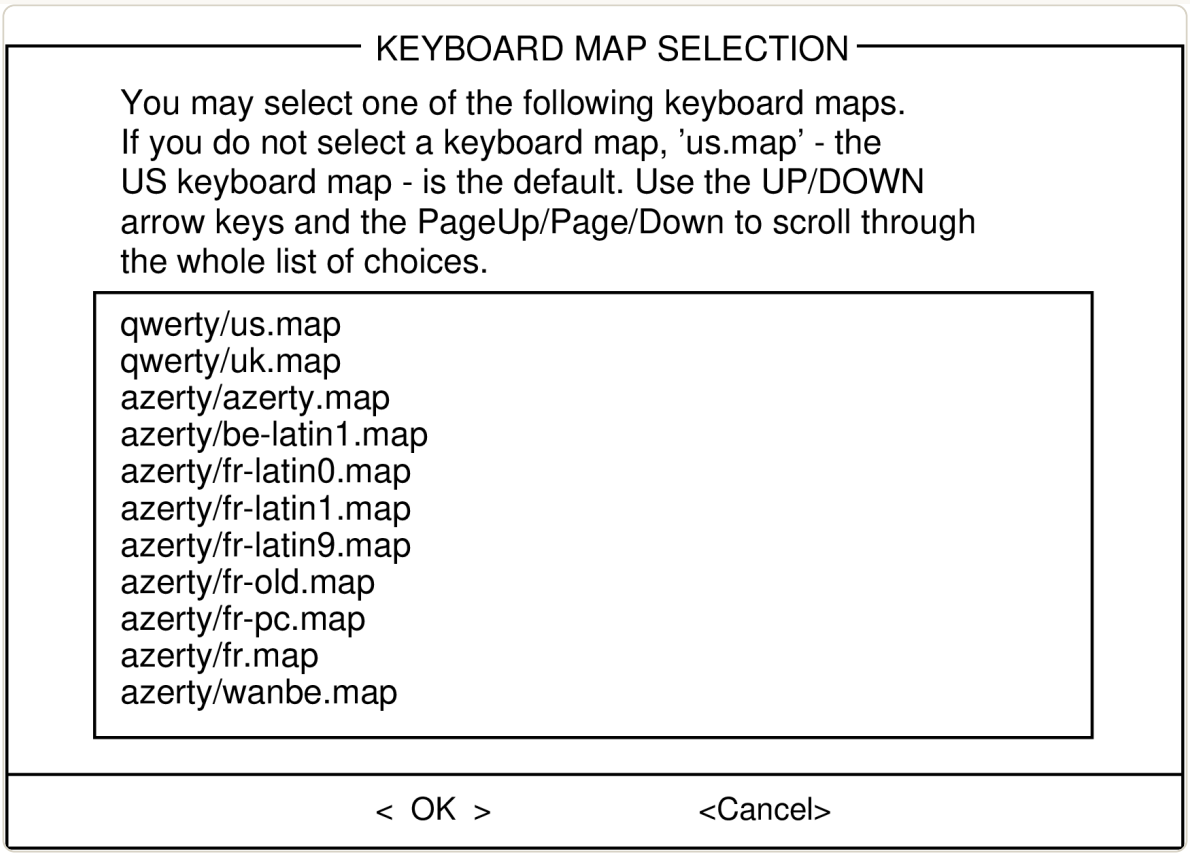
If you've never installed Slackware before, you can get a very basic over-view of the Slackware installer by reading the Help menu. Most of the information here is

on navigating through the installer which should be fairly intuitive, but if you've never used a curses-based program before you may find this useful.



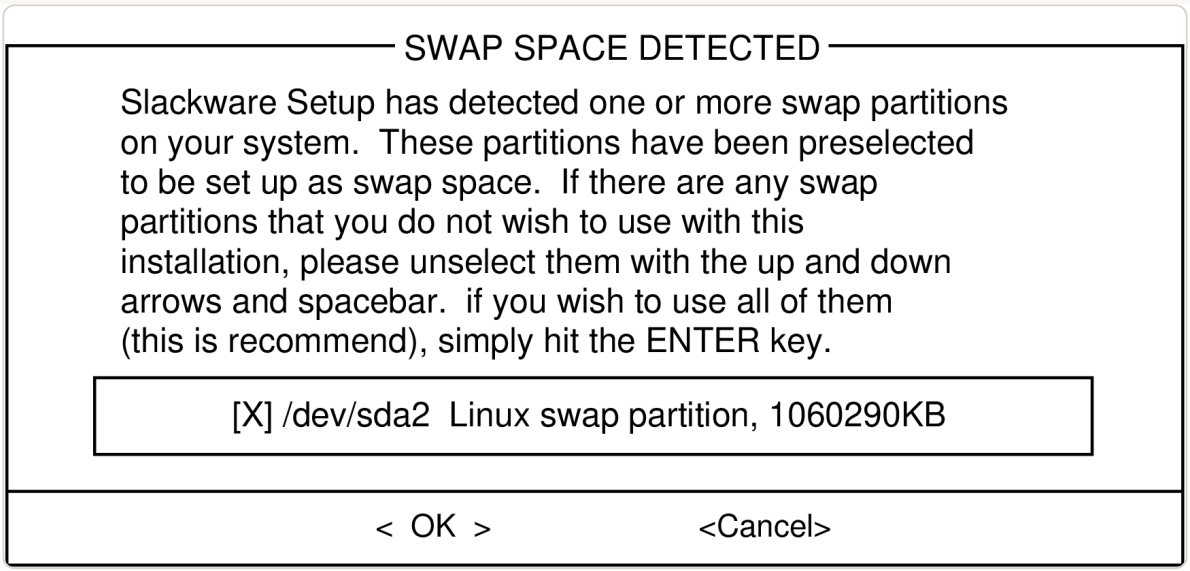
2.3.2 Keymap

Before we go any further, Slackware gives you the opportunity to select a different mapping for your keyboard. If you're using a standard US keyboard you can safely skip this step, but if you're using an international keyboard you will want to select the correct mapping now. This ensures that the keys you press on your keyboard will do exactly what you expect them to do.



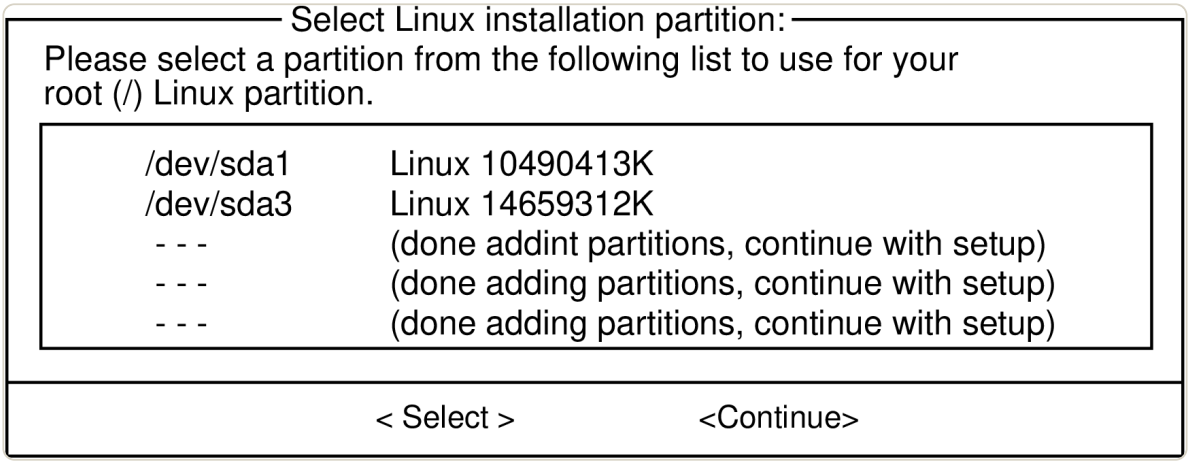
2.3.3 Addswap

If you created a swap partition, this step will allow you to enable it before running any memory-intensive activities like installing packages. swap space is essentially virtual memory. It's a hard drive partition (or a file, though Slackware's installer does not support swap files) where regions of active system memory get copied when your computer is out of useable RAM. This lets the computer "swap" programs in and out of active RAM, allowing you to use more memory than your computer actually has. This step will also add your swap partition to `/etc/fstab` so it will be available to your OS.



2.3.4 Target

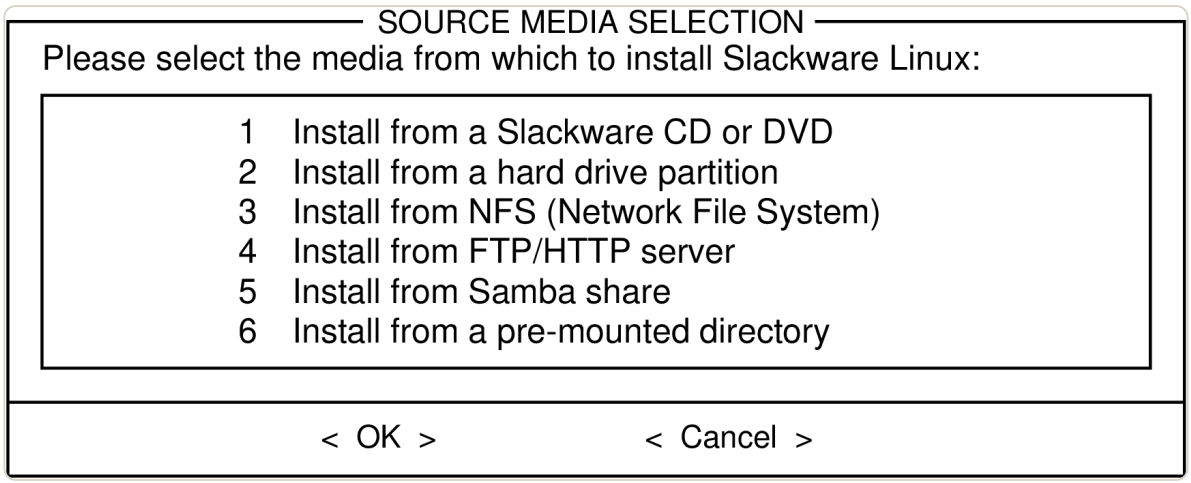
Our next step is selecting our root partition and any other partitions we'd like Slackware to utilize. You'll be given a choice of filesystems to use and whether or not to format the partition. If you're installing to a new partition you must format it. If you have a partition with data on it you'd like to save, don't. For example, many users have a seperate `/home` partition used for user data and elect not to format it on install. This lets them install newer versions of Slackware without having to backup and restore this data.



2.3.5 Source

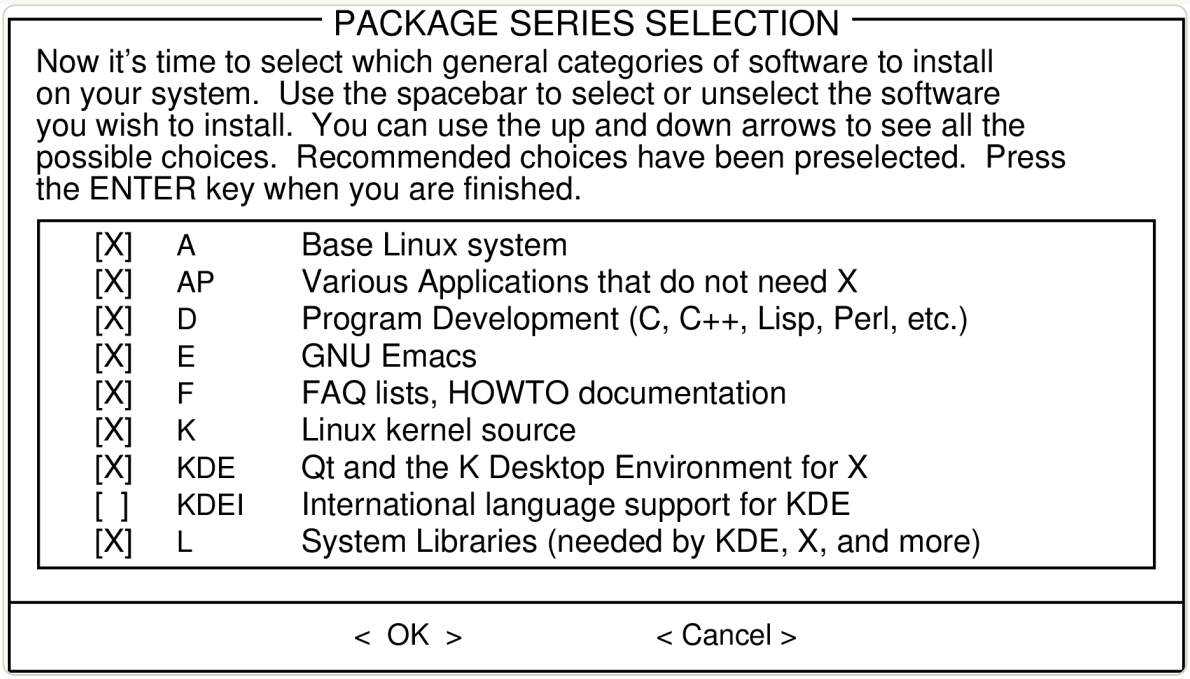
Here you'll tell the installer where to find the Slackware packages. The most common method is to use the Slackware install DVD or CDs, but various other options are available. If you have your packages installed to a partition that you setup in the previous step, you can install from that partition or a pre-mounted

directory. (You may need to mount that partition with `mount` (8) first. See chapter 11 for more details.) Additionally, Slackware offers a variety of networked options such as NFS shares, FTP, HTTP, and Samba. If you select a network installation, Slackware will prompt you for TCP/IP information first. We're only going to discuss installation from the DVD, but other methods are similar and straightforward.



2.3.6 Select

One unique feature of Slackware is its manner of dividing packages into disksets. At the beginning of time, network access to FTP servers was available only through incredibly slow 300 baud modems, so Slackware was split into disk sets that would fit onto floppy disks so users could download and install only those packages they were interested in. Today that practice continues and the installer allows you to chose which sets to install. This allows you to easily skip packages you may not want, such as X and KDE on headless servers or Emacs on everything. Please note that the "A" series is always required.



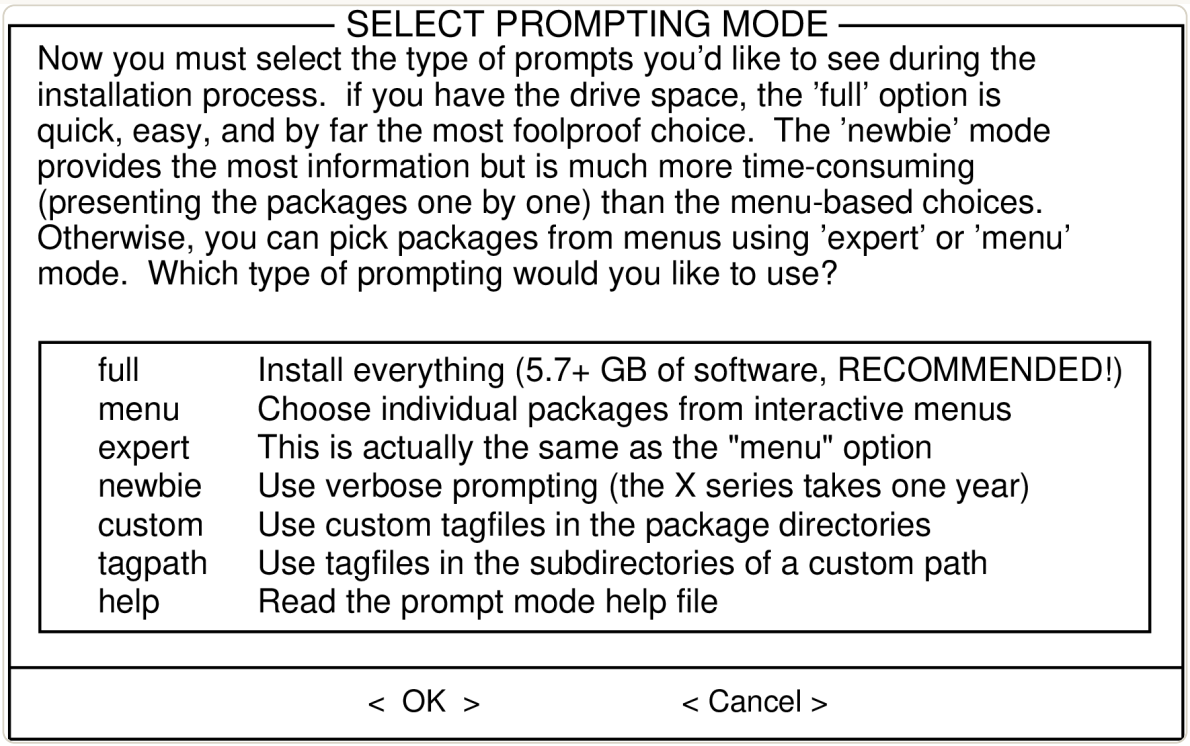
2.3.7 Install

Finally we get to the meat of the installer. At this stage, Slackware will ask you what method to use to chose packages. If this is your first time installing Slackware, the "full" method is highly recommended. Even if this isn't your first time, you'll probably want to use it anyway.

The "menu" and "expert" options allow you to choose individual packages to install and are of use to skilled users familiar with the OS. These methods allow such users to quickly prune packages from the installer to build a very minimal system. If you don't know what you're doing (sometimes even if you do) you're likely to leave out crucial pieces of software and end up with a broken system.

The "newbie" method can be very helpful to a new user, but takes a very long time to install. This method will install all the required packages, then prompt you individually for every other package. The big advantage here is that is pauses and gives you a brief overview of the package contents. For a new user, this introduction into what is included with Slackware can be informative. For most other users it is a long and tedious process.

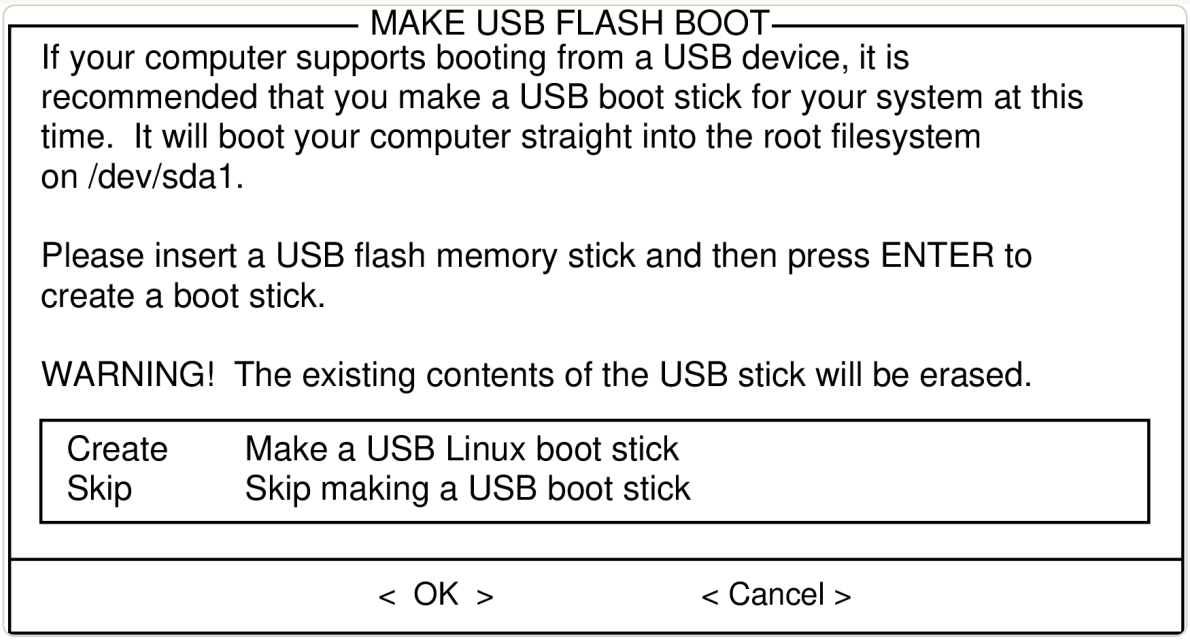
The "custom" and "tagpath" options should only be used by people with the greatest skill and expertise with Slackware. These methods allow the user to install packages from custom tagfiles. Tagfiles are only rarely used. We won't discuss them in this book.



2.3.8 Configure

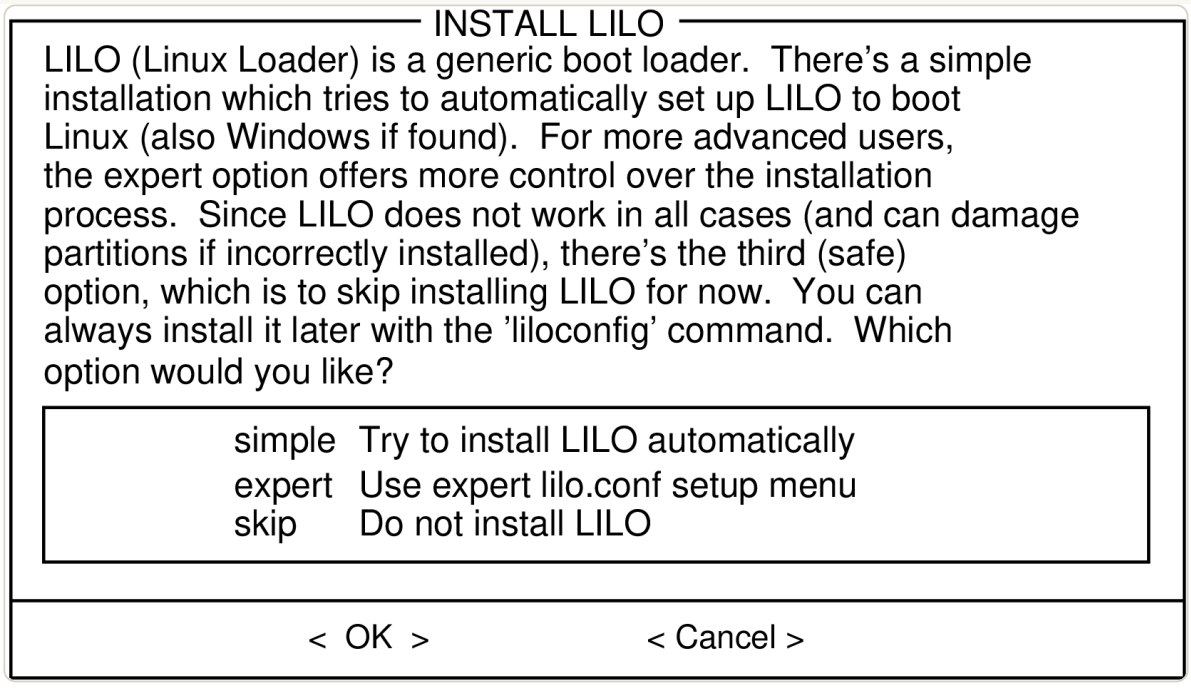
Once all the packages are installed you're nearly finished. At this stage, Slackware will prompt you with a variety of configuration tasks for your new operating system. Many of these are optional, but most users will need to set something up here. Depending on the packages you've installed, you may be offered different configuration options than the ones shown here, but we've included all the really important ones.

The first thing you'll likely be prompted to do is setup a boot disk. In the past this was typically a 1.44MB floppy disk, but today's Linux kernel is far too large to fit on a single floppy, so Slackware offers to create a bootable USB flash memory stick. Of course, your computer must support booting from USB in order to use a USB boot stick (most modern computers do). If you do not intend to use LILO or another traditional boot loader, you should consider making a USB boot stick. Please note that doing so will erase the contents of whatever memory stick you're using, so be careful.

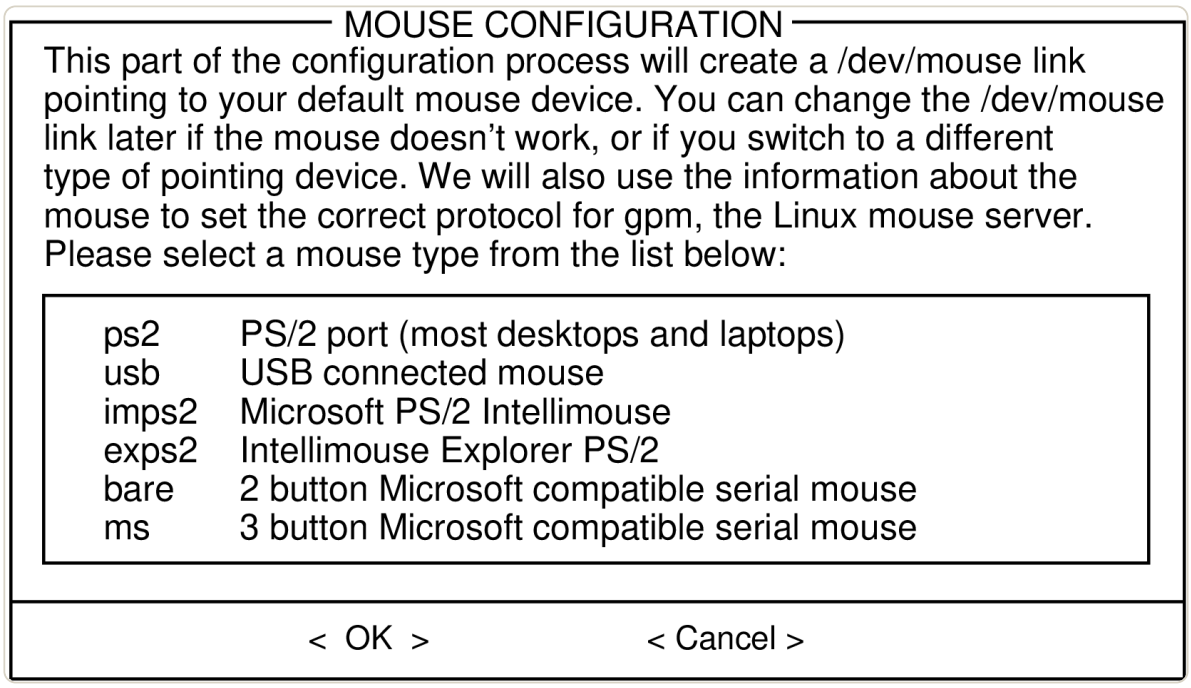


Nearly everyone will need to setup the LInux LOader, LILO. LILO is in charge of booting the Linux kernel and connecting to an initrd or the root filesystem. Without it (or some other boot loader), your new Slackware operating system will not boot. Slackware offers a few options here. The "simple" method attempts to automatically configure LILO for your computer, and works well with very simple systems. If Slackware is the only operating system on your computer, it should configure and install LILO for you without any hassels. If you don't trust the simpler method to work, or if you want to take an in-depth look at how to configure LILO, the "expert" method is really not all that complicated. This method will take you through each step and offer to setup dual-boot for Windows and other Linux operating systems. It also allows you to append kernel command parameters (most users will not need to specify any though).

LILO is a very important part of your Slackware system, so an entire section of the next chapter is devoted to it. If you're having difficulty configuring LILO at this stage, you may want to skip ahead and read Chapter 3 first, then return here.



This simple step allows you to configure and activate a console mouse for use outside of the graphical desktops. By activating a console mouse, you'll be able to easily copy and paste from within the Slackware terminal. Most users will need to choose one of the first three options, but many are offered, and yes those ancient two-button serial mice do work.



The next stage in configuring your install is the network configuration. If you don't wish to configure your network at this stage, you may decline, but otherwise you'll be prompted to provide a hostname for your computer. If you're unsure what to do here, you might want to read through first.

The following screens will prompt you first for a hostname, then for a domainname, such as example.org. The combination of the hostname and the domainname can be used to navigate between computers in your network if you use an internal DNS service or maintain your `/etc/hosts` file. If you skip setting up your network, Slackware will name your computer "darkstar" after a song by the Grateful Dead.

You have three options when setting your IP address; you may assign it a static IP, use DHCP, or configure a loopback connection. The simplest option, and probably the most common for laptops or computers on a basic network, is to let a DHCP server assign IP addresses dynamically. Unless you are installing Slackware for use as a network server, you probably do not need to setup a static IP address. If you're not sure which of these options to choose, pick DHCP.

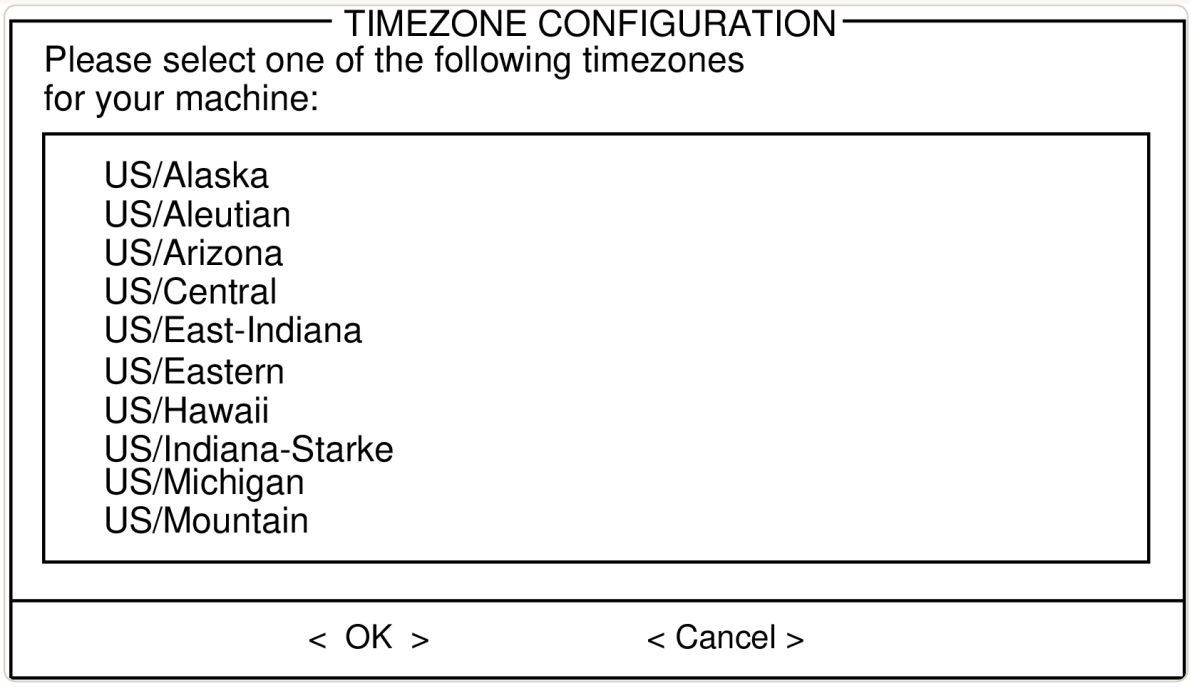
Rarely DHCP servers requires you specify a DHCP hostname before you're permitted to connect. You can enter this on the Set DHCP Hostname screen. This is almost always be the same hostname you entered earlier.

If you choose to set a static IP address, Slackware will ask you to enter it along with the netmask, gateway IP address, and what nameserver to use.

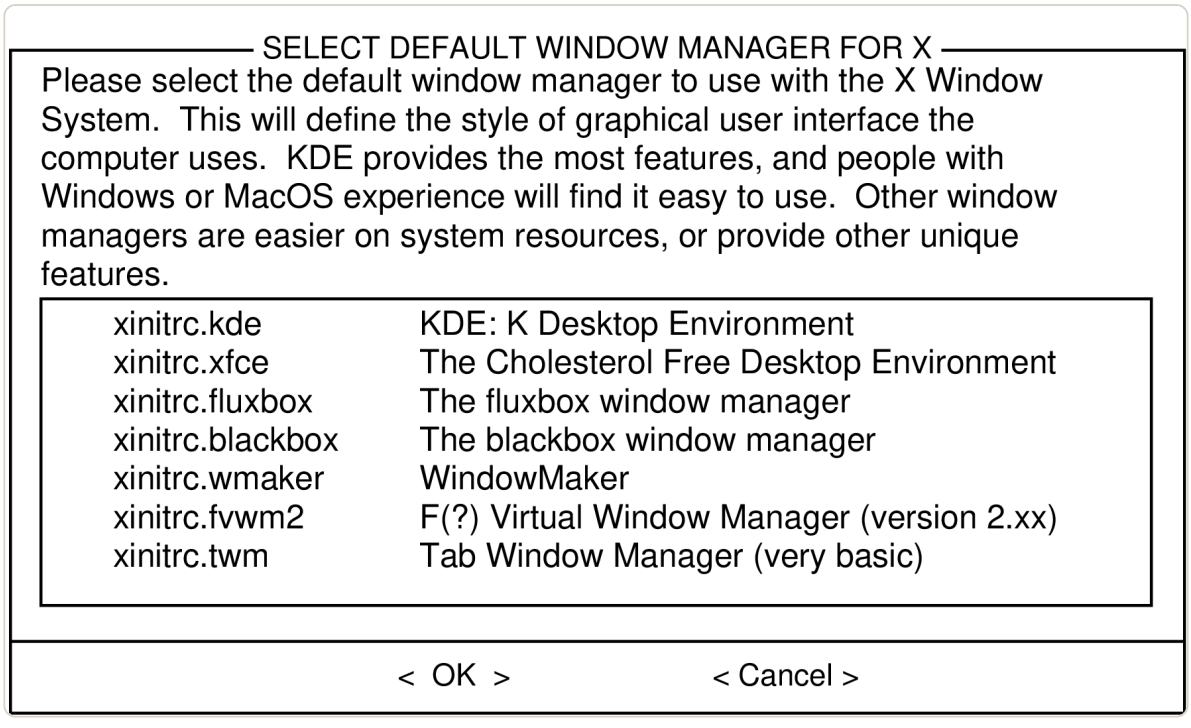
The final screen during static IP address configuration is a confirmation screen, where you're permitted to accept your choices, edit them, or even restart the IP address configuration in case you decide to use DHCP instead.

Once your network configuration is completed Slackware will prompt you to configure the startup services that you wish to run automatically upon boot. Helpful descriptions of each service appear both to the right of the service name as well as at the bottom of the screen. If you're not sure what to turn on, you can safely leave the defaults in place. What services are started at boot time can be easily modified later with `pkgtool` .

Every computer needs to keep track of the current time, and with so many timezones around the world you have to tell Slackware which one to use. If your computer's hardware clock is set to UTC (Coordinated Universal Time), you'll need to select that; most hardware clocks are not set to UTC from the factory (though you could set it that way on your own; Slackware doesn't care). Then simply select your timezone from the list provided and off you go.



If you installed the X disk set, you'll be prompted to select a default window manager or desktop environment. What you select here will apply to every user on your computer, unless that user decides to run `xwmconfig` (1) and choose a different one. Don't be alarmed if the options you see below do not match the ones Slackware offers you. `xwmconfig` only offers choices that you installed. So for example, if you elected to skip the "KDE" disk set, KDE will not be offered.



The last configuration step is setting a root password. The root user is the "super user" on Slackware and all other UNIX-like operating systems. Think of root as the

Administrator user. root knows all, sees all, and can do all, so setting a strong root password is just common sense.

With this last step complete, you can now exit the Slackware installer and reboot with a good old `CTRL + ALT + DELETE` . Remove the Slackware installation disk, and if you performed all the steps correctly, your computer will boot into your new Slackware linux system. If something went wrong, you probably skipped the LILO configuration step or made an error there somehow. Thankfully, the next chapter should help you sort that out.

When you have rebooted into your new Slackware installation, the very first step you should take is to create a user. By default, the only user that exists after the install is the root user, and it's dangerous to use your computer as root, given that there are no restrictions as to what that user can do.

The quickest and easiest way to create a normal user for yourself is to log in as root with the root password that you created at the end of the intallation process, and then issue the `adduser` command. This will interactively assist you in creating a user; see the section called “Managing Users and Groups” for more information.

3. Booting

Ok, now that you've gotten Slackware installed on your system, you should learn exactly what controls the boot sequence of your machine, and how to fix it should you manage to break it somehow. If you use Linux long enough, sooner or later you will make a mistake that breaks your bootloader. Fortunately, this doesn't require a reinstall to fix. Unlike many other operating systems that hide the underlying details of how they work, Linux (and in particular, Slackware) gives you full control over the boot process. Simply by editing a configuration file or two and re-running the bootloader installer, you can quickly and easily change (or break) your system. Slackware even makes it easy to dual-boot multiple operating systems, such as other Linux distributions or Microsoft Windows.

3.1 mkinitrd

Before we go any further, a quick discussion on the Linux kernel is warranted. Slackware Linux includes at least two, but sometimes more, different kernels. While they are all compiled from the same source code, and hence are the "same", they are not identical. Depending on your architecture and Slackware version, the installer may have loaded your system with several kernels. There are kernels for single-processor systems and kernels for multi-processor systems (on 32bit Slackware). In the old days, there were lots of kernels for installing on many different kinds of hard drive controllers. More importantly for our discussion, there are "huge" kernels and "generic" kernels.

If you look inside your `/boot` directory, you'll see the various kernels installed on your system.

```
darkstar:~# ls -1 /boot/vmlinuz*

/boot/vmlinuz-huge-2.6.29.4
/boot/vmlinuz-generic-2.6.29.4
```

Here you can see that I have two kernels installed, `vmlinuz-huge-2.6.29.4` and `vmlinuz-generic-2.6.29.4` . Each Slackware release includes different kernel versions and sometimes even slightly different names, so don't be alarmed if what you see doesn't exactly match what I have listed here.

Huge kernels are exactly what you might think; they're huge. However, that does NOT mean that they have all of the possible drivers and such compiled into them. Instead, these kernels are made to boot (and run) on every conceivable computer on which Slackware is supported (there may very well be a few out there that won't boot/work with them though). They most certainly contain support for hardware your machine does not (and never will) have, but that shouldn't concern you. These kernels are included for several reasons, but probably the most important is their use by Slackware's installer - these are the kernels that the Slackware installation disks run. If you chose to let the installer configure your bootloader for you, it chooses to use these kernels due to the incredible variety of hardware they

support. In contrast, the generic kernels support very little hardware without the use of external modules. If you want to use one of the generic kernels, you'll need to make use of something called an initrd, which is created using the `mkinitrd` (8) utility.

So why should you use a generic kernel? Currently the Slackware development team recommends use of a generic kernel for a variety of reasons. Perhaps the most obvious is size. The huge kernels are currently about twice the size of the generic kernels before they are uncompressed and loaded into memory. If you are running an older machine, or one with some small ammount of RAM, you will appreciate the savings the generic kernels offer you. Other reasons are somewhat more difficult to quantify. Conflicts between drivers included in the huge kernels do appear from time to time, and generally speaking, the huge kernels may not perform as well as the generic ones. Also, by using the generic kernels, special arguments can be passed to hardware drivers seperately, rather than requiring these options be passed on the kernel command line. Some of the tools included with Slackware work better if your kernel uses some drivers as modules rather than statically building them into the kernel. If you're having trouble understanding this, don't be alarmed: just think "huge kernel = good, generic kernel = better".

Unfortunately, using the generic kernels isn't as straightforward as using the huge kernels. In order for the generic kernel to boot your system, you must also include a few basic modules in an initird. Modules are pieces of compiled kernel code that can be inserted or removed from a running kernel (ideally using `modprobe` (8). This makes the system somewhat more flexible at the cost of a tiny bit of added complexity. You might find it easier to think of modules as device drivers, at least for this section. Typically you will need to add the module for whatever filesystem you chose to use for your root partition during the installer, and if your root partition is located on a SCSI disk or RAID controller, you'll need to add those modules as well. Finally, if you're using software RAID, disk encryption, or LVM, you'll also need to create an initrd regardless of whether you're using the generic kernel or not.

An initrd is a compressed `cpio` (1) archive, so creating one isn't very straightforward. Fortunately for you, Slackware includes a tool that makes this very easy: `mkinitrd` . A full discussion of `mkinitrd` is a bit beyond the scope of this book, but we'll show you all the highlights. For a more complete explanation, check the manpage or run `mkinitrd` with the `--help` argument.

```
darkstar:~# mkinitrd --help
mkinitrd creates an initial ramdisk (actually an initramfs cpio+gzip
archive) used to load kernel modules that are needed to mount the
root filesystem, or other modules that might be needed before the
root filesystem is available. Other binaries may be added to the
initrd, and the script is easy to modify. Be creative. :-)
.... many more lines deleted ....
```

When using `mkinitrd`, you'll need to know a few items of information: your root partition, your root filesystem, any hard disk controllers you're using, and whether or not you're using LVM, software RAID, or disk encryption. Unless you're using some kind of SCSI controller (and have your root partition located on the SCSI controller), you should only need to know your root filesystem and partition type. Assuming you've booted into your Slackware installation using the huge kernel, you can easily find this information with the `mount` command or by viewing the contents of `/proc/mounts`.

```
darkstar:~# mount
/dev/sda1 on / type ext4 (rw,barrier=1,data=ordered)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
usbfs on /proc/bus/usb type usbfs (rw)
/dev/sda2 on /home type jfs (rw)
tmpfs on /dev/shm type tmpfs (rw)
```

In the example provided, you can see that the root partition is located on `/dev/sda1` and is an ext4 type partition. If we want to create an initrd for this system, we simply need to tell this information to `mkinitrd`.

```
darkstar:~# mkinitrd -f ext4 -r /dev/sda1
```

Note that in most cases, `mkinitrd` is smart enough to determine this information on its own, but it never hurts to specify it manually. Now that we've created our initrd, we simply need to tell LILO where to find it. We'll focus on that in the next section.

Looking up all those different options for `mkinitrd` or worse, memorizing them, can be a real pain though, especially if you try out different kernels consistently. This became tedious for the Slackware development team, so they came up with a simple configuration file, `mkinitrd.conf` (5). You can find a sample file that can be easily customized for your system at `/etc/mkinitrd.conf.sample` directory. Here's mine.

```
darkstar:~# >/prompt>cat /etc/mkinitrd.conf.sample
'# See "man mkinitrd.conf" for details on the syntax of this file #
SOURCE_TREE="/boot/initrd-tree"
CLEAR_TREE="0"
OUTPUT_IMAGE="/boot/initrd.gz"
KERNEL_VERSION="$(uname -r)"
#KEYMAP="us"
MODULE_LIST="ext3:ext4:jfs"
#LUKSDEV="/dev/hda1"
ROOTDEV="/dev/sda1
ROOTFS="ext4"
#RESUMEDEV="/dev/hda2"
#RAID="0"
LVM="1"
#WAIT="1"
```

For a complete description of each of these lines and what they do, you'll need to consult the man page for `mkinitrd.conf` . Copy the sample file to to `/etc/`

`mkinitrd.conf` and edit it as desired. Once it is setup properly, you need only run `mkinitrd` with the `-F` argument. A proper initrd file will be constructed and installed for you without you having to remember all those obscure arguments.

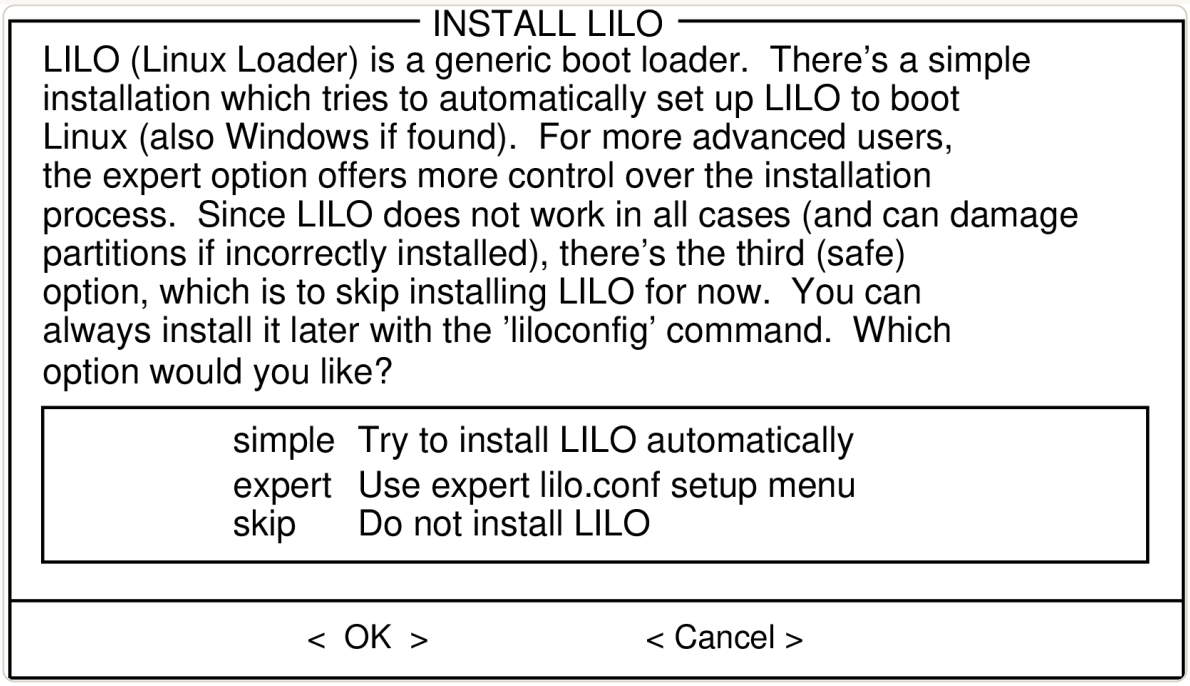
If you're unsure what options to specify in the configuration file or on the command-line, there is one final option. Slackware includes a nifty little utility that can tell what options are required for your currently running kernel `/usr/share/mkinitrd/mkinitrd_command_generator.sh` . When you run this script, it will generate a command line for `mkinitrd` that should work for your computer, but you may wish to check everything anyway.

```
darkstar:~# /usr/share/mkinitrd/mkinitrd_command_generator.sh mkinitrd
```

3.2 LILO

LILO is the Linux Loader and is currently the default boot loader installed with Slackware Linux. If you've used other Linux distributions before, you may be more familiar with GRUB. If you prefer to use GRUB instead, you can easily find it in the `extra/` directory on one of your Slackware CDs. However, since LILO is the default Slackware bootloader, we'll focus exclusively on it.

Configuring LILO can be a little daunting for new users, so Slackware comes with a special setup tool called `liloconfig` . Normally, `liloconfig` is first run by the installer, but you can run it at any time from a terminal.



`liloconfig` has two modes of operation: simple and expert. The "simple" mode tries to automatically configure lilo for you. If Slackware is the only operating system installed on your computer, the "simple" mode will almost always do the right thing quickly and easily. It is also very good at detecting Windows installations and adding them to `/etc/lilo.conf` so that you can choose which operating system to boot when you turn your computer on.

In order to use "expert" mode, you'll need to know Slackware's root partition. You can also setup other linux operating systems if you know their root partitions, but this may not work as well as you expect. `liloconfig` will try to boot each linux operating system with Slackware's kernel, and this is probably not what you want. Fortunately, setting up Windows partitions in expert mode is trivial. One hint when using expert mode: you should almost always install LILO to the Master Boot Record (MBR). Once upon a time, it was recommended to install the boot loader onto the root partition and set that partition as bootable. Today, LILO has matured greatly and is safe to install on the MBR. In fact, you will encounter fewer problems if you do so.

`liloconfig` is a great way to quickly setup your boot loader, but if you really need to know what's going on, you'll need to look at LILO's configuration file: `lilo.conf` (5) under the `/etc` directory. `/etc/lilo.conf` is separated into several sections. At the top, you'll find a "global" section where you specify things like where to install LILO (generally the MBR), any special images or screens to

show on boot, and the timeout after which LILO will boot the default operating system. Here's what the global section of my lilo.conf file looks like in part.

```
#LILO configuration file

boot = /dev/sda
    bitmap = /boot/slack.bmp
    bmp-colors = 255,0,255,0,255,0
    bmp-table = 60,6,1,16
    bmp-timer = 65,27,0,255

append=" vt.default_utf8=0"
prompt
timeout = 50

#VESA framebuffer console @ 1024x768x256
vga = 773
.... many more lines ommitted ....
```

For a complete listing of all the possible LILO options, you should consult the man page for `lilo.conf` . We'll briefly discuss the most common options in this document.

The first thing that should draw your attention is the "boot" line. This determines where the bootloader is installed. In order to install to the Master Boot Record (MBR) of your hard drive, you simply list the hard drive's device entry on this line. In my case, I'm using a SATA hard drive that shows up as a SCSI device `/dev/sda` . In order to install to the boot block of a partition, you'll have to list the partition's device entry. For example, if you are installing to the first partition on the only SATA hard drive in your computer, you would probably use `/dev/sda1` .

The "prompt" option simply tells LILO to ask (prompt) you for which operating system to boot. Operating systems are each listed in their own section deeper in the file. We'll get to them in a minute. The timeout option tells LILO how long to wait (in tenths of seconds) before booting the default OS. In my case, this is 5 seconds.

Some systems seem to take a very long time to display the boot screen, so you may need to use a larger timeout value than I have set. This is in part why the simple LILO installation method utilizes a very long timeout (somewhere around 2 whole minutes). The append line in my case was set up by `liloconfig` . You may (and probably should) see something similar when looking at your own `/etc/lilo.conf` . I won't go into the details of why this line is needed, so you're just going to have to trust me that things work better if it is present. :^)

Now that we've looked into the global section, let's take a look at the operating systems section. Each linux operating system section begins with an "image" line. Microsoft Windows operating systems are specified with an "other" line. Let's take a look at a sample `/etc/lilo.conf` that boots both Slackware and Microsoft Windows.

```
#LILO configuration file
... global section ommitted ....
#Linux bootable partition config begins
image = /boot/vmlinuz-generic-2.6.29.4
    root = /dev/sda1
    initrd = /boot/initrd.gz
    label = Slackware64
    read-only
#Linux bootable partition config ends
#Windows bootable partition config begins
other = /dev/sda3
    label = Windows
    table = /dev/sda
#Windows bootable partition config ends
```

For Linux operating systems like Slackware, the image line specifies which kernel to boot. In this case, we're booting `/boot/vmlinuz-generic-2.6.29.4` . The remaining sections are pretty self-explanatory. They tell LILO where to find the root filesystem, what initrd (if any) to use, and to initially mount the root filesystem read-only. That initrd line is very important for anyone running a generic kernel or using

LVM or software RAID. It tells LILO (and the kernel) where to find the initrd you created using `mkinitrd` .

Once you've gotten `/etc/lilo.conf` set up for your machine, simply run `lilo` (8) to install it. Unlike GRUB and other bootloaders, LILO requires you re-run `lilo` anytime you make changes to its configuration file, or else the new (changed) bootloader image will not be installed, and those changes will not be reflected.

```
darkstar:~# lilo
Warning: LBA32 addressing assumed
Added Slackware *
Added Backup
6 warnings were issued.
```

Don't be too scared by many of the warnings you may see when running `lilo` . Unless you see a fatal error, things should be just fine. In particular, the LBA32 addressing warning is commonplace.

3.3 Dual Booting

A bootloader (like LILO) is a very flexible thing, since it exists only to determine which hard drive, partition, or even a specific kernel on a partition to boot. This inherently suggests a choice when booting, so the idea of having more than one operating system on a computer comes very naturally to a LILO or GRUB user.

People "dual boot" for a number of reasons; some people want to have a stable Slackware install on one partition or drive and a development sandbox on another, other people might want to have Slackware on one and another Linux or BSD distribution on another, and still other people may have Slackware on one partition and a proprietary operating system (for work or for that one application that Linux simply cannot offer) on the other.

Dual booting should not be taken lightly, however, since it usually means that you'll now have two different operating systems attempting to manage the bootloader. If

you dual boot, the likelihood of one OS over-writing or updating the bootloader entries without your direct intervention is great; if this happens, you'll have to modify GRUB or LILO manually so you can get into each OS.

There are two ways to dual (or multi) boot; you can put each operating system on its own hard drive (common on a desktop, with their luxury of having more than one drive bay) or each operating system on its own partition (common on a laptop where only one physical drive is present).

3.3.1 Dual Booting with Partitions

In order to set up a dual-boot system with each operating system on its own partition, you must first create partitions. This is easiest if done prior to installing the first operating system, in which case it's a simple case of pre-planning and carving up your hard drive as you feel necessary. See the section called “Partitioning” for information on using the `fdisk` or `cfdisk` partitioning applications.

> **Important** If you're dual booting two Linux distributions, it is inadvisable to attempt to share a /home directory between the systems. While it is technically possible, doing so will increase the chance of your personal configurations from becoming mauled by competing desktop environments or versions. It is, however, safe to use a common swap partition.

You should partition your drive into at least three parts:

- One partition for Slackware
- One partition for the secondary OS
- One partition for swap

First, install Slackware Linux onto the first partition of the hard drive as described in Chapter, Installation.

After Slackware has been installed, booted, and you've confirmed that everything works as expected, then reboot to the installer for the second OS. This OS will invariably offer to utilize the entire drive; you obviously do *not* want to do that, so constrain it to only the second partition. Furthermore, the OS will attempt to install a boot loader to the beginning of the hard drive, overwriting LILO.

You have a few possible courses of action with regards to the boot loader:

Possible Boot Loader Scenarios

If the secondary OS is Linux, disallow it from installing a boot manager.

If you're dual booting to another Linux distribution, the installer of that distribution usually asks if you want a boot loader installed. You're certainly free to not install a boot manager for it at all, and manually manage both Slackware and the other distribution with LILO.

Depending on the distribution, you might be editing LILO more frequently than you would if you were only running Slackware; some distributions are notorious for frequent kernel updates, meaning that you'll need to edit LILO to reflect the new configuration after such an update. But if you didn't want to edit configuration files every now and again, you probably wouldn't have chosen Slackware.

If the secondary OS is Linux, let it overwrite LILO with GRUB.

If you're dual booting to another Linux distribution, you are perfectly capable of just using GRUB rather than LILO, or install Slackware last and use LILO for both. Both LILO and GRUB have very good auto-detection features, so whichever one gets installed last should pick up the other distribution's presence and make an entry for it.

Since other distributions often attempt to auto-update their GRUB menus, there is always the chance that during an update something will become maligned and you suddenly find you can't boot into Slackware. If this happens, don't panic; just boot into the other Linux partition and manually edit GRUB so that it points to the correct partition, kernel, and initrd (if applicable) for Slackware in its menu.

Allow the secondary OS to overwrite LILO and go back later to manually re-install and re-configure LILO.

This is not a bad choice, especially when Windows is the secondary OS, but potential pitfalls are that when Windows updates itself, it may attempt to overwrite the MBR (master boot record) again, and you'll have to re-install LILO manually again.

To re-establish LILO after another OS has erased it, you can boot from your Slackware install media and enter the setup stage. Do *not* re-partition your drive or re-install Slackware; skip immediately to the section called “Configure”.

Even when using the "simple" option to install, LILO should detect both operating systems and automatically configure a sensible menu for you. If it fails, then add the entries yourself.

3.3.2 Dual Booting from Hard Drives

Dual booting between different physical hard drives is often easier than with partitions since the computer's BIOS or EFI almost invariably has a boot device chooser that allows you to interrupt the boot process immediately after POST and choose what drive should get priority.

The snag key to enter the boot picker is different for each brand of motherboard; consult the motherboard's manual or read the splash screen to find out what your computer requires. Typical keys are `F1` , `F12` , `DEL` . For Apple computers, it is always the `Option` (Alt) key.

If you manage the boot priority via BIOS or EFI, then each boot loader on each hard drive is only aware of its own drive and will never interfere with one another. This is rather contrary to what a boot loader is designed to do but can be a useful workaround when dealing with proprietary operating systems which insist upon being the only OS on the system, to the detriment of the user's preference.

If you don't have the luxury of having multiple internal hard drives and don't feel comfortable juggling another partition and OS on your computer, you might also consider using a bootable USB thumbdrive or even a virtual machine to give you access to another OS. Both of these options is outside the scope of this book, but they've commonplace and might be the right choice for you, depending on your needs.

4. Basic Shell Commands

So you've installed Slackware and you're staring at a terminal prompt, what now? Now would be a good time to learn about the basic command line tools. And since you're staring at a blinking curser, you probably need a little assistance in knowing how to get around, and that is what this chapter is all about.

4.1 System Documentation

Your Slackware Linux system comes with lots of built-in documentation for nearly every installed application. Perhaps the most common method of reading system documentation is `man` (1). `man` (short for *manual*) will bring up the included man-page for any application, system call, configuration file, or library you tell it too. For example, `man man` will bring up the man-page for `man` itself.

Unfortunately, you may not always know what application you need to use for the task at hand. Thankfully, `man` has built-in search abilities. Using the `-k` switch will cause `man` to search for every man-page that matches your search terms.

The man-pages are organized into groups or sections by their content type. For example, section 1 is for user applications. `man` will search each section in order and display the first match it finds. Sometimes you will find that a man-page exists in more than one section for a given entry. In that case, you will need to specify the exact section to look in. In this book, all applications and a number of other things will have a number on their right-hand side in parenthesis. This number is the man page section where you will find information on that tool.

```
darkstar:~$ man -k printf
printf (1) - format and print data
printf (3) - formatted output conversion
darkstar:~$ man 3 printf
```

Section	Contents
1	User Commands
2	System Calls
3	C Library Calls
4	Devices
5	File Formats / Protocols
6	Games
7	Conventions / Macro Packages
8	System Administration
9	Kernel API Descriptions
n	"New" - typically used to Tcl/Tk

4.2 Dealing with Files and Directories

4.2.1 Listing Files and Directory Contents

`ls` (1) is used to list files and directories, their permissions, size, type, inode number, owner and group, and plenty of additional information. For example, let's list what's in the `/` directory for your new Slackware Linux system.

```
darkstar:~$ ls /
bin/ dev/ home/ lost+found/ mnt/ proc/ sbin/ sys/ usr/
boot/ etc/ lib/ media/ opt/ root/ srv/ tmp/ var/
```

Notice that each of the listings is a directory. These are easily distinguished from regular files due to the trailing /; standard files do not have a suffix. Additionally,

executable files will have an asterisk suffix. But `ls` can do so much more. To get a view of the permissions of a file or directory, you'll need to do a "long list".

```
darkstar:~$ ls -l /home/alan/Desktop
-rw-r--r-- 1 alan users 15624161 2007-09-21 13:02 9780596510480.pdf
-rw-r--r-- 1 alan users  3829534 2007-09-14 12:56 imgscan.zip
drwxr-xr-x 3 alan root       168 2007-09-17 21:01 ipod_hack/
drwxr-xr-x 2 alan users      200 2007-12-03 22:11 libgpod/
drwxr-xr-x 2 alan users      136 2007-09-30 03:16 playground/
```

A long listing lets you view the permisions, user and group ownership, file size, last modified date, and of course, the file name itself. Notice that the first two entires are files, and the last three are directories. This is denoted by the very first character on the line. Regular files get a "-"; directories get a "d". There are several other file types with their own denominators. Symbolic links for example will have an "l".

Lastly, we'll show you how to list dot-files, or hidden files. Unlike other operating systems such as Microsoft Windows, there is no special property that differentiates "hidden" files from "unhidden" files. A hidden file simply begins with a dot. To display these files along with all the others, you just need to pass the `-a` argument to `ls` .

```
darkstar:~$ ls -a
.xine/ .xinitrc-backup .xscreensaver .xsession-errors SBo/
.xinitrc .xinitrc-xfce .xsession .xwmconfig/ Shared/
```

You also likely noticed that your files and directories appear in different colors. Many of the enhanced features of `ls` such as these colors or the trailing characters indicating file-type are special features of the `ls` program that are turned on by passing various arguments. As a convienience, Slackware sets up `ls` to use many of these optional arguments by default. These are controlled by

the LS_OPTIONS and LS_COLORS environment variables. We will talk more about environment variables in chapter 5.

4.2.2 Moving Around the Filesystem

`cd` is the command used to change directories. Unlike most other commands, `cd` is actually not it's own program, but is a shell built-in. Basically, that means `cd` does not have its own man page. You'll have to check your shell's documentation for more details on the `cd` you may be using. For the most part though, they all behave the same.

```
darkstar:~$ cd /
darkstar:/$ls
bin/ dev/ home/ lost+found/ mnt/ proc/ sbin/ sys/ usr/
boot/ etc/ lib/ media/ opt/ root/ srv/ tmp/ var/
darkstar:/$cd /usr/local
darkstar:/usr/local$
```

Notice how the prompt changed when we changed directories? The default Slackware shell does this as a quick, easy way to see your current directory, but this is not actually a function of `cd` . If your shell doesn't operate in this way, you can easily get your current working directory with the `pwd` (1) command. (Most UNIX shells have configurable prompts that can be coaxed into providing this same functionality. In fact, this is another convience setup in the default shell for you by Slackware.)

```
darkstar:~$ pwd
/usr/local
```

4.2.3 File and Directory Creation and Deletion

While most applications can and will create their own files and directories, you'll often want to do this on your own. Thankfully, it's very easy using `touch` (1) and `mkdir` (1).

`touch` actually modifies the timestamp on a file, but if that file doesn't exist, it will be created.

```
darkstar:~/foo$ ls -l
-rw-r--r-- 1 alan users 0 2012-01-18 15:01 bar1
darkstar:~/foo$ touch bar2
-rw-r--r-- 1 alan users 0 2012-01-18 15:01 bar1
-rw-r--r-- 1 alan users 0 2012-01-18 15:05 bar2
darkstar:~/foo$ touch bar1
-rw-r--r-- 1 alan users 0 2012-01-18 15:05 bar1
-rw-r--r-- 1 alan users 0 2012-01-18 15:05 bar2
```

Note how `bar2` was created in our second command, and the third command simpl updated the timestamp on `bar1`

`mkdir` is used for (obviously enough) making directories. `mkdir foo` will create the directory "foo" within the current working directory. Additionally, you can use the `-p` argument to create any missing parent directories.

```
darkstar:~$ mkdir foo
darkstar:~$ mkdir /slack/foo/bar/
mkdir: cannot create directory `/slack/foo/bar/': No such file or direc
darkstar:~$ mkdir -p /slack/foo/bar/
```

In the latter case, `mkdir` will first create "/slack", then "/slack/foo", and finally "/slack/foo/bar". If you failed to use the `-p` argument, `man` would fail to create "/slack/foo/bar" unless the first two already existed, as you saw in the example.

Removing a file is as easy as creating one. The `rm` (1) command will remove a file (assuming of course that you have permission to do this). There are a few very common arguments to `rm` . The first is `-f` and is used to force the removal of a file that you may lack explicit permission to delete. The `-r` argument will remove directories and their contents recursively.

There is another tool to remove directories, the humble `rmdir` (1). `rmdir` will only remove directories that are empty, and complain noisely about those that contain files or sub-directories.

```
darkstar:~$ ls
foo_1/ foo_2/
darkstar:~$ ls foo_1
bar_1
darkstar:~$ rmdir foo_1
rmdir: foo/: Directory not empty
darkstar:~$ rm foo_1/bar
darkstar:~$ rmdir foo_1
darkstar:~$ ls foo_2
bar_2/
darkstar:~$ rm -fr foo_2
darkstar:~$ ls
```

4.3 Archive and Compression

Everyone needs to package a lot of small files together for easy storage from time to time, or perhaps you need to compress very large files into a more manageable size? Maybe you want to do both of those together? Thankfully there are several tools to do just that.

4.3.1 zip and unzip

You're probably familiar with .zip files. These are compressed files that contain other files and directories. While we don't normally use these files in the Linux world, they are still commonly used by other operating systems, so we occasionally have to deal with them.

In order to create a zip file, you'll (naturally) use the `zip` (1) command. You can compress either files or directories (or both) with `zip` , but you'll have to use the `-r` argument for recursive action in order to deal with directories.

```
darkstar:~$ zip -r /tmp/home.zip /home
darkstar:~$ zip /tmp/large_file.zip /tmp/large_file
```

The order of the arguments is very important. The first filename must be the zip file to create (if the .zip extension is ommitted, `zip` will add it for you) and the rest are files or directories to be added to the zip file.

Naturally, `unzip` (1) will decompress a zip archive file.

```
darkstar:~$ unzip /tmp/home.zip
```

4.3.2 gzip

One of the oldest compression tools included in Slackware is `gzip` (1), a compression tool that is only capable or operating on a single file at a time. Whereas `zip` is both a compression and an archival tool, `gzip` is only capable of compression. At first glance this seems like a draw-back, but it is really a strength. The UNIX philosophy of making small tools that do their small jobs well allows them to be combined in myriad ways. In order to compress a file (or multiple files), simply pass them as arguments to `gzip` . Whenever `gzip` compresses a file, it adds a .gz extension and removes the original file.

```
darkstar:~$ gzip /tmp/large_file
```

Decompressing is just as straight-forward with `gunzip` which will create a new uncompressed file and delete the old one.

```
darkstar:~$ gunzip /tmp/large_file.gz
darkstar:~$ ls /tmp/large_file*
/tmp/large_file
```

But suppose we don't want to delete the old compressed file, we just want to read its contents or send them as input to another program? The `zcat` program will read the gzip file, decompress it in memory, and send the contents to the standard output (the terminal screen unless it is redirected, see for more details on output redirection).

```
darkstar:~$ zcat /tmp/large_file.gz Wed Aug 26 10:00:38 CDT 2009 Slackw
```

4.3.3 bzip2

One alternative to `gzip` is the `bzip2` (1) compression utility which works in almost the exact same way. The advantage to `bzip2` is that it boasts greater compression strength. Unfortunately, achieving that greater compression is a slow and CPU-intensive process, so `bzip2` typicall takes much longer to run than other alternatives.

4.3.4 XZ / LZMA

The latest compression utility added to Slackware is `xz` , which impliments the LZMA compression algorithm. This is faster than `bzip2` and often compresses

better as well. In fact, its blend of speed and compression strength caused it to replace `gzip` as the compression scheme of choice for Slackware. Unfortuantely, `xz` does not have a man page at the time of this writing, so to view available options, use the `--help` argument. Compressing files is accomplished with the `-z` argument, and decompression with `-d` .

```
darkstar:~$ xz -z /tmp/large_file
```

4.3.5 tar

So great, we know how to compress files using all sorts of programs, but none of them can archive files in the way that `zip` does. That is until now. The Tape Archiver, or `tar` (1) is the most frequently used archival program in Slackware. Like other archival programs, `tar` generates a new file that contains other files and directories. It does not compress the generated file (often called a "tarball") by default; however, the version of `tar` included in Slackware supports a variety of compression schemes, including the ones mentioned above.

Invoking `tar` can be as easy or as complicated as you like. Typically, creating a tarball is done with the `-cvzf` arguments. Let's look at these in depth.

Argument	**Meaning**
c	Create a tarball
x	Extract the contents of a tarball
t	Display the contents of a tarball
v	Be more verbose
z	Use gzip compression
j	Use bzip2 compression
J	Use LZMA compression

Argument	Meaning
p	Preserve permissions

`tar` requires a bit more precision than other applications in the order of its arguments. The `-f` argument must be present when reading or writing to a file for example, and the very next thing to follow must be the filename. Consider the following examples.

```
darkstar:~$ tar -xvzf /tmp/tarball.tar.gz
darkstar:~$ tar -xvfz /tmp/tarball.tar.gz
```

Above, the first example works as you would expect, but the second fails because `tar` has been instructed to open the `z` file rather than the expected `/tmp/tarball.tar.gz` .

Now that we've got our arguments straightened out, lets look at a few examples of how to create and extract tarballs. As we've noted, the `-c` argument is used to create tarballs and `-x` extracts their contents. If we want to create or extract a compressed tarball though, we also have to specify the proper compression to use. Naturally, if we don't want to compress the tarball at all, we can leave these options out. The following command creates a new tarball using the `gzip` compression alogrithm. While it's not a strict requirement, it's also good practice to add the .tar extension to all tarballs as well as whatever extension is used by the compression algorithm.

```
darkstar:~$ tar -czf /tmp/tarball.tar.gz /tmp/tarball/
```

4.4 Reading Documents

Traditionally, UNIX and UNIX-like operating systems are filled with text files that at some point in time the system's users are going to want to read. Naturally, there are plenty of ways of reading these files, and we'll show you the most common ones.

In the early days, if you just wanted to see the contents of a file (any file, whether it was a text file or some binary program) you would use `cat` (1) to view them. `cat` is a very simple program, which takes one or more files, concatenates them (hence the name) and sends them to the standard output, which is usually your terminal screen. This was fine when the file was small and wouldn't scroll off the screen, but inadequate for larger files as it had no built-in way of moving within a document and reading it a paragraph at a time. Today, `cat` is still used quite extensively, but predominately in scripts or for joining two or more files into one.

```
darkstar:~$ cat /etc/slackware-version
Slackware 14.0
```

Given the limitations of `cat` some very intelligent people sat down and began to work on an application to let them read documents one page at a time. Naturally, such applications began to be known as "pagers". One of the earliest of these was `more` (1), named because it would let you see "more" of the file whenever you wanted.

4.4.1 more

`more` will display the first few lines of a text file until your screen is full, then pause. Once you've read through that screen, you can proceed down one line by pressing ENTER, or an entire screen by pressing SPACE , or by a specified number of lines by typing a number and then the SPACE bar. `more` is also capable of searching through a text file for keywords; once you've displayed a file in `more` , press the / key and enter a keyword. Upon pressing ENTER , the text will scroll until it finds the next match.

This is clearly a big improvement over `cat` , but still suffers from some annoying flaws; `more` is not able to scroll back up through a piped file to allow you to read something you might have missed, the search function does not highlight its results, there is no horizontal scrolling, and so on. Clearly a better solution is possible.

Note

In fact, modern versions of `more` , such as the one shipped with Slackware, do feature a *back* function via the b key. However, the function is only available when opening files directly in `more` ; not when a file is piped to `more` .

4.4.2 less

In order to address the short-comings of `more` , a new pager was developed and ironically dubbed `less` (1). `less` is a very powerful pager that supports all of the functions of `more` while adding lots of additional features. To begin with, `less` allows you to use your arrow keys to control movement within the document.

Due to its popularity, many Linux distributions have begun to exclude `more` in favor of `less` . Slackware includes both. Moreover, Slackware also includes a handy little pre-processor for `less` called `lesspipe.sh` . This allows a user to exectute `less` on a number of non-text files. `lesspipe.sh` will generate text output from running a command on these files, and display it in `less` .

`Less` provides nearly as much functionality as one might expect from a text editor without actually being a text editor. Movement line-by-line can be done `vi` -style with `j` and `k` , or with the arrow keys, or `ENTER` . In the event that a file is too wide to fit on one screen, you can even scroll horizontally with the left and right arrow keys. The `g` key takes you to the top of the file, while `G` takes you to the end.

Searching is done as with `more` , by typing the / key and then your search string, but notice how the search results are highlighted for you, and typing `n` will take you to the next occurence of the result while `N` takes you to the previous occurrence.

Also as with `more` , files may be opened directly in `less` or piped to it:

```
darkstar:~$ less
/usr/doc/less-*/README
darkstar:~$ cat
/usr/doc/less*/README
/usr/doc/util-linux*/README | less
```

There is much more to `less` ; from within the application, type h for a full list of commands.

4.5 Linking

Links are a method of referring to one file by more than one name. By using the `ln` (1) application, a user can reference one file with more than one name. The two files are not carbon-copies of one another, but rather are the exact same file, just with a different name. To remove the file entirely, all of its names must be deleted. (This is actually the result of the way that `rm` and other tools like it work. Rather than remove the contents of the file, they simply remove the reference to the file, freeing that space to be re-used. `ln` will create a second reference or "link" to that file.)

```
darkstar:~$ ln /etc/slackware-version foo
darkstar:~$ cat foo
Slackware 14.0
darkstar:~$ ls -l /etc/slackware-version foo
-rw-r--r-- 1 root root 17 2007-06-10 02:23 /etc/slackware-version
-rw-r--r-- 1 root root 17 2007-06-10 02:23 foo
```

Another type of link exists, the symlink. Symlinks, rather than being another reference to the same file, are actually a special kind of file in their own right. These symlinks point to another file or directory. The primary advantage of symlinks is that

they can refer to directories as well as files, and they can span multiple filesystems. These are created with the `-s` argument.

```
darkstar:~$ ln -s /etc/slackware-version foo
darkstar:~$ cat foo
Slackware 14.0
darkstar:~$ ls -l /etc/slackware-version foo
-rw-r--r-- 1 root root 17 2007-06-10 02:23 /etc/slackware-version
lrwxrwxrwx 1 root root 22 2008-01-25 04:16 foo -> /etc/slackware-versio
```

When using symlinks, remember that if the original file is deleted, your symlink is useless; it simply points at a file that doesn't exist anymore.

5. The Bourne Again Shell

5.1 What Is A Shell?

Yeah, what exactly is a shell? Well, a shell is basically a command-line user environment. In essence, it is an application that runs when the user logs in and allows him to run additional applications. In some ways it is very similar to a graphical user interface, in that it provides a framework for executing commands and launching programs. There are many shells included with a full install of Slackware, but in this book we're only going to discuss `bash` (1), the Bourne Again Shell. Advanced users might want to consider using the powerful `zsh` (1), and users familiar with older UNIX systems might appreciate `ksh` . The truly masochistic might choose the `csh` , but new users should stick to `bash` .

5.2 Environment Variables

All shells make certain tasks easier for the user by keeping track of things in environment variables. An environment variable is simply a shorter name for some bit of information that the user wishes to store and make use of later. For example, the environment variable PS1 tells `bash` how to format its prompt. Other variables may tell applications how to run. For example, the LESSOPEN variable tells `less` to run that handy `lesspipe.sh` preprocessor we talked about, and LS_OPTIONS tuns on color for `ls` .

Setting your own envirtonment variables is easy. `bash` includes two built-in functions for handling this: `set` and `export` . Additionally, an environment variable can be removed by using `unset` . (Don't panic if you accidently unset an environment variable and don't know what it would do. You can reset all the default variables by logging out of your terminal and logging back in.) You can reference a variable by placing a dollar sign ($) in front of it.

```
darkstar:~$ set FOO=bar
darkstar:~$ echo $FOO bar
```

The primary difference between `set` and `export` is that `export` will (naturally) export the variable to any sub-shells. (A sub-shell is simply another shell running inside a parent shell.) You can easily see this behavior when working with the PS1 variable that controls the `bash` prompt.

```
darkstar:~$ set PS1='FOO '
darkstar:~$ export PS1='FOO ' FOO
```

There are many important environment variables that `bash` and other shells use, but one of the most important ones you will run across is PATH. PATH is simply a list of directories to search through for applications. For example, `top` (1) is located at `/usr/bin/top` . You could run it simply by specifying the complete

path to it, but if `/usr/bin` is in your PATH variable, `bash` will check there if you don't specify a complete path one your own. You will most likely first notice this when you attempt to run a program that is not in your PATH as a normal user, for instance, `ifconfig` (8).

```
darkstar:~$ ifconfig
bash: ifconfig: command not found
darkstar:~$ echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/X11R6/bin:/usr/games:/opt/www/htdig/b
```

Above, you see a typical PATH for a mortal user. You can change it on your own the same as any other environment variable. If you login as root however, you'll see that root has a different PATH.

```
darkstar:~$ su -
Password:
darkstar:~# echo $PATH
/usr/local/sbin:/usr/sbin:/sbin:/usr/local/bin:/usr/bin:/bin:/usr/X11R6
```

5.3 Wildcards

Wildcards are special characters that tell the shell to match certain criteria. If you have experience with DOS, you'll recognize * as a wildcard that matches anything. `bash` makes use of this wildcard and several others to enable you to easily define exactly what you want to do.

This first and most common of these is, of course, *. The asterisk matches any character or combination of characters, including none. Thus `b*` would match any files named b, ba, bab, babc, bcdb, and so forth. Slightly less common is the ?. This wildcard matches one instance of any character, so `b?` would match ba and bb, but not b or bab.

```
darkstar:~$ touch b ba bab
darkstar:~$ ls *
b ba bab
darkstar:~$ ls b?
ba
```

No, the fun doesn't stop there! In addition to these two we also have the bracket pair "[ ]" which allows us to fine tune exactly what we want to match. Whenever `bash` see the bracket pair, it substitutes the contents of the bracket. Any combination of letters or numbers may be specified in the bracket as long as they are comma seperated. Additionally, ranges of numbers and letters may be specified as well. This is probably best shown by example.

```
darkstar:~$ ls a[1-4,9]
a1 a2 a3 a4 a9
```

Since Linux is case-sensitive, capital and lower-case letters are treated differently. All capital letters come before all lower-case letters in "alphabetical" order, so when using ranges of capital and lower-case letters, make sure to get them right.

```
darkstar:~$ ls 1[W-b]
1W 1X 1Y 1Z 1a 1b
darkstar:~$ ls 1[w-B]
/bin/ls: cannot access 1[b-W]: No such file or directory
```

In the second example, 1[b-W] isn't a valid range, so the shell treats it as a filename, and since that file doesn't exist, `ls` tells you so.

5.4 Tab Completion

Still think there's entirely too much work involved with using wildcards? You're right. There's an even easier way when you're dealing with long filenames: tab completion. Tab completion enables you to type just enough of the filename to uniquely identify it, then by hitting the TAB key, `bash` will fill in the rest for you. Even if you haven't typed in enough text to uniquely identify a filename, the shell will fill in as much as it can for you. Hitting TAB a second time will make it display a list of all possible matches for you.

5.5 Input and Output Redirection

One of the defining features of Linux and other UNIX-like operating systems is the number of small, relatively simple applications and the ability to stack them together to create complex systems. This is achieved by redirecting the output of one program to another, or by drawing input from a file or second program.

To get started, we're going to show you how to redirect the output of a program to a file. This is easily done with the '>' character. When `bash` sees the '>' character, it redirects all of the standard output (also known as stdout) to whatever file name follows.

```
darkstar:~$ echo foo
foo
darkstar:~$ echo foo > /tmp/bar
darkstar:~$ cat /tmp/bar
foo
```

In this example, we show you what `echo` would do if its stdout was not redirected to a file, then we re-direct it to the `/tmp/bar` file. If `/tmp/bar` does not exist, it is created and the output from `echo` is placed within it. If `/tmp/bar` did exist, then its contents are over-written. This might not be the best idea if you want to keep those contents in place. Thankfully, `bash` supports '>>' which will append the output to the file.

```
darkstar:~$ echo foo
foo
darkstar:~$ echo foo > /tmp/bar
darkstar:~$ cat /tmp/bar
foo
darkstar:~$ echo foo2 >> /tmp/bar
darkstar:~$ cat /tmp/bar
foo
foo2
```

You can also re-direct the standard error (or stderr) to a file. This is slightly different in that you must use '2>' instead of just '>'. (Since `bash` can re-direct input, stdout, and stderr, each must be uniquely identifiable. 0 is input, 1 is stdout, and 2 is stderr. Unless one of these is specified, `bash` will make its best guess as to what you actually meant, and assumed anytime you use '>' you only want to redirect stdout. 1> would have worked just as well.)

```
darkstar:~$ rm bar
rm: cannot remove `bar': No such file or directory
darkstar:~$ rm bar 2> /tmp/foo
darkstar:~$ cat /tmp/foo
rm: cannot remove `bar': No such file or directory
```

You may also redirect the standard input (known as stdin) with the '<' character, though it's not used very often.

```
darkstar:~$ fromdos < dosfile
```

Finally, you can actually redirect the output of one program as input to another. This is perhaps the most useful feature of `bash` and other shells, and is accomplished using the '|' character. (This character is referred to as 'pipe'. If you here some one talk of piping one program to another, this is exactly what they mean.)

```
darkstar:~$ ps auxw | grep getty
root      2632  0.0  0.0   1656   532 tty2     Ss+  Feb21   0:00 /sbin/
root      3199  0.0  0.0   1656   528 tty3     Ss+  Feb15   0:00 /sbin/
root      3200  0.0  0.0   1656   532 tty4     Ss+  Feb15   0:00 /sbin/
root      3201  0.0  0.0   1656   532 tty5     Ss+  Feb15   0:00 /sbin/
root      3202  0.0  0.0   1660   536 tty6     Ss+  Feb15   0:00 /sbin/
```

5.6 Task Management

`bash` has yet another cool feature to offer, the ability to suspend and resume tasks. This allows you to temporarily halt a running process, perform some other task, then resume it or optionally make it run in the background. Upon pressing `CTRL-Z` , `bash` will suspend the running process and return you to a prompt. You can return to that process later. Additionally, you can suspend multiple processes in this way indefinitely. The `jobs` built-in command will display a list of suspended tasks.

```
darkstar:~$ jobs
[1]- Stopped vi TODO
[2]+ Stopped vi chapter_05.xml
```

In order to return to a suspended task, run the `fg` built-in to bring the the most recently suspended task back into the foreground. If you have mutiple suspended tasks, you can specify a number as well to bring one of them to the foreground.

```
darkstar:~$ fg # "vi TODO"
darkstar:~$ fg 1 # "vi chapter_05.xml"
```

You can also background a task with (surprize) `bg` . This will allow the process to continue running without maintaining control of your shell. You can bring it back to the foreground with `fg` in the same way as suspended tasks.

5.7 Terminals

Slackware Linux and other UNIX-like operating systems allow users to interact with them in many ways, but the most common, and arguably the most useful, is the terminal. In the old days, terminals were keyboards and monitors (sometimes even mice) wired into a mainframe or server via serial connections. Today however, most terminals are virtual; that is, they exist only in software. Virtual terminals allow users to connect to the computer without requiring expensive and often incompatible hardware. Rather, a user needs only to run the software and they are presented with a (usually) highly customizable virtual terminal.

The most common virtual terminals (in that every Slackware Linux machine is going to have at least one) are the gettys. `agetty` (8) runs six instances by default on Slackware, and allows local users (those who can physically sit down in front of the computer and type at the keyboard) to login and run applications. Each of these gettys is available on different tty devices that are accessible seperately by pressing the ALT key and one of the function keys from F1 through F6 . Using these gettys allows you to login multiple times, perhaps as different users, and run applications in those users' shells silmutaneously. This is most commonly done with servers which do not have `X` installed, but can be done on any machine.

On desktops, laptops, and other workstations where the user prefers a graphical interface provided by `X` , most terminals are graphical. Slackware includes many different graphical terminals, but the most commonly used are KDE's `konsole` and XFCE's `Terminal` (1) as well as the old standby, xterm(1). If you are using a graphical interface, check your tool bars or menus. Each desktop environment or window manager has a virtual terminal (often called a terminal emulater), and they

are all labelled differently. Typically though, you will find them under a "System" sub-menu in desktop environments. Executing any of these will give you a graphical terminal and automatically run your default shell.

5.8 Customization

By now you should be pretty familiar with `bash` and you may have even noticed some odd behavior. For example, when you login at the console, you're presented with a prompt that looks a bit like this.

```
alan@darkstar:~$
```

However, sometimes you'll see a much less helpful prompt like this one.

```
bash-3.1$
```

The cause here is a special environment variable that controls the `bash` prompt. Some shells are considered "login" shells and others are "interactive" shells, and both types read different configuration files when started. Login shells read `/etc/profile` and `~/.bash_profile` when executed. Interactive shells read `~/.bashrc` instead. This has some advantages for power users, but is a common annoyance for many new users who want the same environment anytime they execute `bash` and don't care about the difference between login and interactive shells. If this applies to you, simply edit your own ~/.bashrc file and include the following lines. (For more information on the different configuration files used, read the INVOCATION section of the `bash` man page.)

```
#~/.bashrc
. /etc/profile
```

```
. ~/.bash_profile
```

When using the above, all your login and interactive shells will have the same environment settings and behave identically. Now, anytime we wish to customize a shell setting, we only have to edit `~/.bash_profile` for user-specific changes and `/etc/profile` for global settings. Let's start by configuring the prompt.

`bash` prompts come in all shapes, colors, and sizes, and every user has their own preferances. Personally, I prefer short and simple prompts that take up a minimum of space, but I've seen and used mutli-line prompts many times. One personal friend of mine even included ASCII-art in his bash prompt. To change your prompt you need only to change your PS1 variable. By default, Slackware attempts to configure your PS1 variable thusly:

```
darkstar:~$ echo $PS1
\u@\h:\w\$
```

Yes, this tiny piece of funny-looking figures controls your `bash` prompt. Basicaly, every character in the PS1 variable is included in the prompt, unless it is a escaped by a `\` , which tells `bash` to interpret it. There are many different escape sequences and we can't discuss them all, but I'll explain these. The first "\u" translates to the username of the current user. "\h" is the hostname of the machine the terminal is attached to. "\w" is the current working directory, and "\$" displays either a # or a $ sign, depending on whether or not the current user is root. A complete listing of all prompt escape sequences is listed in the `bash` man page under the PROMPTING section.

Since we've gone through all this trouble to discuss the default prompt, I thought I'd take some time to show you a couple example prompts and the PS1 variable values needed to use them.

```
Wed Jan 14 12:08 AM
```

```
alan@raven:~$ echo $PS1
\d \@\n\u@\h:\w$
HOST: raven - JOBS: 0 - TTY: 3
alan@~/Desktop/sb_3.0:$ echo $PS1
HOST: \H - JOBS: \j - TTY: \l\n\u@\w:\$
```

For even more information on configuring your bash prompt, including information on setting up colored prompts, refer to `/usr/doc/Linux-HOWTOs/Bash-Prompt-HOWTO` . After reading that for a short while, you'll get an idea of just how powerful your `bash` prompts can be. I once even had a prompt that gave me up to date weather information such as temperature and barometric pressure!

6. Process Control

Slackware systems often run hundreds or thousands of programs, each of which is referred to as a process. Managing these processes is an important part of system administration. So how exactly do we handle all of these seperate processes?

6.1 ps

The first step in managing processes is figuring out what processes are currently running. The most popular and powerful tool for this is `ps` (1). Without any arguments, `ps` won't tell you much information. By default, it only tells you what processes are running in your currently active shell. If we want more information, we'll need to look deeper.

```
darkstar:~$ ps
  PID TTY          TIME CMD
12220 pts/4    00:00:00 bash
```

```
12236 pts/4     00:00:00 ps
```

Here you can see what processes you are running in your currently active shell or terminal and only some information is included. The PID is the "Process ID"; every process is assigned a unique number. The TTY tells you what terminal device the process is attached to. Naturally, CMD is the command that was run. You might be a little confused by TIME though, since it seems to move so slowly. This isn't the amount of real time the process has been running, but rather the amount of CPU time the process has consumed. An idle process uses virtually no CPU time, so this value may not increase quickly.

Viewing only our own processes isn't very much fun, so let's take a look at all the processes on the system with the `-e` argument.

```
darkstar:~$ ps -e
PID TTY          TIME     CMD
  1 ?            00:00:00 init
  2 ?            00:00:00 kthreadd
  3 ?            00:00:00 migration/0
  4 ?            00:00:00 ksoftirqd/0
  7 ?            00:00:11 events/0
  9 ?            00:00:01 work_on_cpu/0
 11 ?            00:00:00 khelper
102 ?            00:00:02 kblockd/0
105 ?            00:01:19 kacpid
106 ?            00:00:01 kacpi_notify
... many more lines omitted ...
```

Note

The above example uses the standard `ps` syntax, but much more information can be discovered if we use BSD syntax. In order to do so, we must use the `aux` argument.

This is distinct from the `-aux` argument, but in most cases the two arguments are equivilant. This is a decades-old relic. For more information, see the man page for `ps` .

```
darkstar:~$ ps aux
USER    PID     %CPU    %MEM    VSZ     RSS TTY STAT START TIME COMMAND
root      1     0.0     0.0     3928    632 ?   Ss   Apr05 0:00 init [3
root      2     0.0     0.0     0       0 ?     S    Apr05 0:00 [kthrea
root      3     0.0     0.0     0       0 ?     S    Apr05 0:00 [migrat
root      4     0.0     0.0     0       0 ?     S    Apr05 0:00 [ksofti
root      7     0.0     0.0     0       0 ?     S    Apr05 0:11 [events
root      9     0.0     0.0     0       0 ?     S    Apr05 0:01 [work_c
root     11     0.0     0.0     0       0 ?     S    Apr05 0:00 [khelpe
... many more lines omitted ....
```

As you can see, BSD syntax offers much more information, including what user controls the process and what percentage of RAM and CPU the process is consuming when `ps` is run.

To accomplish bits of this, on a per process basis, `ps` allows one or more process IDs (PIDs) to be provided in the command line, and has the '-o' flag to show a particular attribute of the PID.

```
darkstar:~$ ps -o cmd -o etime $$
CMD         ELAPSED
/bin/bash   12:22
```

What this is displaying, is the PID's command name (cmd), and its elapsed time (etime). The PID in this example, is a shell variable for the PID of the current shell. So you can see, in this example, the shell process has existed for 12 minutes, 22 seconds.

Using the `pgrep` (1) command, this can get more automatable.

```
darkstar:~$ ps -o cmd -o rss -o vsz $(pgrep httpd)
CMD                          RSS   VSZ
/usr/sbin/httpd -k restart 33456 84816
/usr/sbin/httpd -k restart 33460 84716
/usr/sbin/httpd -k restart 33588 84472
/usr/sbin/httpd -k restart 30424 81608
/usr/sbin/httpd -k restart 33104 84900
/usr/sbin/httpd -k restart 33268 85112
/usr/sbin/httpd -k restart 30640 82724
/usr/sbin/httpd -k restart 15168 67396
/usr/sbin/httpd -k restart 33180 84416
/usr/sbin/httpd -k restart 33396 84592
/usr/sbin/httpd -k restart 32804 84232
```

In this example, a subshell execution, using `pgrep` , is returning the PIDs of any process, whose command name includes 'httpd'. Then `ps` displaying the command name, resident memory size, and virtual memory size.

Finally, `ps` can also create a process tree. This shows you which processes have children processes. Ending the parent of a child process also ends the child. We do this with the `-ejH` argument.

```
darkstar:~$ ps -ejH
... many lines omitted ...
 3660  3660 3660 tty1 00:00:00 bash
29947 29947 3660 tty1 00:00:00  startx
29963 29947 3660 tty1 00:00:00   xinit
29964 29964 29964 tty7 00:27:11   X
29972 29972 3660 tty1 00:00:00    sh
29977 29972 3660 tty1 00:00:05     xscreensaver
29988 29972 3660 tty1 00:00:04     xfce4-session
```

```
29997 29972 3660 tty1 00:00:16      xfwm4
29999 29972 3660 tty1 00:00:02      Thunar
... many more lines omitted ...
```

As you can see, `ps` (1) is an incredibly powerful tool for determining not only what processes are currently active on your system, but also for learning lots of important information about them.

As is the case with many of the applications, there is often several tools for the job. Similar to the `ps -ejH` output, but more terse, is `pstree` (1). It displays the process tree, a bit more visually.

```
darkstar:~$ pstree
init-+-atd
|-crond
|-dbus-daemon
|-httpd---10*[httpd]
|-inetd
|-klogd
|-mysqld_safe---mysqld---8*[{mysqld}]
|-screen-+-4*[bash]
|        |-bash---pstree
|        |-2*[bash---ssh]
|        `-bash---irssi
|-2*[sendmail]
|-ssh-agent
|-sshd---sshd---sshd---bash---screen
`-syslogd
```

6.2 kill and killall

Managing processes isn't only about knowing which ones are running, but also about communicating with them to change their behavior. The most common way

of managing a program is to terminate it. Thus, the tool for the job is named `kill` (1). Despite the name, `kill` doesn't actually terminate processes, but sends signals to them. The most common signal is a SIGTERM, which tells the process to finish up what it is doing and terminate. There are a variety of other signals that can be sent, but the three most common are SIGTERM, SIGHUP, and SIGKILL.

What a process does when it receives a signal varies. Most programs will terminate (or attempt to terminate) whenever they receive any signal, but there are a few important differences. For starters, the SIGTERM signal informs the process that it should terminate itself at its earliest convenience. This gives the process time to finish up any important activities, such as writing information to the disk, before it closes. In contrast, the SIGKILL signal tells the process to terminate itself immediately, no questions asked. This is most useful for killing processes that are not responding and is sometimes called the "silver bullet". Some processes (particularly daemons) capture the SIGHUP signal and reload their configuration files whenever they receive it.

In order to signal a process, we first need to know it's PID. You can get this easily with `ps` as we discused. In order to send different signals to a running process, you simply pass the signal number and `-s` as an argument. The `-l` argument lists all the signals you can choose and their number. You can also send signals by their name with `-s` .

```
darkstar:~$ kill -l
 1) SIGHUP   2) SIGINT   3) SIGQUIT  4) SIGILL
 5) SIGTRAP  6) SIGABRT  7) SIGBUS   8) SIGFPE
 9) SIGKILL 10) SIGUSR1 11) SIGSEGV 12) SIGUSR2
13) SIGPIPE 14) SIGALRM 15) SIGTERM 16) SIGSTKFLT
... many more lines omitted ...
darkstar:~$ kill 1234 # SIGTERM
darkstar:~$ kill -s 9 1234 # SIGKILL
darkstar:~$ kill -s 1 1234 # SIGHUP
darkstar:~$ kill -s HUP 1234 # SIGHUP
```

Sometimes you may wish to terminate all running processes with a certain name. You can kill processes by name with `killall` (1). Just pass the same arguments to `killall` that you would pass to `kill` .

```
darkstar:~$ killall bash # SIGTERM
darkstar:~$ killall -s 9 bash # SIGKILL
darkstar:~$ killall -s 1 bash # SIGHUP
darkstar:~$ killall -s HUP bash # SIGHUP
```

6.3 top

So far we've learned how to look at the active processes for a moment in time, but what if we want to monitor them for an extended period? `top` (1) allows us to do just that. It displays an ordered list of the processes on your system, along with vital information about them, and updates periodically. By default, processes are ordered by their CPU percentage and updates occur every three seconds.

```
darkstar:~$ top
top - 16:44:15 up 26 days,  5:53,  5 users,  load average: 0.08, 0.03,
Tasks: 122 total,   1 running, 119 sleeping,   0 stopped,   2 zombie
Cpu(s):  3.4%us,  0.7%sy,  0.0%ni, 95.5%id,  0.1%wa,  0.0%hi,  0.2%si,
Mem:   3058360k total,  2853780k used,   204580k free,   154956k buffe
Swap:        0k total,        0k used,        0k free,  2082652k cache

  PID USER      PR  NI  VIRT  RES  SHR S %CPU %MEM    TIME+  COMMAND
    1 root      20   0  3928  632  544 S    0  0.0   0:00.99 init
    2 root      15  -5     0    0    0 S    0  0.0   0:00.00 kthreadd
    3 root      RT  -5     0    0    0 S    0  0.0   0:00.82 migration/
    4 root      15  -5     0    0    0 S    0  0.0   0:00.01 ksoftirqd/
    7 root      15  -5     0    0    0 S    0  0.0   0:11.22 events/0
    9 root      15  -5     0    0    0 S    0  0.0   0:01.19 work_on_cp
   11 root      15  -5     0    0    0 S    0  0.0   0:00.01 khelper
```

```
  102 root      15  -5     0    0    0 S    0  0.0   0:02.04 kblockd/0
  105 root      15  -5     0    0    0 S    0  0.0   1:20.08 kacpid
  106 root      15  -5     0    0    0 S    0  0.0   0:01.92 kacpi_noti
  175 root      15  -5     0    0    0 S    0  0.0   0:00.00 ata/0
  177 root      15  -5     0    0    0 S    0  0.0   0:00.00 ata_aux
  178 root      15  -5     0    0    0 S    0  0.0   0:00.00 ksuspend_u
  184 root      15  -5     0    0    0 S    0  0.0   0:00.02 khubd
  187 root      15  -5     0    0    0 S    0  0.0   0:00.00 kseriod
  242 root      20   0     0    0    0 S    0  0.0   0:03.37 pdflush
  243 root      15  -5     0    0    0 S    0  0.0   0:02.65 kswapd0
```

The man page has helpful details on how to interact with `top` such as changing its delay interval, the order processes are displayed, and even how to terminate processes right from within `top` itself.

6.4 cron

Ok, so we've learned many different ways of viewing the active processes on our system and means of signalling them, but what if we want to run a process periodically? Fortunately, Slackware includes just the thing, `crond` (8). cron runs processes for every user on the schedule that user demands. This makes it very useful for processes that need to be run periodically, but don't require full daemonization, such as backup scripts. Every user gets their own entry in the cron database, so non-root users can periodically run processes too.

In order to run programs from cron, you'll need to use the `crontab` (1). The man page lists a variety of ways to do this, but the most common method is to pass the `-e` argument. This will lock the user's entry in the cron database (to prevent it from being overwritten by another program), then open that entry with whatever text editor is specified by the VISUAL environment variable. On Slackware systems, this is typically the `vi` editor. You may need to refer to the chapter on `vi` before continuing.

The cron database entries may seem a little archaic at first, but they are highly flexible. Each uncommented line is processed by `crond` and the command specified is run if all the time conditions match.

```
darkstar:~$ crontab -e
#Keep current with slackware
30 02 * * * /usr/local/bin/rsync-slackware64.sh 1>/dev/null 2>&1
```

As mentioned before, the syntax for cron entries is a little difficult to understand at first, so let's look at each part individually. From left to right, the different sections are: Minute, Hour, Day, Month, Week Day, and Command. Any asterisk * entry matches every minute, hour, day, and so on. So from the example above, the command is "/usr/local/bin/rsync-slackware64.sh 1>/dev/null 2>&1", and it runs every weekday or every week of every month at 2:30 a.m.

`crond` will also e-mail the local user with any output the command generates. For this reason, many tasks have their output redirected to `/dev/null` , a special device file that immediately discards everything it receives. In order to make it easier for you to remember these rules, you might wish to paste the following commented text at the top of your own cron entries.

```
#Redirect everything to /dev/null with:
#1>/dev/null 2>&1
#
#MIN HOUR DAY MONTH WEEKDAY COMMAND
```

By default, Slackware includes a number of entries and comments in root's crontab. These entries make it easier to setup periodic system tasks by creating a number of directories in `/etc` corresponding to how often the tasks should run. Any script placed within these directories will be run hourly, daily, weekly, or

monthly. The names should be self-explanatory: `/etc/cron.hourly` , `/etc/cron.daily` , `/etc/cron.weekly` , and `/etc/cron.monthly` .

7. The X Window System

7.1 What Is (And Isn't) X

Eons ago computer terminals came with a screen and a keyboard and not much else. Mice hadn't come into common use and everything was menu driven. Then came the Graphical User Interface (GUI) and the world was changed. Today users are accustomed to moving a mouse around a screen, clicking on icons and running tasks with fancy images and animation, but UNIX systems predated this and so GUIs were added almost as an afterthought. For many years, Linux and its UNIX brethren were primarily used without graphics of any sort, but today it is perhaps more common than not for users to prefer their Linux computers come with shiney, flashy, clickable GUIs, and all these GUIs run on `X` (7).

So what is X? Is it the desktop with the icons? Is it the menus? Is it the window manager? Does it mark the spot? The answer to all these is a resounding "no". There are many parts to a GUI, but X is the most fundamental. X is that application that receives input from the mouse, keyboard, and possibly other devices. X is that application that tells the graphics card what to do. In short, X is the application that talks to your computer's hardware for graphical purposes; all other graphical applications simply talk to X.

Let's stop for a moment and talk about nomenclature. X is just one of a dozen names that you may encounter. It is also called X11, xorg, the X Window System, X Window, X11R6, X Version 11, and several others. Whatever you hear it called, simply understand that the speakers are referring to X.

7.2 Configuring the X Server

Once upon a time, configuring X was a difficult and painful process that caused the magic smoke to come gushing out of hundreds of monitors. Today X is a lot more user friendly. In fact, most users will not need to configure X at all, Slackware will simply figure out all the proper settings on its own. There are, however, still some computers that X can't properly auto-configure and will need a little bit of work on your part.

Once upon a time, the X configuration file was located at `/etc/X11/xorg.conf` , and if you create a file there, X will honor whatever settings you place within it. Fortunately, with `X.Org 1.6.3` an `/etc/X11/xorg.conf` does not even need to be present for `X` to generate a working display. If for whatever reason, you need to make configuration changes to X, try to avoid using this file; it's antiquated and inflexible. Rather, the `/etc/X11/xorg.conf.d/` directory is where you should put such tweaks. Any file you place within that directory will be read when X starts up. This allows you to split-up your configuration into more easily manageable parts. For example, here's my `/etc/X11/xorg.conf.d/synaptics.conf` file for my laptop.

```
darkstar:~$ cat /etc/X11/xorg.conf.d/synaptics.conf
Section "InputDevice"
    Identifier      "Synaptics Touchpad"
    Driver          "synaptics"
    Option          "SendCoreEvents"        "true"
    Option          "Device"                "/dev/psaux"
    Option          "Protocol"              "auto-dev"
    Option          "SHMConfig"             "on"
    Option          "LeftEdge"              "100"
    Option          "RightEdge"             "1120"
    Option          "TopEdge"               "50"
    Option          "BottomEdge"            "310"
    Option          "FingerLow"             "25"
    Option          "FingerHigh"            "30"
    Option          "VertScrollDelta"       "20"
```

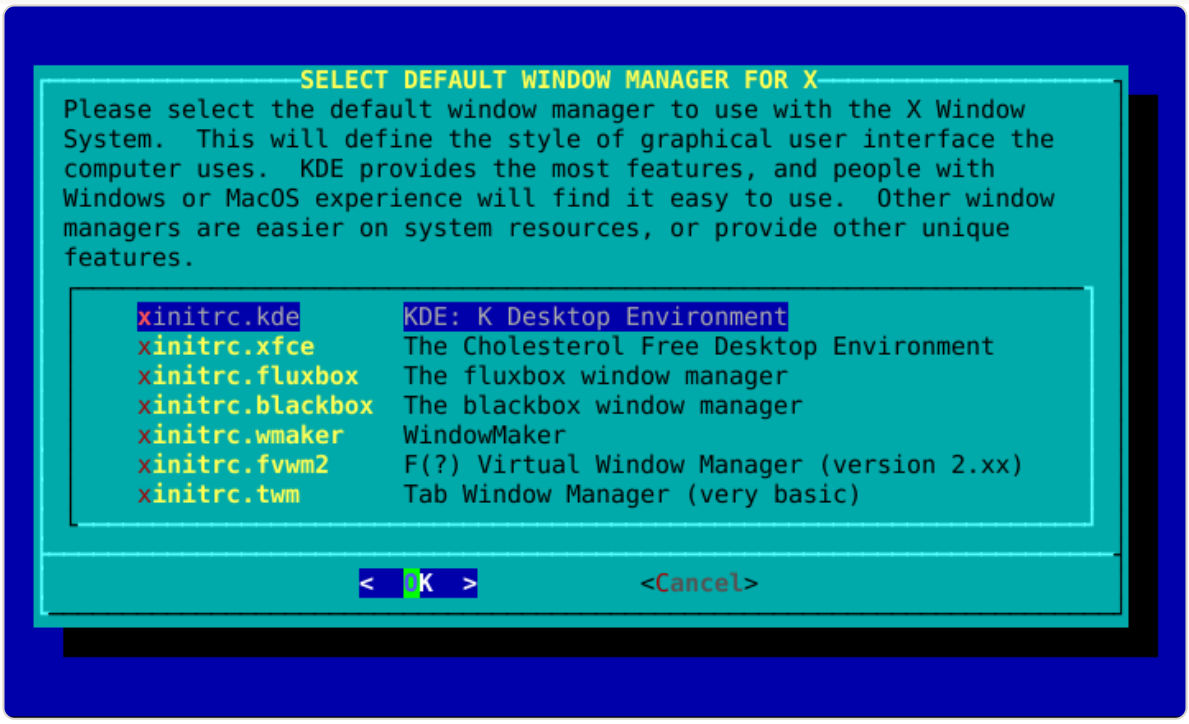
```
    Option          "HorizScrollDelta"      "50"
    Option          "MinSpeed"              "0.79"
    Option          "MaxSpeed"              "0.88"
    Option          "AccelFactor"           "0.0015"
    Option          "TapButton1"            "1"
    Option          "TapButton2"            "2"
    Option          "TapButton3"            "3"
    Option          "MaxTapMove"            "100"
    Option          "HorizScrollDelta"      "0"
    Option          "HorizEdgeScroll"       "0"
    Option          "VertEdgeScroll"        "1"
    Option          "VertTwoFingerScroll"   "0"
EndSection
```

By placing such options in individual files, you can easily manage your X configuration by sections.

7.3 Choosing a Window Manager

Slackware Linux includes many different window managers and desktop environments. Window managers are the applications responsible for painting application windows on the screen, resizing these windows, and similar tasks. Desktop environments include a window manager, but also add task bars, menus, icons, and more. Slackware includes both the KDE and XFCE desktop environments and several additional window managers. Which you use is entirely your own decision, but in general, window managers tend to be faster than desktop environments and more suitable to older systems with less memory and slower processors. Desktop environments will be more comfortable for users accustomed to Microsoft Windows.

The easiest way to choose a window manager is `xwmconfig` (1), included with Slackware Linux. This application allows a user to choose what window manager to run with `startx` .



7.4 Setting Up A Graphical Login

By default, when you boot your Slackware Linux system you are presented with a login prompt on a virtual terminal. This is more than adequate for most people's needs. If you need to run commandline applications, you may login and do so right away. If you want to run X, simply executing `startx` will do that for you nicely. But suppose you almost exclusively use your system for graphical duties like many laptop owners? Wouldn't it be nice for Slackware to take you straight into a GUI? Fortunately, there's an easy way to do just that.

Slackware uses the System V init system which allows the administrator to boot into or change to different runlevels, which are really just different "states" the computer can be in. In fact, shutting down the computer is really only a case of changing to a runlevel which accomplishes just that. Runlevels can be rather complicated, so we won't delve into them any further than necessary.

Runlevels are configured in `inittab` (5). The most common ones are runlevel 3 (Slackware's default) and runlevel 4 (GUI). In order to tell Slackware to boot to a GUI screen, simply open `/etc/inittab` with your favorite editor of choice. (You may wish to refer to one of the chapters on `vi` or `emacs` at this point.) Near the top, you'll see the relevant entries.

```
#These are the default runlevels in Slackware:
#0 = halt
#1 = single user mode
#2 = unused (but configured the same as runlevel 3)
#3 = multiuser mode (default Slackware runlevel)
#4 = X11 with KDM/GDM/XDM (session managers)
#5 = unused (but configured the same as runlevel 3)
#6 = reboot

#Default runlevel. (Do not set to 0 or 6)
id:3:initdefault:
```

In this file (along with most configuration files) anything following a hash symbol # is a comment and not interpreted by init(8). Don't worry if you don't understand everything about inittab, as many veteran users don't either. The only line we are interested in is the last on above. Simply change the 3 to a 4 and reboot.

```
#These are the default runlevels in Slackware:
#0 = halt
#1 = single user mode
#2 = unused (but configured the same as runlevel 3)
#3 = multiuser mode (default Slackware runlevel)
#4 = X11 with KDM/GDM/XDM (session managers)
#5 = unused (but configured the same as runlevel 3)
#6 = reboot

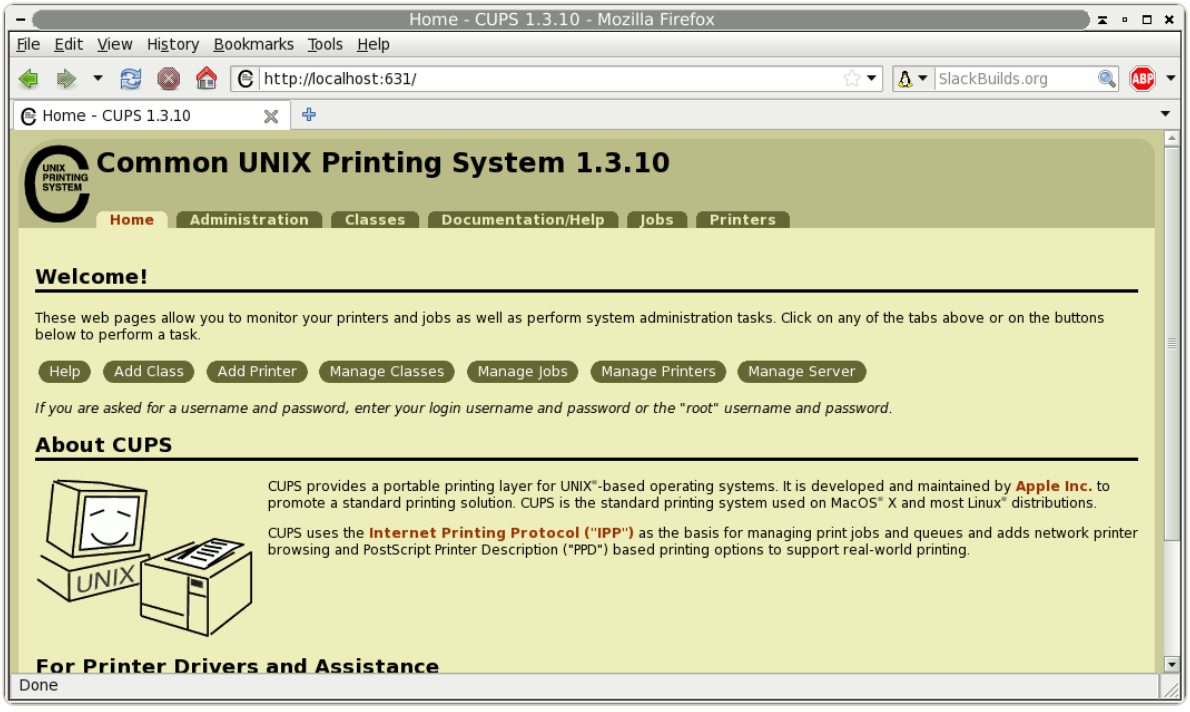
#Default runlevel. (Do not set to 0 or 6)
id:4:initdefault:
```

8. Printing

8.1 Choosing A Printer

Linux hasn't always had a great history with printers. For many years, printing was a black art to many Linux users, and very few printers worked reliably. Today, most printers will work well with Linux, but some still do not. If you're purchasing a new printer, be aware that many of the cheap inkjet models aren't as well supported in Linux as more expensive laser printers. If you're unsure about a printer, you can check online to see if others have had success with it.

All these warnings are perhaps a bit overkill though, as the large majority of printers work with Linux after only a brief and simple setup. The progress in this direction is largely due to the efforts of the Common UNIX Printing System, (CUPS). CUPS is a printing system used by Slackware and most other Linux distributions today. It primarily uses a graphical setup procedure accessed via a web browser. In order to setup a printer with CUPS, you'll need to open a web browser such as firefox, konqueror, or links and go to http://localhost:631 .



You might find that a quick click-through of CUPS configures your printer nearly automatically. Or, you may find that further configuration is required. To learn more about how printing works or how to get a stubborn printer configured, read on.

8.2 Getting the driver

There are, essentially, three types of printer drivers:

1. Postscript printers use the unversal-ish language of Postscript to communicate with computers. A driver for postscript printers is usually not needed, since a postscript-compatible subsystem called `Ghostscript` is already installed.
2. Gutenprint are drivers engineered by GNU Linux developers. It provides support for roughly 700 printers.
3. Manufacturers may provide Linux drivers for their printers. Find out by going to the manufacturer's driver and support website and searching for your model.

Since `gutenprint` is already installed on Slackware, from this set of three categories, we have two methods of installing drivers:

1. For the manufacturer's drivers, installation is usually the same as any other software on your system; use `installpkg` or `rpm2tgz` to install the driver package. Be sure to read the documentation bundled with the drivers.
2. For Postscript printers, there is no "installation" as such; simply download the appropriate `PPD` file and keep it in a sensible location on your hard drive.

Once you've located and installed or downloaded the necessary components, you're ready to run `CUPS` .

8.3 Setting Up a Printer in CUPS

From this point onward, setting up a printer is just a series of following the step-by-step instructions with CUPS, but understanding how the printing configuration actually works might help clarify what CUPS does is doing.

The file `/etc/cups/printers.conf` consists of definitions which detail the printing devices your computer will be able to access, with one marked as the default device. If you wish to edit this file manually (and you probably don't), you must stop the `cupsd` `CUPS` daemon.

A typical entry would look something like this:

```
<Printer r1060>
Info Ricoh 1060
Location Downstairs
MakeModel Ricoh Aficio 1060 - CUPS+Gutenprint v5.2.6
DeviceURI lpd://192.168.4.8
State Idle
StateTime 1316011347
Type 12308
Filter application/vnd.cups-raw 0 -
Filter application/vnd.cups-raster 100 rastertogutenprint.5.2
#standard-ish stuff below here
Accepting Yes
Shared No
JobSheets none none
QuotaPeriod 0
PageLimit 0
KLimit 0
OpPolicy default
ErrorPolicy stop-printer
</Printer>
```

In this example, we have given the printer the name `r1060` , a human-readable identifier `Ricoh 1060` .

The `MakeModel` attribute is gained from `lpinfo` , which lists all available printer drivers on your system. So, if you know that you have a Ricoh 1060 that you want to print to, then you would issue this command as root:

```
darkstar:~# lpinfo -m | grep 1060
```

This lists the drivers that you have installed, grepping for the string 1060:

```
gutenprint.5.2://brother-hl-1060/expert Brother HL-1060 - CUPS+Gutenpri
gutenprint.5.2://ricoh-afc_1060/expert Ricoh Aficio 1060 - CUPS+Gutenpr
```

The `MakeModel` is the last half of the appropriate result; in this case `Ricoh Aficio 1060 - CUPS+Gutenprint v5.2.6`

The final vital entry is the device URI, or where on the network (or physical location, such as the USB port), the printer can be found. In this example, we use `DeviceURI lpd://192.168.4.8` because we are using the `lpd` (line printer daemon) protocol to send data to the printer.

Now you understand what is being configured, and you can use the more common (and easier) method of doing this from the configuration tool that runs inside of a web browser.

In the CUPS interface, choose the Administration tab, and choose to Add Printer . You should be asked to enter administrative authorization here; enter `root` as the admin and your `root password` .

You will be presented with a list of printer interfaces and protocols that you can use for a printer. In many cases, you will want to add the printer via the `LPD/LPR` protocol (unless you've managed to find a printer that requires some other protocol).

Note that if the printer is plugged directly into your computer, and is on, you should see it listed as a Local Printer.

Assuming the printer is networked, the next screen will ask for the location of the printer. Using lpd:// as the protocol, enter the IP address of the printer. To find the

IP address of the printer, you will probably need to look at the printer's settings, or you may be able to determine it from your router.

Whether your printer is connected via USB or network, the following screen will ask for human-readable details about the printer; this is for your reference only, so enter a name for the printer that makes sense to you and your users (the model number usually), a description (something that is distinctive about the printer if you have more than one of the same printers), and the location (describing where it is in the building).

On the next screen, point CUPS to the printer driver. If the printer is a postscript printer (as most laserprinters are) then you may need only the `PPD` for that printer. If your printer is not postscript or has special features that require additional drivers, then define the make (manufacturer) and you will then be presented with a list of available drivers. Select the appropriate driver.

The printer is now installed and will be the default printer for all of your applications.

8.4 Printing from the Command Line

Now that you have successfully installed and configured your printer, you may also use `lpr` to print from the command line.

`lpr` sends documents to a printer but before using it, you might want to define a default printer by using `lpadmin` as root:

```
#lpadmin -d r1060
```

Note In this example, `r1060` is the human readable name given to the printer in either `/etc/cups/printers.conf` or in the `CUPS` configuration.

If you do not have root privileges on the workstation you are using, you can also set the `PRINTER` environment variable:

```
$ PRINTER=r1060
```

```
$ export PRINTER
```

Once the printer has been set, then you may print:

```
$ lpr foo.txt
```

8.4.1 Formatting for Print

`lpr` , like so many other UNIX applications, does one thig: sends files to a printer. It doesn't much care if the file looks good or even fits on a page. When printing large text files that have not been formatted for print, use `pr` (1).

`pr` is a simple text formatter that takes any text document and makes sure that it contains line breaks and page breaks, with an optional header and footer, page numbering, and much more. It has many options, but the defaults are usually good enough. `pr` outputs the results of the formatting to standard out, meaning it simply takes the text document, formats it, and displays the results in the terminal. This, of course, means that it can be redirected to `lpr` :

```
darkstar:~$ pr foo.txt | lpr
```

This will format `foo.txt` and send the formatted output to the default printer.

As usual, see the `pr` man page for a list of the customizations you can make to the default formatting.

9. Users and Groups

9.1 What Are Users and Groups?

Slackware Linux inherits a strong multi-user tradition from its UNIX inspiration. This means that multiple people may use the system at once, but it also means that each of these people may have different permissions. This allows users to prevent others from modifying their files, or lets system administrators explicitly define what users can and cannot do on the system. Moreover, users need not be actual people at all. In fact, Slackware includes several dozen pre-defined user and group accounts that are not typically used by regular users. Rather these accounts allow the system administrator to segment the system for security reasons. We'll see how that's done in the next chapter on filesystem permissions.

9.2 Managing Users and Groups

The easiest way to add new users in Slackware is through the use of our very fine `adduser` shell script. `adduser` will prompt you to enter the details of the new user you wish to creature and step you through the process quickly and easily. It will even create a password for the new user.

```
darkstar:~# adduser

Login name for new user []: david

User ID ('UID') [ defaults to next available ]:

Initial group [ users ]:
Additional UNIX groups:

Users can belong to additional UNIX groups on the system.
For local users using graphical desktop login managers such
as XDM/KDM, users may need to be members of additional groups
```

```
to access the full functionality of removable media devices.

* Security implications *
Please be aware that by adding users to additional groups may
potentially give access to the removable media of other users.

If you are creating a new user for remote shell access only,
users do not need to belong to any additional groups as standard,
so you may press ENTER at the next prompt.

Press ENTER to continue without adding any additional groups
Or press the UP arrow to add/select/edit additional groups
: audio cdrom floppy plugdev video

Home directory [ /home/david ]

Shell [ /bin/bash ]

Expiry date (YYYY-MM-DD) []:

New account will be created as follows:

---------------------------------------
Login name.......: david
UID..............: [ Next available ]
Initial group....: users
Additional groups: audio,cdrom,floppy,plugdev,video
Home directory...: /home/david
Shell............: /bin/bash
Expiry date......: [ Never ]

This is it... if you want to bail out, hit Control-C. Otherwise, press
ENTER to go ahead and make the account.

Creating new account...

Changing the user information for david
```

```
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Changing password for david
Enter the new password (minimum of 5, maximum of 127 characters)
Please use a combination of upper and lower case letters and numbers.
New password:
Re-enter new password:
Password changed.

Account setup complete.
```

The addition of optional groups needs a little explaining. Every user in Slackware has a single group that it is always a member of. By default, this is the "users" group. However, users can belong to more than one group at a time and will inherit all the permissions of every group they belong to. Typical desktop users will need to add several group memberships in order to do things like play sound or access removeable media like cdroms or USB flash drives. You can simply press the up arrow key at this section and a list of default groups for desktop users will magically appear. You can of course, add to or remove groups from this listing.

Now that we've demonstrated how to use the interactive `adduser` program, lets look at some powerful non-interactive tools that you may wish to use. The first is `useradd` (8). `useradd` is a little less friendly, but much faster for creating users in batches. This makes it ideal for use in shell scripts. In fact, `adduser` is just such a shell script and uses `useradd` for most of the heavy lifting. `useradd` has many options and we can't explain them all here, so refer to its man page for the complete details. Now, let's make a new user.

```
darkstar:~# useradd -d /data/home/alan -s /bin/bash -g users -G audio,
```

Here I have added the user "alan". I specified the user's home directory as `/data/home/alan` and used `bash` as my shell. Also, I specified my default group as "users" and added myself to a number of useful groups for dekstop use. You'll note that `useradd` does not do any prompting like `adduser` . Unless you want to accept the defaults for everything, you'll need to tell `useradd` what to do.

Now that we know how to add users, we should learn how to add groups. As you might have guessed, the command for doing this is `groupadd` (8). `groupadd` works in the same way as `useradd` , but with far fewer options. The following command adds the group "slackers" to the system.

```
darkstar:~# groupadd slackers
```

Deleting users and groups is easy as well. Simply run the `userdel` (8) and `groupdel` (8) commands. By default, `userdel` will leave the user's home directory on the system. You can remove this with the `-r` argument.

9.3 Other User and Group Tools

Several other tools exist for managing users and groups. Perhaps the most important one is `passwd` (1). This command changes a user account's password. Normal users may change their own passwords only, but root can change anyone's password. Also, root can lock a user account with the `-l` argument. This doesn't actually shutout the account, but instead changes the user's encrypted password to a value that can't be matched.

The easiest way for modifying a user's information is the `usermod` (8) utility which is capable of modifying everything from group membership to home directories. A full listing of its features won't be given here, so check the man page.

`usermod` is perhaps the best tool to use for modifying a user's group members. The `-s` and `-G` arguments accomplish this.

```
darkstar:~# usermod -a -G wheel alan
darkstar:~# usermod -G wheel alan
```

It important to note the differences in the two commands above. The first command adds the user "alan" to the "wheel" group without modifying any other groups "alan" belongs to. The second command also makes "alan" a member of the "wheel" group, but also removes the user's membership from any other groups, something you will very rarely want to do!

Another useful tool is `chsh` (1) which changes a user's default shell. Like `passwd` , normal users can only change their own shell, but the root user can change anyone's.

The last tool we're going to discuss is `chfn` (1). This is used to enter identifying information on the user such as his phone number and real name. This information is stored in the `passwd` (5) file and retrieved using `finger` (1).

9.4 Managing Users and Groups Manually

Like most things in Slackware Linux, users and groups are stored in plain-text files. This means that you can edit all the details of a user, or even create a new user or group simply by editing these files and doing a few other tasks like creating the user's home directory. Of course, after you see how this is done you'll appreciate just how simple the included tools make this task.

Our first stop is the `/etc/passwd` file. Here, all the information about a user is stored, except for (oddly enough) the user's password. The reason for this is rather simple. `/etc/passwd` must be readable by all users on the system, so you wouldn't want passwords stored there, even if they are encrypted. Let's take a quick look at my entry in this file.

```
alan:x:1000:100:,,,:/home/alan:/bin/bash
```

Each line in this file contains a number of fields seperated by a colon. They are, from left to right: username, password, UID, GUID, a comment field, home directory, and shell. You'll notice that the password field for every entry is an `x` . That is because Slackware uses shadow passwords, so the actual encrypted password is stored in `/etc/shadow` . Let's take a look there.

```
alan:$1$HlR?M3fkL@oeJmsdLfhsLFM*4dflPh8:14197:0:99999:7:::
```

The `shadow` file contains more than just the encrypted password as you'll notice. The fields here, again from left to right, are: username, encrypted password, last day the password was changed, days before the password may be changed again, how many days before the password expires, days that the account will be disabled after expiring, when the account was disabled, and a reserved field. You may notice on some accounts that the various "days" fields often include very large numbers. The reason for this is that Slackware counts time from the "Epoch" which is January 1, 1970 for historical reasons.

To create a new user account, you'll just need to open these files using `vipw` (8). This will open `/etc/passwd` in the editor defined by your VISUAL variable or your EDITOR variable if VISUAL isn't defined. If neither is present, it will fall back to `vi` by default. If you pass the `-s` argument, it will open `/etc/shadow` instead. It's important to use `vipw` instead of using any other editor, because `vipw` will lock the file and prevent other programs from editing it right underneath your feet.

That isn't all you'll need to do however; you must also create the user's home directory and change the user's password using `passwd` .

10. Filesystem Permissions

10.1 Permissions Overview

As we've discussed, Slackware Linux is a multi-user operating system. Because of this, its filesystems are mutli-user as well. This means that every file or directory has a set of permissions that can grant or deny privileges to different users. There are three basic permissions and three sets of permissions for each file. Let's take a look at an example file.

```
darkstar:~$ ls -l /bin/ls
-rwxr-xr-x 1 root root 81820 2007-06-08 21:12 /bin/ls
```

Recall from chapter 4 that `ls -l` lists the permissions for a file or directory along with the user and group that "own" the file. In this case, the permissions are rwxr-xr-x, the user is root and the group is also root. The permissions section, while grouped together, is really three seperate pieces. The first set of three letters are the permissions granted to the user that owns the file. The second set of three are those granted to the group owner, and the final three are permissions for everyone else.

Set	Listing	Meaning
Owner	rwx	The owner "root" may read, write, and execute
Group	r-x	The group "root" may read and execute
Others	r-x	Everyone else may read and execute

The permissions are pretty self explainatory of course, at least for files. Read, write, and execute allow you to read a file, write to it, or execute it. But what do these permissions mean for directories? Simply put, the read permissions grants

the ability to list the directory's contents (say with `ls`). The write permission grants the ability to create new files in the directory as well as delete the entire directory, even if you otherwise wouldn't be able to delete some of the other files inside it. The execute permission grants the ability to actually enter the directory (with the `bash` built-in command cd for example).

Let's look at the permissions on a directory now.

```
darkstar:~$ ls -ld /home/alan
drwxr-x--- 60 alan users 3040 2008-06-06 17:14 /home/alan/
```

Here we see the permissions on my home directory and its ownership. The directory is owned by the user alan and the group users. The user is granted all rights (rwx), the group is granted only read and execute permissions (r-x), and everyone else is prohibited from doing anything.

10.2 chmod, chown, and chgrp

So now that we know what permissions are, how do we change them? And for that matter, how do we assign user and group ownership? The answer is right here in this section.

The first tool we'll discuss is the useful `chown` (1) command. Using `chown`, we can (you guessed it), change the ownership of a file or directory. `chown` is historically used only to change the user ownership, but can change the group ownership as well.

```
darkstar:~# ls -l /tmp/foo
total 0
-rw-r--r-- 1 alan users 0 2008-06-06 22:29 a
-rw-r--r-- 1 alan users 0 2008-06-06 22:29 b
darkstar:~# chown root /tmp/foo/a
darkstar:~# ls -l /tmp/foo
total 0
```

```
-rw-r--r-- 1 root users 0 2008-06-06 22:29 a
-rw-r--r-- 1 alan users 0 2008-06-06 22:29 b
```

By using a colon after the user account, you may also specify a new group account.

```
darkstar:~# chown root:root /tmp/foo/b
darkstar:~#  ls -l /tmp/foo
total 0
-rw-r--r-- 1 root users 0 2008-06-06 22:29 a
-rw-r--r-- 1 root root  0 2008-06-06 22:29 b
```

`chown` can also be used recursively to change the ownership of all files and directories below a target directory. The following command would change all the files under the directory `/tmp/foo` to have their ownership set to root:root.

```
darkstar:~# chown -R root:root /tmp/foo/b
```

Specifying a colon and a group name without a user name will simply change the group for a file and leave the user ownership intact.

```
darkstar:~# chown :wheel /tmp/foo/a
darkstar:~# ls -l /tmp/foo
ls -l /tmp/foo
total 0
-rw-r--r-- 1 root wheel 0 2008-06-06 22:29 a
-rw-r--r-- 1 root root 0 2008-06-06 22:29 b
```

The younger brother of `chown` is the slightly less useful `chgrp` (1). This command works just like `chown` , except it can only change the group ownership of a file. Since `chown` can already do this, why bother with `chgrp` ? The answer is simple. Many other operating systems use a different version of `chown` that cannot change the group ownership, so if you ever come across one of those, now you know how.

There's a reason we discussed changing ownership before changing permissions. The first is a much easier concept to grasp. The tool for changing permissions on a file or directory is `chmod` (1). The syntax for it is nearly identical to that for `chown` , but rather than specify a user or group, the administrator must specify either a set of octal permissions or a set of alphabetic permissions. Neither one is especially easy to grasp the first time. We'll begin with the less complicated octal permissions.

Octal permissions derive their name from being assigned by one of eight digits, namely the numbers 0 through 7. Each permissions is assigned a number that is a power of 2, and those numbers are added together to get the final permissions for one of the permission sets. If this sounds confusing, maybe this table will help.

Permission	**Meaning**
Read	4
Write	2
Execute	1

By adding these values together, we can reach any number between 0 and 7 and specify all possible permission combinations. For example, to grant both read and write privilages while denying execute, we would use the number 6. The number 3 would grant write and execute permissions, but deny the ability to read the file. We must specify a number for each of the three sets when using octal permissions. It's not possible to specify only a set of user or group permissions this way for example.

```
darkstar:~# ls -l /tmp/foo/a
```

```
-rw-r--r-- 1 root root  0 2008-06-06 22:29 a
darkstar:~# chmod 750 /tmp/foo/a
darkstar:~# ls -l /tmp/foo/a
-rwxr-x--- 1 root root  0 2008-06-06 22:29 a
```

`chmod` can also use letter values along with + or - to grant or deny permissions. While this may be easier to remember, it's often easier to use the octal permissions.

Permission	**Letter Value**
Read	r
Write	w
Execute	x

Accounts Affected	**Letter Value**
User/Owner	u
Group	g
Others/World	o

To use the letter values with `chmod` , you must specify which set to use them with, either "u" for user, "g" for group, and "o" for all others. You must also specify whether you are adding or removing permissions with the "+" and "-" signs. Multiple sets can be changed at once by seperating each with a comma.

```
darkstar:/tmp/foo# ls -l
total 0
-rw-r--r-- 1 alan users 0 2008-06-06 23:37 a
-rw-r--r-- 1 alan users 0 2008-06-06 23:37 b
-rw-r--r-- 1 alan users 0 2008-06-06 23:37 c
-rw-r--r-- 1 alan users 0 2008-06-06 23:37 d
```

```
darkstar:/tmp/foo# chmod u+x a
darkstar:/tmp/foo# chmod g+w b
darkstar:/tmp/foo# chmod u+x,g+x,o-r c
darkstar:/tmp/foo# chmod u+rx-w,g+r,o-r d
darkstar:/tmp/foo# ls -l
-rwxr--r-- 1 alan users 0 2008-06-06 23:37 a*
-rw-rw-r-- 1 alan users 0 2008-06-06 23:37 b
-rwxr-x--- 1 alan users 0 2008-06-06 23:37 c*
-r-xr----- 1 alan users 0 2008-06-06 23:37 d*
```

Which you prefer to use is entirely up to you. There are places where one is better than the other, so a real Slacker will know both inside out.

10.3 SUID, SGID, and the "Sticky" Bit

We're not quite done with permissions just yet. There are three other "special" permissions in addition to those mentioned above. They are SUID, SGID, and the sticky bit. When a file has one or more of these permissions set, it behaves in special ways. The SUID and SGID permissions change the way an application is run, while the sticky bit restricts deletion of files. These permissions are applied with `chmod` like read, write, and execute, but with a twist.

SUID and SGID stand for "Set User ID" and "Set Group ID" respectively. When an application with one of these bits is set, the application runs with the user or group ownership permissions of that application regardless of what user actually executed it. Let's take a look at a common SUID application, the humble `passwd` and the files it modifies.

```
darkstar:~# ls -l /usr/bin/passwd \ /etc/passwd \ /etc/shadow -rw-r--r
```

Notice the permissions on `passwd` . Instead of an `x` in the user's execute slot, we have an `s` . This tells us that `passwd` is a SUID program, and when we run it, the process will run as the user "root" rather than as the user that actually executed it.

The reason for this is readily apparent as soon as you look at the two files it modifies. Neither `/etc/passwd` nor `/etc/shadow` are writeable by anyone other than root. Since users need to change their personal information, `passwd` must be run as root in order to modify those files.

So what about the sticky bit? The sticky bit restricts the ability to move or delete files and is only ever set on directories. Non-root users cannot move or delete any files under a directory with the sticky bit set unless they are the owner of that file. Normally anyone with write permission to the file can do this, but the sticky bit prevents it for anyone but the owner (and of course, root). Let's take a look at a common "sticky" directory.

```
darkstar:~# ls -ld /tmp
drwxrwxrwt 1 root root 34844 2008-03-24 16:11 /tmp
```

Naturally, being a directory for the storage of temporary files sytem wide, `/tmp` needs to be readable, writeable, and executable by anyone and everyone. Since any user is likely to have a file or two stored here at any time, it only makes good sense to prevent other users from deleting those files, so the sticky bit has been set. You can see it by the presence of the `t` in place of the `x` in the world permissions section.

Permission Type	Octal Value	Letter Value
SUID	4	s
SGID	2	s
Sticky	1	t

When using octal permissions, you must specify an additional leading octal value. For example, to recreate the permission on `/tmp` , we would use 1777. To recreate those permissions on `/usr/bin/passwd` , we would use 4711. Essentially, any time this leading fourth octet isn't specified, `chmod` assumes its value to be 0.

```
darkstar:~# chmod 1777 /tmp
darkstar:~# chmod 4711 /usr/bin/passwd
```

Using the alphabetic permission values is slightly different. Assuming the two files above have permissions of 0000 (no permissions at all), here is how we would set them.

```
darkstar:~# chmod ug+rwx,o+rwt /tmp
darkstar:~# chmod u+rws,go+x /usr/bin/passwd
```

11. Working with Filesystems

11.1 The Filesystem Hierarchy

Slackware Linux stores all of its files and directories under a single `/` directory, typically referred to as "root". This is in stark contract to what you may be familiar with in the form of Microsoft Windows. Different hard disk partitions, cdroms, usb flash drives, and even floppy disks can all be mounted in directories under `/` , but do not have anything like "drive letters". The contents of these devices can be found almost anywhere, but there are some sane defaults that Slackware sets up for you. For example, cd-rw drives are most often found at `/mnt/cd-rw` . Here are a few common directories present on nearly all Slackware Linux installations, and what you can expect to find there.

/	**The root directory, under which all others exist**
/bin	Minimal set of binary programs for all users
/boot	The kernel, initrd, and other requirements for booting Slackware
/etc/	System configuration files
/dev	Collection of special files allowing direct access to hardware
/home	User directories where personal files and settings are stored
/media	Directory for auto-mounting features in DBUS/HAL
/mnt	Places to temporarily mount removable media
/opt	Directory where some (typicaly proprietary) software may be installed
/proc	Kernel exported filesystem for process information
/root	The root user's home directory
/sbin	Minimal set of system or superuser binaries
/srv	Site-specific data such as web pages served by this system
/sys	Special kernel implimentation details
/tmp	Directory reserved for temporary files for all users
/usr	All non-essential programs, libraries, and shared files
/var	Regularly changing data such as log files

11.2 Local Filesystem Types

The Linux kernel supports a wide variety of filesystems, which allows you to choose from a long list of features to tailor to your particular need. Fortunately, most of the default filesystem types are adequate for any needs you may have.

Some filesystems are geared towards particular media. For example, the iso9660 filesystem is used almost exclusively for CD and DVD media.

11.2.1 ext2

ext2 is the oldest filesystem included in Slackware Linux for storing data on hard disks. Compared to other filesystems, ext2 is simplistic. It is faster than most others for reading and writing data, but does not include any journaling capability. This means that after a hard crash, the filesystem must be exhaustively checked to discover and (hopefully) fix any errors.

11.2.2 ext3

ext3 is the younger cousin of ext2. It was designed to replace ext2 in most situations and shares much the same code-base, but adds journaling support. In fact, ext3 and ext2 are so much alike that it is possible to convert one to the other on the fly without lose of data. ext3 enjoys a lot of popularity for these reasons. There are many tools available for recovering data from this filesystem in the event of catastrophic hardware failure as well. ext3 is a good general purpose filesystem with journaling support, but fails to perform as well as other journaling filesystems in specific cases. One pitfall to ext3 is that the filesystem must still go through this exhaustive check every so often. This is done when the filesystem is mounted, usually when the computer is booted, and causes an annoying delay.

11.2.3 ext4

ext4 is the latest in the ext series of filesystems. It was designed to build upon ext3 with new ideas on what filesystems should do. While Slackware supports ext4, you should remember that this filesystem is still very new (particularly in file system terms) and is under heavy development. If you require stability over performance, you may wish to use a different filesystem such as ext3. With that said, ext4 does boast some major improvements over ext3 in the performance arena, but many people don't yet trust it for stable use.

11.2.4 reiserfs

reiserfs is one of the oldest journaling filesystems for the Linux kernel and has been supported by Slackware for many years. It is a very fast filesystem

particularly well suited for storing, retrieving, and writing lots of small files. Unfortunately there are few tools for recovering data should you experience a drive failure, and reiserfs partitions experience corruption more often than ext3.

11.2.5 XFS

XFS was contributed to the Linux kernel by SGI and is one of the best filesystems for working with large volumes and large files. XFS uses more RAM than other filesystems, but if you need to work with large files its performance there is well worth the penalty in memory usage. XFS is not particularly ill-suited for desktop or laptop use, but really shines on a server that handles medium to large size files all day long. Like ext3, XFS is a fully journaled filesystem.

11.2.6 JFS

JFS was contributed to the Linux kernel by IBM and is well known for its responsiveness even under extreme conditions. It can span colossal volumes making it particularly well-suited for Network Attached Storage (NAS) devices. JFS's long history and thorough testing make it one of the most reliable journaling filesystems available for Linux.

11.2.7 iso9660

iso9660 is a filesystem specifically designed for optical media such as CDs and DVDs. Since optical disks are read-only media, the linux kernel does not even include write support for this filesystem. In order to create an iso9660 filesystem, you must use user-land tools like `mkisofs` (8) or `growisofs` (8).

11.2.8 vfat

Sometimes you may need to share data between Windows and Linux computers, but can't transfer the files over a network. Instead you require a shared hard drive partition or a USB flash drive. The humble vfat filesystem is the best choice here since it is supported by the largest variety of operating systems. Unfortuantely, being a Microsoft designed filesystem, it does not store permissions in the same way as traditional Linux filesystems. This means that special options must be used to allow multiple users to access data on this filesystem.

11.2.9 swap

Unlike other filesystems which hold files and directories, swap partitions hold virtual memory. This is very useful as it prevents the system from crashing should all your RAM be consumed. Instead, the kernel copies portions of the RAM into swap and frees them up for other applications to use. Think of it as adding virtual memory to your computer, very slow virtual memory. swap is typically a fail-safe and shouldn't be relied upon for continual use. Add more RAM to your system if you find yourself using lots of swap.

11.3 Using mount

Now that we've learned what (some of) the different filesystems available in Linux are, it's time we looked at how to use them. In order to read or write data on a filesystem, that filesystem must first be mounted. To do this, we (naturally) use `mount` (8). The first thing we must do is decide where we want the filesystem located. Recall that there are no such things are drive letters denoting filesystems in Linux. Instead, all filesystems are mounted on directories. The base filesystem on which you install Slackware is always located at `/` and others are always located in subdirectories of `/` . `/mnt/hd` is a common place to temporarily locate a partition, so we'll use that in our first example. In order to mount a filesystem's contents, we must tell mount what kind of filesystem we have, where to mount it, and any special options to use.

```
darkstar:~# mount -t ext3 /dev/hda3 /mnt/hd -o ro
```

Let's disect this. We have an ext3 filesystem located on the third partition of the first IDE device, and we've decided to mount its contents on the directory `/mnt/hd` . Additionally, we have mounted it read-only so no changes can be made to these contents. The `-t ext3` argument tells `mount` what type of filesystem we are using, in this case it is ext3. This lets the kernel know which driver to use. Often `mount` can determine this for itself, but it never hurts to explicitly declare it. Second, we tell `mount` where to locate the filesystem's contents. Here we've

chosen `/mnt/hd` . Finally, we must decide what options to use if any. These are declared with the `-o` argument. A short-list of the most common options follows.

ro	**read-only**
rw	read-write (default)
uid	user to own the contents of the filesystem
gid	group to own the contents of the filesystem
noexec	prevent execution of any files on the filesystem
defaults	sane defaults for most filesystems

If this is your first Linux installation, the only options you typically need to be concerned about are `ro` and `rw` . The exception to this rule comes when you are dealing with filesystems that don't handle traditional Linux permissions such as vfat or NTFS. In those cases you'll need to use the `uid` or `gid` options to allow non-root users access to these filesystems.

```
darkstar:~# mount -t vfat /dev/hda4 /mnt/hd -o uid=alan
```

But Alan, that's appalling! I don't want to have to tell mount what filesystem or options to use everytime I load a CD. It should be easier than that. Well thankfully, it is. The `/etc/fstab` file contains all this information for filesystems that the installer sets up for you, and you can make additions to it as well. `fstab` (5) looks like a simple table containing the device to mount along with its filesystem type and optional arguments. Let's take a look.

```
darkstar:~# cat /etc/fstab
/dev/hda1       /               reiserfs    defaults              1
/dev/hda2       /home           reiserfs    defaults              1
/dev/hda3       swap            swap        defaults              0
```

```
/dev/cdrom       /mnt/cdrom       auto        noauto,owner,ro,users  0
/dev/fd0         /mnt/floppy      auto        noauto,owner           0
devpts           /dev/pts         devpts      gid=5,mode=620         0
proc             /proc            proc        defaults               0
```

If you have an entry in `fstab` for your filesystem, you need only tell mount the device node or the mount location.

```
darkstar:~# mount /dev/cdrom
darkstar:~# mount /home
```

One final use for `mount` is to tell you what filesystems are currently mounted and with what options. Simply run `mount` without any arguments to display these.

11.4 Network Filesystems

In addition to local filesystems, Slackware supports a number of network filesystems as both client and server. This allows you to share data between multiple computers transparently. We'll discuss the two most common: NFS and SMB.

11.4.1 NFS

NFS is the Network File System for Linux as well as several other common operating systems. It has modest performance but supports the full range of permissions for Slackware. In order to use NFS as either a client or a server, you must run the remote procedure call daemon. This is easily accomplished by setting the `/etc/rc.d/rc.rpc` file executable and telling it to start. Once it has been set executable, it will run automatically every time you boot into Slackware.

```
darkstar:~# chmod +x /etc/rc.d/rc.rpc
```

```
darkstar:~# /etc/rc.d/rc.rpc start
```

Mounting an NFS share is little different than mounting a local filesystem. Rather than specifying a local device, you must tell mount the domain name or IP address of the NFS server and the directory to mount with a colon between them.

```
darkstar:~# mount -t nfs darkstar.example.com:/home /home
```

Running an NFS server is a little bit different. First, you must configure each directory to be exported in the `/etc/exports` file. `exports` (5) contains information about what directories will be shared, who they will be shared with, and what special permissions to grant or deny.

```
#See exports(5) for a description.
#This file contains a list of all directories exported to other compute
#It is used by rpc.nfsd and rpc.mountd.

/home/backup 192.168.1.0/24(sync,rw,no_root_squash)
```

The first column in `exports` is a list of the files to be exported via NFS. The second column is a list of what systems may access the export along with special permissions. You can specify hosts via domain name, IP address, or netblock address (as I have here). Special permissions are always a parenthetical list. For a complete list, you'll need to read the man page. For now, the only special option that matters is `no_root_squash` . Usually the root user on an NFS client cannot read or write an exported share. Instead, the root user is "squashed" and forced to act as the nobody user. `no_root_squash` prevents this.

You'll also need to run the NFS daemon. Starting and stopping NFS server support is done with the `/etc/rc.d/rc.nfsd` rc script. Set it executable and run it just like we did for `rc.rpc` and you are ready to go.

11.4.2 SMB

SMB is the Windows network file-sharing protocol. Connecting to SMB shares (commonly called samba shares) is fairly straight forward. Unfortuantely, SMB isn't as strongly supported as NFS. Still, it offers higher performance and connectivity with Windows computers. For these reasons, SMB is the most common network file-sharing protocol deployed on local networks. Exporting SMB shares from Slackware is done through the samba daemon and configured in `smb.conf` (5). Unfortunately configuring samba as a service is beyond the scope of this book. Check online for additional documentation, and as always refer to the man page.

Thankfully mounting an SMB share is easy and works almost exactly like mounting an NFS share. You must tell mount where to find the server and what share you wish to access in exactly the same way. Additionally, you must specify a username and password.

```
darkstar:~# mount -t cifs //darkstar/home /home -o username=alan,passwo
```

You may be wondering why the filesystem type is cifs instead of smbfs. In older versions of the Linux kernel, smbfs was used. This has been deprecated in favor of the better performing and more secure general purpose cifs driver.

All SMB shares require the `username` and `password` arguments. This can create a security problem if you wish to place your samba share in fstab. You may avoid this problem by using the `credentials` argument. `credentials` points to a file which contains the username and password information. As long as this file is safely guarded and readable only by root, the likelyhood that your authentication credentials will be compromised is lessened.

```
darkstar:~# echo "username=alan" > /etc/creds-home
```

```
darkstar:~# echo "password=secret" >> /etc/creds-home
darkstar:~# mount -t cifs //darkstar/home -o credentials=/etc/creds-hom
```

12. vi

12.1 What is vi?

Scattered all around your computer are thousands of text files. To a new user, this may seem inconsequential, but almost everything in Slackware Linux uses a plain-text file for configuration. This allows users to make changes to the system quickly, easily, and intuitively. In chapter 5, we looked at a few commands such as `cat` and `less` that can be used to read these files, but what if we want to make changes to them? For that, we need a text editor, and `vi` is up to the task.

In short, `vi` is one of the oldest and most powerful text editors still used today. It's beloved by system administrators, programmers, hobbiests, and others the world over. In fact, nearly this entire book was written using `vi` ; only the next chapter on `emacs` was written with that editor.

A little further explanation is needed to learn exactly what `vi` is today though, as Slackware Linux technically doesn't include `vi` . Rather, Slackware includes two vi "clones", `elvis` (1) and `vim` (1). These clones add many additional features to vi such as syntax highlighting, binary editing modes, and network support. We won't go too deeply into all these details. By default, if you execute `vi` on Slackware Linux, you'll be using `elvis` , so all examples in this chapter will assume that is what you are using. If you've used another Linux distribution before, you may be more familiar with `vim` . If so, you might wish to change the symlink for `/usr/bin/vi` to point to `/usr/bin/vim` , or add an alias to your shell's startup scripts. `vim` is generally considered to be more feature-rich than

`elvis` , but `elvis` is a much smaller program and contains more features than most users will ever need.

`vi` is very powerful, but also somewhat cumbersome and challening for a new user to learn. However, mastering `vi` is an important skill for any self-respecting system administrator to learn, as `vi` is included on nearly every Linux distribution, every BSD system, and every UNIX system in existance. It's even included in Mac OS X. Once you've learned `vi` , you'll not have to learn another text editor to work on any of these systems. In fact, `vi` clones have even been ported to Microsoft Windows systems, so you can use it there too.

12.2 The Different Modes of vi

New users are often frustrated when using `vi` for the first time. When invoked without any arguments, `vi` will display a screen something like this.

```
~
~
~
~
~
~
~
~
~
~
~
                                Command
```

At this point, the user will being typing and expect the keys he presses to appear in the document. Instead, something really strange happens. The reason for this is simple. `vi` has different operation "modes". There is a command mode and an insert mode. Command mode is the default; in this mode, each keystroke performs

a particular action such as moving the cursor around, deleting text, yanking (copying) text, searching, etc.

12.3 Opening, Saving, and Quitting

Ok, so you've decided that you want to learn how to use `vi` . The first thing to do is learn how to open and save files. Opening files is actually pretty easy. Simply type the filename as an argument on the command-line and `vi` will happily load it for you. For example, `vi chapter_11.xml` will open the file `chapter_11.xml` and load its content onto the screen, simple enough. But what if we've finished with one document and wish to save it? We can do that in command mode using the `:w` command. When in command mode, pressing the `:` key temporarily positions the cursor on the very bottom line of the window and allows you to enter special commands. (This is technically known as ex-mode after the venerable `ex` application which we will not document here.) The command to save your current work is `:w` . Once this is done, `vi` will write your changes to the buffer back into the file. If you wish to open another document, simply use the `:e other_document` command and `vi` will happily open it for you. If you've made changes to the buffer but haven't saved it yet, `:e` will fail and print a warning message on the bottom line. You can bypass this with the `:e!` command. Most ex-mode commands in `vi` can be "forced" by adding `!` to them. This tells `vi` that you want to abandon any changes you've made to the buffer and open the other document immediately.

But what if I don't like my changes and want to quit or start over? That's easily done as well. Executing the `:e!` command without any arguments will re-open the current document from the beginning. Quitting `vi` is as simple as running the `:q` command if you haven't made any changes to the buffer, or `:q!` if you'd like to quit and abandon those changes.

12.4 Moving Around

Moving around in `vi` is perhaps the hardest thing for a new user to learn. `vi` does not traditionally use the directional arrow keys for cursor movement, although in Slackware Linux that is an option. Rather, movement is simply another command issued in command-mode. The reason for this is rather simple. `vi` actually

predates the inclusion of directional arrow keys on keyboards. Thus, movement of the cursor had to be accomplished by using the few keys available, so the right-hand "home row" keys of h , `j` , k , and `l` were chosen. These keys will move the cursor about whenever `vi` is in command mode. Here's a short table to help you remember how they work.

Command	Result
h	Move the cursor one character left.
j	Move the cursor one line down
k	Move the cursor one line up
l	Move the cursor one character right

Moving around is a little more powerful than that though. Like many command keys, these movement keys accept numerical arguments. For example, `10j` will move the cursor down 10 lines. You can also move to the end or beginning of the current line with `$` and `^` , respectively.

12.5 Editing A Document

Now that we're able to open and save documents, as well as move around in them, it's time to learn how to edit them. The primary means of editing is to enter insert mode using either the `i` or a command keys. These either insert text at the cursor's current location, or append it after the cursor's current location. Once into insert mode, you can type any text normally and it will be placed into your document. You can return to command mode in order to save your changes by pressing the ESC key.

12.6 vi Cheat Sheet

Since `vi` can be difficult to learn, I've prepared a short cheat sheat that should help you with the basics until you begin to feel comfortable.

Command	Result
h	Move the cursor one character left.
j	Move the cursor one line down
k	Move the cursor one line up
l	Move the cursor one character right
10j	Move the cursor ten lines down
G	Move to the end of the file
^	Move to the beginning of the line
$	Move to the end of the line
dd	Remove a line (and store it in the copy buffer)
5dd	Remove 5 lines (and store them in the copy buffer)
dw	Remove a single word (and store it in the copy buffer)
5dw	Remove five words (and store them in the copy buffer)
yy	Yank (copy) a line (and store it in the copy buffer)
yw	Yank (copy) a single word (and store it in the copy buffer)
5yw	Yank five words (and store them in the copy buffer)
p	Paste the contents of the copy buffer at the cursor's location
P	Paste the contents of the copy buffer above the cursor's location
r	Replace a single character
R	Replace multiple characters
x	Delete a character

Command	Result
X	Delete the previous character
u	Undo the last action
:s'old'new'g	Replace all occurances of 'old' with 'new' (current line only)
:%s'old'new'g	Replace all occurances of 'old' with 'new' (all lines)
/asdf	Locate next occurance of asdf
:q	Quit (without saving)
:w	Save the current document
:w file	Save the current document as 'file'
:x	Save and quit

13. Emacs

13.1 What is Emacs?

`vi` and its clones are very functional and powerful editors. However, they are often considered not particularly extensible. `vim` is a successful and powerful `vi` variant that shrugs this trend, being both extremely extensible and lightweight. However, many users prefer a more "heavy" and extensible editor. This is why many people (including the author of this chapter) prefer `Emacs` .

Emacs takes extensibility up to eleven. Outside of a core of C, the rest of `Emacs` is written in a Lisp variant, nearly all of which is exposed to you, so that you may configure it or even extend it at will (many good Emacs Lisp tutorials can be found

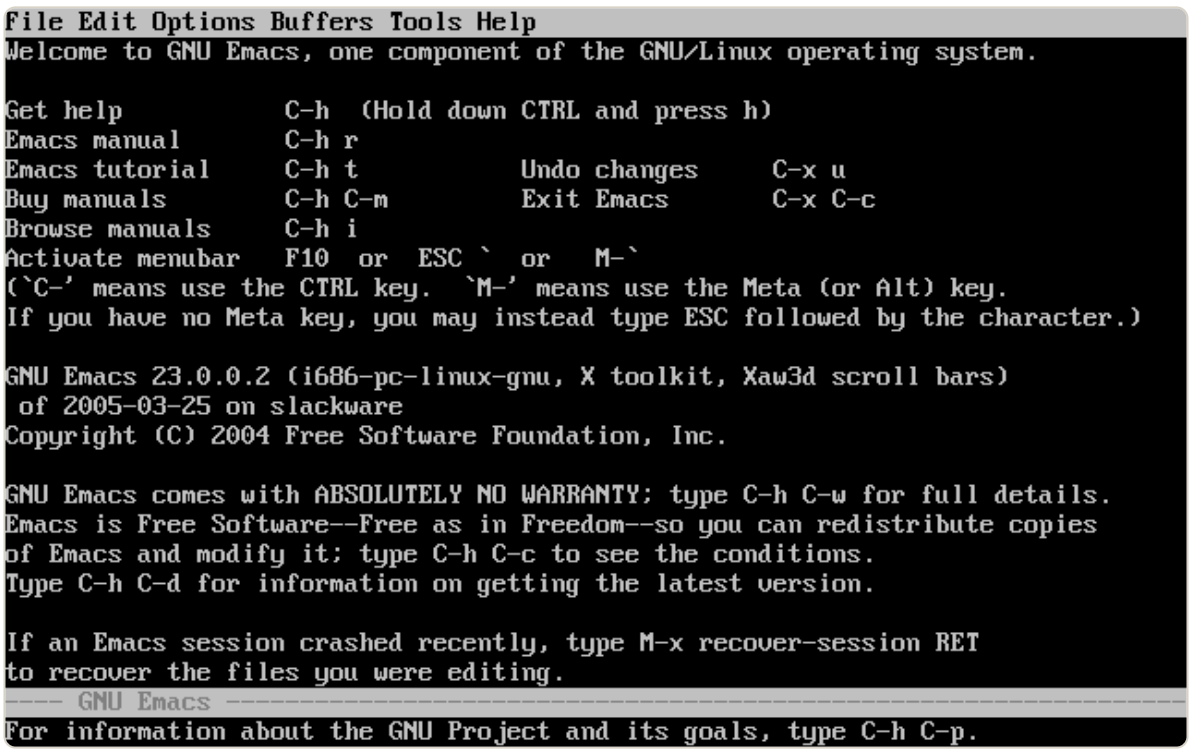
on the Internet). People have written all sorts of extensions in Emacs Lisp, from syntax highlighting for an obscure language, to a built-in terminal. In fact, there's even a `vi` emulation mode within `Emacs` (called viper), so you can still get the modal editing that comes with vi, while having access to the power of the `Emacs` core.

Like `vi` , there are many variants of `Emacs` (termed "emacsen"). However, the one most commonly used (and the only one in Slackware) is GNU Emacs. When people reference "Emacs" directly, they almost always mean GNU Emacs.

Unlike `vi` , `Emacs` operates more like a traditional editor by default. This means that most keyboard shortcuts can be performed without repeatedly changing modes. You can open up a file and start typing away without having to learn what the modes do, or forgetting which one you are currently using.

13.2 Starting Out

`Emacs` can be started simply by running the `emacs` command in your terminal. When you first start it in a console without arguments, you will see something that resembles this:



If you are in X windows, `Emacs` may start a GUI instead of running in your console. If this is the case and you don't want a GUI, you can invoke it with the flag `'-nw'` .

While here, you can browse around using the keyboard arrow keys. Underlined elements are links, and you can activate them by moving over them and pressing 'enter'. The documentation mentioned is very good, and can help you get your bearings should you have any problems. Also note how they describe key sequences such as `C-h` , meaning press the h key while holding down the `CTRL` key. Same deal with `M-</kbd> , meaning to hold the the <kbd>Meta</kbd> key (usually " <kbd>Alt</kbd> ") and press the backtick "" key. When they say (e.g.) C-x C-c , this means to press the "x" key while holding down the CTRL key, then press the "x" key while also holding down the CTRL key. Conveniently, this is also one of the more important commands in Emacs : to close it.`

Alternatively, if you call `emacs` with a file name as an argument, it will open that file, just like `vi` . Upon doing this, you will be presented with the contents of the file in question. Here, you can navigate the document using traditional arrow keys and type in information at will without any issues.

Say you make some edits, and you now want to save your file. The following key sequence will do that: `C-x C-s` . If you made a new file, you will be prompted for this in what is called the "minibuffer", the blank line below the gray line at the bottom of the screen. Type in the file name of your choice, then hit `Enter` . If you don't want to save the file, you can press `C-g` , which aborts operations that ask for input. Do note that tab-completion is usually available for operations that use the minibuffer.

Should you want to open a new file within your same `Emacs` session, type in `C-x C-f` . You will be prompted for a file name in the minibuffer. `Emacs` doesn't care whether it exists or not. If it doesn't exist, a new buffer will be created for it (the file will be created upon saving with `C-x C-s`), or it will be opened as expected. However, the old file will still be open! You can switch back to it using `C-x C-b` ,

entering in the file's name (or more technically, the buffer's name), then hitting `Enter` .

13.3 How to Move Around

Like `vi` , `Emacs` is also older than the arrow keys on your keyboard. Also, like in `vi` , using the arrow keys to navigate files is also supported. While the `vi` movement keys are more ergonomic, `emacs` 's are more "mnemonic". However, it is still very possible to operate using the main `Emacs` keybindings quickly and efficiently. Here is a table of the basic movement keybindings:

Command	Result
C-f	Move the cursor one character to the right (forward)
C-b	Move the cursor one character to the left (backward)
C-n	Move the cursor one line down (next)
C-p	Move the cursor one line up (previous)

Of course, like with `vi` it is also possible to repeat these commands with a numeric argument. If you type in `M-1 M-0 C-p` , or `C-u 10 C-p` , the cursor will move ten lines up. If you type in `M-5 C-f` or `C-u 5 C-f` , the cursor will move five characters to the right.

13.4 Getting Help

`Emacs` contains a great deal of documentation, to the point that it is often called a "self-documenting" editor. This is because it provides mechanisms for providing users with documentation while you are using it. Here are some useful functions that display documentation (they all start with `C-h`):

Command	Result
C-h f FUNCTION-NAME Enter	Show documentation for function FUNCTION-NAME

Command	Result
C-h k C-x C-c	Show documentation for the function bound to the keys C-x C-c
C-h t	Show the Emacs tutorial
C-h ?	Show all help-related functions

`C-h t` is especially useful if you want or need practice using `Emacs` .

13.5 Calling Functions

As noted earlier, `Emacs` exports a large number of functions to for interactive use. Some of these, like those opening and saving files, are mapped to keys. Others (like the ones for moving to the beginning and end of lines) are not. To call them, you have to invoke them. Say we want to call the function "end-of-line". We would do this:

```
M-x end-of-line Enter
```

And the cursor would move to the end of the line, as the function name suggests.

13.6 Emacs Cheat Sheet

While Emacs can be simple to use, its scope can easily be overwhelming. Below are some useful Emacs commands. Some aspects have been simplified, most notably regarding text selection. These concepts, and more, are described the `Emacs` manual, and various on-line tutorials. Decent summaries can also be gleaned from web searches.

Command	Result
C-f	Move the cursor one character to the right (forward)
C-b	Move the cursor one character to the left (backward)
C-n	Move the cursor one line down (next)

Command	Result
C-p	Move the cursor one line up (previous)
C-h f FUNCTION-NAME Enter	Show documentation for function FUNCTION-NAME
C-h k C-x C-c	Show documentation for the function bound to the keys C-x C-c
C-h t	Show the Emacs tutorial
C-h ?	Show all help-related functions
M-`	Access the Menu Bar
C-g	Cancel the current operation. This is most useful when in the minibuffer.
M-x FUNCTION-NAME Enter	Call the interactive function FUNCTION-NAME
M-1 M-0 C-n	Move the cursor ten lines down
C-u 10 C-n	Move the cursor ten lines down (same as above)
M-x beginning-of-line	Move the cursor to the beginning of the current line
M-x end-of-line	Move the cursor to the end of the current line
M->	Move the cursor to the end of the buffer
M-<	Move the cursor to the beginning of the buffer
C-k	Remove text from the cursor to the end of the line and place it into the kill ring
C-space	Enter selection mode (use normal motion keys to move around). Press C-space again to leave it.

Command	Result
C-w	While in selection mode, delete the selected region, and store the result into the kill ring
M-w	While in selection mode, store the selected region into the kill ring.
C-y	"Yanks" the contents of the kill ring and places them at the cursor's location
C-/	Undo the previous action. Unlike most other editors, this includes previous undo actions.
insert	Enable or disable overwriting characters
C-s asdf Enter	Forward incremental search for the string "asdf". Repeat C-s as needed to search for future items, or C-r (below) to search backwards.
C-r asdf Enter	Backward incremental search for the string "asdf". Repeat C-r as needed to search for future items, or C-s (above) to search forwards.
M-% old Enter new Enter	Search for each instance of "old" and prompt you to replace it with "new". You can force replacement of all items by typing "!" at the replacement prompt.
C-x C-c	Exit Emacs, prompting you to save each unsaved buffer before doing so
C-x C-s	Save the currrent buffer to its file
C-x C-w new-file.txt Enter	Save the current buffer to a file "new-file.txt"

14. Networking

14.1 netconfig

Computers aren't very interesting on their own. Sure, you can install games on them, but that just turns them into glorified entertainment consoles. Today, computers need to be able to talk to one another; they need to be networked. Whether you're installing a business network with hundreds or thousands of computers or just setting up a single PC for Internet access, Slackware is simple and easy. This chapter should teach you how to setup typical wired networks. Common wireless setup will be thoroughly discussed in the next section, but much of what you read here will be applicable there as well.

There are many different ways to configure your computer to connect to a network or the Internet, but they fall into two main categories: static and dymanic. Static addresses are solid; they are set with the understanding that they will not be changed, at least not anytime soon. Dynamic addresses are fluid; the assumption is that the address will change at some time in the future. Typically any sort of network server requires a static address simply so other machines will know where to contact it when they need services. Dynamic addresses tend to be used for workstations, Internet clients, and any machine that doesn't require a static address for any reason. Dynamic addresses are more flexible, but present complications of their own.

There are many different kinds of network protocols that you might encounter, but most people will only ever need to deal with Internet Protocol (IP). For that reason, we'll focus exclusively on IP in this book.

14.2 Manual Configuration

Ok, so you've installed Slackware, you've setup a desktop, but you can't get it to connect to the Internet or your business's LAN (local area network), what do you do? Fortunately, the answer to that question is simple. Slackware includes a number of tools to configure your network connection. The first we will look at is the very powerful `ifconfig` (8), which is used to setup or modify the configuration of

the most common hardware for connecting to networks: a Network Interface Card (NIC or Ethernet Card). `ifconfig` is an incredibly powerful tool capable of doing much more than setting IP addresses. For a complete introduction, you should read its man page. For now, we're just going to use it to display and change the network addresses of some ethernet controllers.

```
darkstar:~# ifconfig
lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:699 errors:0 dropped:0 overruns:0 frame:0
          TX packets:699 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:39518 (38.5 KiB)  TX bytes:39518 (38.5 KiB)

wlan0     Link encap:Ethernet  HWaddr 00:1c:b3:ba:ad:4c
          inet addr:192.168.1.198  Bcast:192.168.1.255  Mask:255.255.25
          inet6 addr: fe80::21c:b3ff:feba:ad4c/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1630677 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1183224 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1627370207 (1.5 GiB)  TX bytes:163308463 (155.7 MiB

wmaster0  Link encap:UNSPEC  HWaddr 00-1C-B3-BA-AD-4C-00-00-00-00-00-00
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

As you can clearly see here, when run without any arguments, `ifconfig` will display all the information it has on all the ethernet cards (and wireless ethernet

cards) present on your system. The above represents a typical wireless connection from my laptop, so don't be afraid if what you see on your system doesn't match. If you don't see any ethX or wlanX interfaces though, the interface may be down. To show all currently present NICs whether they are "up" or "down", simply pass the `-a` argument.

```
darkstar:~# ifconfig -a
eth0      Link encap:Ethernet  HWaddr 00:19:e3:45:90:44
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:122780 errors:0 dropped:0 overruns:0 frame:0
          TX packets:124347 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:60495452 (57.6 MiB)  TX bytes:17185220 (16.3 MiB)
          Interrupt:16

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:699 errors:0 dropped:0 overruns:0 frame:0
          TX packets:699 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:39518 (38.5 KiB)  TX bytes:39518 (38.5 KiB)

wlan0     Link encap:Ethernet  HWaddr 00:1c:b3:ba:ad:4c
          inet addr:192.168.1.198  Bcast:192.168.1.255  Mask:255.255.25
          inet6 addr: fe80::21c:b3ff:feba:ad4c/4 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1630677 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1183224 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1627370207 (1.5 GiB)  TX bytes:163308463 (155.7 MiB

wmaster0  Link encap:UNSPEC  HWaddr 00-1C-B3-BA-AD-4C-00-00-00-00-00-00
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
```

```
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

Notice that the eth0 interface is now listed among the returns. `ifconfig` can also change the current settings on a NIC. Typically, you would need to change the IP address and subnet mask, but you can change virtually any parameters.

```
darkstar:~# ifconfig eth0 192.168.1.1 netmask 255.255.255.0
darkstar:~# ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 00:19:e3:45:90:44
          inet addr:192.168.1.1  Bcast:192.168.1.255  Mask:255.255.255
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:122780 errors:0 dropped:0 overruns:0 frame:0
          TX packets:124347 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:60495452 (57.6 MiB)  TX bytes:17185220 (16.3 MiB)
          Interrupt:16
```

If you look carefully, you'll notice that the interface now has the 192.168.1.1 IP address and a 255.255.255.0 subnet mask. We've now setup the basics for connecting to our network, but we still need to setup a default gateway and our DNS servers. In order to do that, we'll need to look at a few more tools.

Next on our stop through networking land is the equally powerful `route` (8). This tool is responsible for modifying the Linux kernel's routing table which affects all data transmission on a network. Routing tables can become immensely complex or they can be straight-forward and simple. Most users will only ever need to setup a default gateway, so we'll show you how to do that here. If for some reason you need a more complex routing table, you would be well advised to read the entire man page for `route` as well as consulting other sources. For now, let's take a look at our routing table immediately after setting up eth0.

```
darkstar:~# route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use
192.168.1.0     *               255.255.255.0   U     0      0        0
loopback        *               255.0.0.0       U     0      0        0
```

I won't explain everything here, but the general information should be easy to pick up if you're familiar with networking at all. The Destination and Genmask fields specify a range of IP addresses to match. If a Gateway is defined, information in the form of packets will be sent to that host for forwarding. We also specify an interface in the final field that the information should traverse. Right now, we can only communicate with computers with addresses between 192.168.1.0 and 192.168.1.255 and ourselves through the loopback interface, a type of virtual NIC that is used for routing information from this computer to itself. In order to reach the rest of the world, we'll need to setup a default gateway.

```
darkstar:~# route add default gw 192.168.1.254
darkstar:~# route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use
192.168.1.0     *               255.255.255.0   U     0      0        0
loopback        *               255.0.0.0       U     0      0        0
default         192.168.1.254   0.0.0.0         UG    0      0        0
```

You should immediately notice the addition of a default route. This specifies what router should be used to reach any addresses that aren't specified elsewhere in our routing table. Now, when we try to connect to say, 64.57.102.34, the information will be sent to 192.168.1.254 which is responsible for delivering the data for us. Unfortunately, we're still not quite through. We need some way of converting domain names like slackware.com into IP addresses that the computer can use. For that, we need to make use of a DNS server.

Fortunately, setting up your computer to use an external (or even an internal) DNS server is very easy. You'll need to use your favorite text editor and open the `/etc/resolv.conf` file. Don't ask me what happened to the e . On my computer, `resolv.conf` looks like this.

```
#/etc/resolv.conf
search lizella.net
nameserver 192.168.1.254
```

Many users won't need the `search` line. This is used to map hostnames to domain names. Basically, if I attempt to connect to "barnowl", the computer knows to look for "barnowl.lizella.net" thanks to this search line. We're mainly interested in the `nameserver` line. This tells Slackware what domain name servers (DNS) to connect to. Generally speaking, these should always be specified by IP address. If you know what DNS servers you should use, you can just add them one at a time to individual nameserver lines. In fact, I don't know of any practical limit to the number of nameservers that can be specified in `resolv.conf` , so add as many as you like. Once this is done, you should be able to communicate with other hosts via their fully qualified domain name.

But Alan! That's a lot of hard work! I don't want to do this time and again for dozens or even hundreds of machines. You're absolutely right, and that's why smarter people than you and me created DHCP. DHCP stands for Dynamic Host Control Protocol and is a method for automatically configuring computers with unique IP addresses, netmasks, gateways, and DNS servers. Most of the time, you'll want to use DHCP. The majority of wireless routers, DSL or cable modems, even firewalls all have DHCP servers to can make your life much easier. Slackware includes two main tools for connecting to an exising DHCP server and can even act as a DHCP server for other computers. For now though, we're just going to look at DHCP clients.

First on our list is `dhcpcd` (8), part of the ISC DHCP utilities. Assuming your computer is physically connected to your network, and that you have an operating DHCP server on that network, you can configure your NIC in one shot.

```
darkstar:~/ dhcpcd eth0
```

If everything went according to plan, your NIC should be properly configured, and you should be able to communicate with other computers on your network, and with the Internet at large. If for some reason, `dhcpcd` fails, you may want to try `dhclient` (8). `dhclient` is an alternative to `dhcpcd` and works in basically the same way.

```
darkstar:~/ dhclient eth0
Listening on LPF/eth0/00:1c:b3:ba:ad:4c
Sending on   LPF/eth0/00:1c:b3:ba:ad:4c
Sending on   Socket/fallback
DHCPREQUEST on eth0 to 255.255.255.255 port 67
DHCPACK from 192.168.1.254
bound to 192.168.1.198 -- renewal in 8547 seconds.
```

Why does Slackware include two DHCP clients? Sometimes a particular DHCP server may be broken and not respond well to either `dhcpcd` or `dhclient` . In those cases, you can fall back to the other DHCP client in hopes of getting a valid response from the server. Traditionally, Slackware uses `dhcpcd` , and this works in the vast majority of cases, but it may become necessary at some point for you to use `dhclient` instead. Both are excellent DHCP clients, so use whichever you prefer.

14.3 Automatic Configuration with rc.inet1.conf

Manually configuring interfaces is an important skill to have, but it can become tedious. No one wants to manually setup their Internet connection every time the system boots. More importantly, you may not always have physical access to the machine when it boots. Slackware makes it easy to automatically configure ethernet (and wireless) cards at system startup with `/etc/rc.d/`

`rc.inet1.conf` . For now, we're going to focus on traditional wired ethernet networking; the next chapter will discuss various wireless options.

`rc.inet1.conf` is an incredibly powerful configuration file, capable of configuring most of your network cards automatically when Slackware is started. The file is filled with useful comments, but there is also a man page that more thoroughly discusses its use. To begin, we're going to look at some of the options used on one of my personal machines.

```
#Config information for eth0:
IPADDR[0]="192.168.1.250"
NETMASK[=]"255.255.255.0"
USE_DHCP[0]=""
DHCP_HOSTNAME[0]=""
#Some lines ommitted.
GATEWAY="192.168.1.254"
```

This represents most of the information necessary to configure a static IP address on a single ethernet controller. `netconfig` will usually fill in these values for a single ethernet device for you. If you have multiple network cards in your machine and need all of them activated automatically at boot time, then you'll need to edit or add additional entries into this file in the same manner as above. First, let me go over some of the basics.

As you may have already guessed, IPADDR[n] is the Internet Protocol Address for the n network interface card. Typically, n corrosponds to `eth0` , `eth1` , and so on, but this isn't always the case. You can specify these values to pertain to a different network controller with the IFNAME[n] variable, but we will reserve that for , as it more commonly pertains to wireless network controllers.

Likewise, NETMASK[n] is the subnet mask to use for the network controller. If these lines are left empty, then static IP addresses will not be automatically assigned to this network controller. The USE_DHCP[n] variable tells Slackware (naturally) to use DHCP to configure the interface. DHCP_HOSTNAME[n] is rarely used, but some DHCP servers may require it. In that case, it must be set to a valid

hostname. Finally, we come to the GATEWAY variable. It is actually set lower in the file than it appears in my example, and it controls the default gateway to use. You may be wondering why there is no GATEWAY[n] variable. The answer to that lies in how Internet Protocol works. I won't go into an in-depth discussion on that subject, but suffice it to say that there is only ever one default route that a computer can use no matter how many interfaces are attached to it.

If you need to use static IP addressing, you will have to obtain a unique static IP address and the subnet mask for the interface, as well as the default gateway address, and enter those here. There is no place to enter DNS information in `rc.inet1.conf` , so DNS servers will have to be manually placed into `resolv.conf` as discussed in . Of course, if you use `netconfig` , this will be handled for you by that program. Now let's take a look at another interface on my computer.

```
#Config information for eth1:
IPADDR[1]=""
NETMASK[1]=""
USE_DHCP[1]="yes"
DHCP_HOSTNAME[1]=""
```

Here I am telling Slackware to configure eth1 using DHCP. I do not need to set the IPADDR[1] or NETMASK[1] variables when using DHCP (in fact, if they are set, they will be ignored). Slackware will happily contact a DHCP server as soon as the machine begins to boot.

15. Wireless Networking

15.1 iwconfig

Wireless networking is somewhat more complicated than traditional wired networking, and requires additional tools for setup. Slackware includes a diverse collection of wireless networking tools to allow you to configure your wireless network interface card (WNIC) at the most basic level. We won't cover everything here, but should give you a solid foundation to get up and running quickly. The first tool we are going to look at is `iwconfig` (8). When run without any argument, `iwconfig` displays the current wireless information on any and all NICs on your computer.

```
darkstar:~# iwconfig
lo        no wireless extensions.

eth0      no wireless extensions.

wmaster0  no wireless extensions.

wlan0     IEEE 802.11abgn  ESSID:"nest"
          Mode:Managed  Frequency:2.432 GHz  Access Point:
00:13:10:EA:4E:BD
          Bit Rate=54 Mb/s   Tx-Power=17 dBm
          Retry min limit:7   RTS thr:off   Fragment thr=2352 B
          Encryption key:off
          Power Management:off
          Link Quality=100/100  Signal level:-42 dBm
          Rx invalid nwid:0  Rx invalid crypt:0  Rx invalid frag:0
          Tx excessive retries:0  Invalid misc:0   Missed beacon:0
```

```
tun0      no wireless extensions.
```

Unlike wired networks, wireless networks are "fuzzy". Their borders are hard to define, and multiple networks may overlap one another. In order to avoid confusion, each wireless network has (hopefully) unique identifiers. The two most basic identifiers are the Extended Service Set Identifier (ESSID) and the channel or frequency for radio transmission. The ESSID is simply a name that identifies the wireless network in question; you may have heard it referred to as the "network name" or something similar.

Typical wireless networks operate on 11 different frequencies. In order to connect to even the most basic wireless network, you will have to setup these two pieces of information, and possibly others, before setting up things like the WNIC's IP address. Here you can see that my ESSID is set to "nest" and my laptop is transmitting at 2.432 GHz. This is all that is required to connect to an unencrypted wireless LAN. (For any of you out there expecting to come to my house and use my unencrypted wireless, you should know that you'll have to break a 2048-bit SSL key before the access point will let you communicate with my LAN.)

```
darkstar:~# iwconfig wlan0 essid nest \
freq 2.432G
```

The `freq` and `channel` arguments control basically the same thing. You only need to use one. If you are unsure what frequency or channel to use, Slackware can usually figure this out for you.

```
darkstar:~# iwconfig wlan0 essid nest
\ channel auto
```

Now Slackware will attempt to connect to the strongest access point on the "nest" essid operating at any frequency.

15.2 Wired Equivilant Protection (or Lack Thereof)

Wireless networking is by its very nature less secure than wired networking. Having your information travelling on the airwaves makes it highly susceptible to interception by third parties, so over the years a number of methods have been devised to make wireless networking more secure. The first was called Wired Equivilant Protection, or WEP for short, and fell far short of its goal. If you are still using WEP today, I encourage you to consider using WPA2 or some other form of stronger encryption. Attacks against WEP are trivial and take only minutes to perform. Unfortunately there are still access points configured for WEP, and you may need to connect to one from time to time. Connecting to WEP encrypted access points is fairly simple, particularly if you have the key in hexidecimal format. We'll need to pass the `key` argument along with the password in hexidecimal or ASCII format. If using an ASCII password, you'll need to prepend it with "s;" but generally speaking, hexidecimal format is preferred.

```
darkstar:~# iwconfig wlan0 \
    key cf80baf8bf01a160de540bfb1c
darkstar:~# iwconfig wlan0 \
    key s:thisisapassword
```

15.3 Wifi Protected Access

Wifi Protected Access (or WPA for short) was the successor for WEP that aimed to fix several problems with wireless encryption. Unfortunately, WPA had some flaws as well. An update called WPA2 offers even stronger protection. At this time, WPA2 is supported by nearly all wireless network cards and access points, but some older devices may only support WEP. If you need to secure your wireless network traffic, WPA2 should be considered the minimum level of protection required.

Unfortunately, `iwconfig` is unable to setup WPA2 encryption on its own. For that, we need a helper daemon, `wpa_supplicant` (8).

Unfortunately, there's no easy way to manually configure a WPA2 protected network; you'll have to edit `/etc/wpa_supplicant.conf` directly with a text editor. Here we will discuss the simplest form of WPA2 protection, the Pre-Shared Key, or PSK for short. For details on setting up Slackware to connect to more complicated WPA2 encrypted networks, see the man page for `wpa_supplicant.conf` .

```
#/etc/wpa_supplicant.conf
#========================
#This line enables the use of wpa_cli which is used by rc.wireless
#if possible (to check for successful association)
ctrl_interface=/var/run/wpa_supplicant
#By default, only root (group 0) may use wpa_cli
ctrl_interface_group=0
eapol_version=1
ap_scan=1
fast_reauth=1
#country=US

#WPA protected network, supply your own ESSID and WPAPSK here:
network={
  scan_ssid=1
  ssid="nest"
  key_mgmt=WPA-PSK
  psk="secret passphrase"
}
```

The block of text we're interested in is the network block enclosed by curly braces. Here we have set the ssid for the network "nest" , and "secret passphrase" as the PSK to be used. At this point, WPA2 is properly configured. You can run

`wpa_supplicant` and then obtain an IP address via DHCP or set a static address. Of course, this is a lot of work; there must be an easier way to do this.

15.3.1 rc.inet1.conf revisited

Welcome back to `rc.inet1.conf` . You're recall that in we used this configuration file to automatically configure NICs whenever Slackware boots. Now, we will use it to configure wifi as well.

Note If you're using WPA2, you'll still need to setup `wpa_supplicant.conf` properly first, however.

Recall that each NIC had a name or number that identified the variables that correspond with it? The same hold true for wifi NICs, only they have even more variables due to the added complexity of wireless networking.

```
#rc.inet1.conf (excert)
#======================
##Example config information for wlan0.  Uncomment the lines you need a
##in your info.  (You may not need all of these for your wireless netwo
IFNAME[4]="wlan0"
IPADDR[4]=""
NETMASK[4]=""
USE_DHCP[4]="yes"
#DHCP_HOSTNAME[4]="icculus-wireless"
#DHCP_KEEPRESOLV[4]="yes"
#DHCP_KEEPNTP[4]="yes"
#DHCP_KEEPGW[4]="yes"
#DHCP_IPADDR[4]=""
WLAN_ESSID[4]="nest"
#WLAN_MODE[4]=Managed
#WLAN_RATE[4]="54M auto"
#WLAN_CHANNEL[4]="auto"
#WLAN_KEY[4]="D5AD1F04ACF048EC2D0B1C80C7"
#WLAN_IWPRIV[4]="set AuthMode=WPAPSK | \
#set EncrypType=TKIP | \
```

```
#set WPAPSK=96389dc66eaf7e6efd5b5523ae43c7925ff4df2f8b7099495192d44a774
WLAN_WPA[4]="wpa_supplicant"
#WLAN_WPADRIVER[4]="ndiswrapper"
```

When we discussed wired ethernet, each n in the variable corresponded with the n in `ethn` . Here however, that no longer holds true. Notice that the variable IFNAME[4] has a value of wlan0 . It is common for wireless cards to have an interface name other than ethn and that is reflected here. When `rc.inet1.conf` is read by the start-up scripts, Slackware knows to apply all these options to the wlan0 wifi NIC instead of the (probably non-existant) eth4 wired NIC. Many of the other options are the same. IP address information is added in exactly the same way we discussed for wired network cards in Chapter, Networking; however, we have a lot of new variables that need some explanation.

To begin, WLAN_ESSID[n] and WLAN_CHANNEL[n] should be self-explainatory by now; they refer the the essid and frequency to use. WLAN_MODE[n] is either `managed` or `ad-hoc` . Anyone connecting to an access point will want to use managed mode. WLAN_KEY[n] is the WEP key to use, if you're forced to use WEP. WLAN_IWPRIV[n] is a very complicated variable that sets other variables inside itself. WLAN_IWPRIV[n] is used for WPA2 networks. Here you tell Slackware what authentication mode, encryption type, and key to use for WPA2 connections. Please note that WLAN_KEY[n] and WLAN_IWPRIV[n] are mutually exclusive; you can't use both on the same interface. If you successfully configure all this, then Slackware will attempt to connect to your wireless network as soon as the system boots.

But wait, that's so much work! And what if I need to connect to multiple wireless networks? I take my laptop to work and school and need to seemlessly setup those wireless connections as soon as one is within range. Doing things this way is simply too much work. You're absolutely correct.

15.4 wicd

Introducing `wicd` (8), the premier wired and wireless network connection manager for the laptop user on the go. Pronounced "wicked", `wicd` is capable of storing information for any number of wireless networks you need and connecting

to them with a simple command or the click of a mouse. `wicd` is not part of the default Slackware installation at this time, as it interferes somewhat with the normal way of configuring network adapters, but you can find it in the `/extra` directory of your Slackware install disks or at your favorite mirror. `wicd` is both a network connection daemon and a graphical application for configuring networks. The CLI isn't forgotten either, as `wicd-curses` (8) is every bit as powerful as the traditional GUI front-end. In order to use `wicd` , you will need to disable support for any interfaces you have in `rc.inet1.conf` first.

```
#rc.inet1.conf
#=============
#Config information for eth0:
IPADDR[0]=""
NETMASK[0]=""
USE_DHCP[0]="no"
DHCP_HOSTNAME[0]=""
#Default gateway IP address:
GATEWAY=""
```

Now we can install `wicd` , setup the daemon to run on system boot-up, and begin using a more friendly application.

```
darkstar:~# installpkg /path/to/extra/wicd/wicd-1.6.2.1-1.txz
darkstar:~# chmod +x /etc/rc.d/rc.wicd
darkstar:~# /etc/rc.d/rc.wicd start
```

If you're predominately using the console, simply run `wicd-curses` from your command line. If instead, you are using a graphical desktop provided by `X` , you can start the graphical front-end from either the KDE or XFCE menu.

Optionally, you could manually run `wicd-client` (1) from a terminal or `run dialogue` .

On the graphical front-end, options for different networks are available via the Preferences button adjacent to the ESSID listed. In the terminal client, the same options can be reached by highlighting the ESSID you wish to use and pressing the right arrow key, which opens a configuration page for that network.

16. Basic Networking Utilities

So you've finally managed to setup your network connection, now what? How do you know that it's working? How do you know that you set it up correctly? And just what do you do now that it's setup? Well this chapter is for you.

16.1 Network Diagnostic Tools

Slackware Linux includes a great many networking tools for troubleshooting and diagnosing network connection troubles, or just for seeing what's out there on the network. Most of these tools are command-line tools, so you can run them from a virtual terminal or in a console window on your graphical desktop. A few of them even have graphical front-ends, but we're going to deal almost exclusively with command-line tools for now.

16.1.1 ping

`ping` (8) is a handy tool for determining if a computer is operational on your network or on the Internet at large. You can think of as a type of sonar for computers. By using it, you send out a "ping" and listen for an echo to determine if another computer or network device is listening. By default, `ping` checks for the remote computer once per second indefinitely, but you can change the interval between checks and the total number of checks easily, just check the man page. You can terminate the application at any time with `CTRL - c` . When `ping` is finished, it displays a handy summary of its activity. `ping` is very useful for

determining if a computer on your network or the Internet is available, but some systems block the packets `ping` sends, so sometimes a system may be functioning properly, but still not send replies.

```
darkstar:~# ping -c 3 www.slackware.com
64 bytes from slackware.com (64.57.102.34): icmp_seq=1 ttl=47 time=87.1
64 bytes from slackware.com (64.57.102.34): icmp_seq=2 ttl=47 time=86.2
64 bytes from slackware.com (64.57.102.34): icmp_seq=3 ttl=47 time=86.7

--- slackware.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 86.282/86.718/87.127/0.345 ms
```

16.1.2 traceroute

`traceroute` (8) is a handy tool for determining what route your packets take to reach some other computer. It's mainly of use for determining which computers are "near" or "far" from you. This distance isn't strictly geographical, as your Internet Service Provider may route traffic from your computer in strange ways. `traceroute` shows you each router between your computer and any other machine you wish to connect to. Unfortunately, many providers, firewalls, and routers will block `traceroute` so you might not get a complete picture when using it. Still, it remains a handy tool for network troubleshooting.

```
darkstar:~# traceroute www.slackware.com
traceroute to slackware.com (64.57.102.34), 30 hops max, 46 byte
packets
 1  gw.ctsmacon.com (192.168.1.254)  1.468 ms  2.045 ms  1.387 ms
 2  10.0.0.1 (10.0.0.1)  7.642 ms  8.019 ms  6.006 ms
 3  68.1.8.49 (68.1.8.49)  10.446 ms  9.739 ms  7.003 ms
 4  68.1.8.69 (68.1.8.69)  11.564 ms  6.235 ms  7.971 ms
 5  dalsbbrj01-ae0.r2.dl.cox.net (68.1.0.142)  43.859 ms  43.287 ms 44
 6  dpr1-ge-2-0-0.dallasequinix.savvis.net (204.70.204.146)  41.927 ms
```

```
 7  cr2-tengige0-7-5-0.dallas.savvis.net (204.70.196.29)  42.577 ms 46
 8  cr1-pos-0-3-3-0.losangeles.savvis.net (204.70.194.53)  78.070 ms 7
 9  bpr1-ge-3-0-0.LosAngeles.savvis.net (204.70.192.222)  77.533 ms 108
10  wiltel-communications-group-inc.LosAngeles.savvis.net (208.173.55.
11  tg9-4.cr01.lsancarc.integra.net (209.63.113.57)  84.789 ms  85.436
12  tg13-1.cr01.sntdcabl.integra.net (209.63.113.106)  87.608 ms 84.278
13  tg13-4.cr02.sntdcabl.integra.net (209.63.113.134)  87.284 ms 85.924
14  tg13-1.cr02.rcrdcauu.integra.net (209.63.114.169)  85.578 ms 85.285
15  209.63.99.166 (209.63.99.166)  84.515 ms  85.424 ms  85.956 ms
16  208.186.199.158 (208.186.199.158)  86.557 ms  85.822 ms  86.072 ms
17  sac-main.cwo.com (209.210.78.20)  88.105 ms  87.467 ms  87.526 ms
18  slackware.com (64.57.102.34)  85.682 ms  86.322 ms  85.594 ms
```

16.1.3 telnet

Once upon a time, `telnet` (1) was the greatest thing since sliced bread. Basically, `telnet` opens an unencrypted network connection between two computers and hands control of the session to the user rather than some other application. Using `telnet` , people could connect to shells on other computers and execute commands as if they were physically present. Due to its unencrypted nature this is no longer recommended; however, `telnet` is still used for this purpose by many devices.

Today, `telnet` is put to better use as a network diagnostic tool. Because it passes control of the session directly to the user, it can be used for a great variety of testing purposes. As long as you know what ASCII commands to send to the receiving computer, you can do any number of activies, such as read web pages or check your e-mail. Simply inform `telnet` what network port to use, and you're all set.

```
darkstar:~$ telnet www.slackware.com 80
Trying 64.57.102.34...
Connected to www.slackware.com.
Escape character is '^]'.
```

```
HEAD / HTTP/1.1
Host: www.slackware.com

HTTP/1.1 200 OK
Date: Thu, 04 Feb 2010 18:01:35 GMT
Server: Apache/1.3.27 (Unix) PHP/4.3.1
Last-Modified: Fri, 28 Aug 2009 01:30:27 GMT
ETag: "61dc2-5374-4a973333"
Accept-Ranges: bytes
Content-Length: 21364
Content-Type: text/html
```

16.1.4 ssh

As we mentioned, `telnet` may be useful as a diagnostic tool, but its unencrypted nature makes it a security concern for shell access. Thankfully, there's the secure shell protocol. Nearly every Linux, UNIX, and BSD distribution today makes use of OpenSSH, or `ssh` (1) for short. It is one of the most commonly used network tools today and makes use of the strongest cryptographic techniques. `ssh` has many features, configuration options, and neat hacks, enough to fill its own book, so we'll only go into the basics here. Simply run `ssh` with the user name and the host and you'll be connected to it quickly and safely. If this is the first time you are connecting to this computer, `ssh` will ask you to confirm your desire, and make a local copy of the encryption key to use. Should this key later change, `ssh` will warn you and refuse to connect because it is possible that some one is attempting to hijack the connection using what is known as a man-in-the-middle attack.

```
darkstar:~# ssh alan@slackware.com
alan@slackware.com's password: secret
alan@slackware.com:~$
```

The user and hostname are in the same form used by e-mail addresses. If you leave off the username part, `ssh` will use your current username when establishing the connection.

16.1.5 tcpdump

So far all the tools we've looked at have focused on making connections to other computers, but now we're going to look at the traffic itself. `tcpdump` (1) (which must be run as root) allows us to view all or part of the network traffic originating or received by our computer. `tcpdump` displays the raw data packets in a variety of ways with all the network headers intact. Don't be alarmed if you don't understand everything it displays, `tcpdump` is a tool for professional network engineers and system administrators. By default, it probes the first network card it finds, but if you have multiple interfaces, simply use the `-i` argument to specify which one you're interested in. You can also limit the data displayed using expressions and change the manner in which it is displayed, but that is best explained by the man page and other reference material.

```
darkstar:~# tcpdump -i wlan0
tcpdump: verbose output suppressed, use -v or -vv for full protocol
decode
listening on wlan0, link-type EN10MB (Ethernet), capture size 96 bytes
13:22:28.221985 IP gw.ctsmacon.com.microsoft-ds > 192.168.1.198.59387:
Flags [P.], ack 838190560, win 3079, options [nop,nop,TS val 138269748
ecr 339048583], length 164WARNING: Short packet. Try increasing the
snap length by 140
SMB PACKET: SMBtrans2 (REPLY)

13:22:28.222392 IP 192.168.1.198.59387 > gw.ctsmacon.com.microsoft-ds:
Flags [P.], ack 164, win 775, options [nop,nop,TS val 339048667 ecr
1382697489], length 134WARNING: Short packet. Try increasing the snap
length by 110
SMB PACKET: SMBtrans2 (REQUEST)
```

16.1.6 nmap

Suppose you need to know what network services are running on a machine, or multiple machines, or you wish to determine if multiple machines are responsive? You could `ping` each one individually, `telnet` to each port you're interested in, and note every detail, but that's very tedious and time consuming. A much easier alternative is to use a port scanner, and `nmap` (1) is just the tool for the job.

`nmap` is capable of scanning TCP and UDP ports, determining the operating system of a network device, probing each located service to determine its specific type, and much much more. Perhaps the simplist way to use `nmap` is to "ping" multiple computers at once. You can use network address notation (CIDR) or specify a range of addresses and `nmap` will scan every one and return the results to you when it's finished. You can even specify host names as you like.

In order to "ping" hosts, you'll have to use the `-sP` argument. The following command instructs `nmap` to "ping" www.slackware.com and the 16 IP addresses starting at 72.168.24.0 and ending at 72.168.24.15.

```
darkstar:~# nmap -sP www.slackware.com 72.168.24.0/28
```

Should you need to perform a port scan, `nmap` has many options for doing just that. When run without any arguments, `nmap` performs a standard TCP port scan on all hosts specified. There are also options to make `nmap` more or less aggressive with its scanning to return results quicker or fool intrusion detection services. For a full discussion, you should refer to the rather exhaustive man page. The following three commands perform a regular port scan, a SYN scan, and a "Christmas tree" scan.

```
darkstar:~# nmap www.example.com
darkstar:~# nmap -sS www.example.com
```

```
darkstar:~# nmap -sX www.example.com
```

Warning Be warned! Some Internet Service Providers frown heavily on port scanning and may take measures to prevent you from doing it. `nmap` and applications like it are best used on your own systems for maintenance and security purposes, not as general purpose Internet scanners.

16.1.7 host

Often network problems stem from a failure of DNS (Domain Name Service) which maps domain names to IP addresses. An easy way to perform quick DNS lookups is the `host` (1) command. When this is run, your computer will perform a few common DNS lookups and return the results.

```
darkstar:~# host www.slackware.com
www.slackware.com is an alias for slackware.com.
slackware.com has address 64.57.102.34
slackware.com mail is handled by 1 mail-mx.cwo.com.
```

16.1.8 dig

More complex DNS lookups can be manually performed with the `dig` (1) tool. `dig` is "the meanest dog in the pound" when it comes to troubleshooting DNS issues. With this tool, you can perform virtually any DNS lookup from reverse lookups to A, CNAME, MX, SP, TXT records and more. There are far too many command-line options and lookup types to go into depth here, but the man page lists all the common use cases.

```
darkstar:~# dig @207.69.188.185 www.slackware.com a

; <<>> DiG 9.4.3-P4 <<>> @207.69.188.185 www.slackware.com a
; (1 server found)
```

```
;; global options:  printcmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 57965
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 2, ADDITIONAL: 0

;; QUESTION SECTION:
;www.slackware.com.     IN  A

;; ANSWER SECTION:
www.slackware.com.  86400   IN  CNAME   slackware.com.
slackware.com.      86400   IN  A   64.57.102.34

;; AUTHORITY SECTION:
slackware.com.      86400   IN  NS  ns2.cwo.com.
slackware.com.      86400   IN  NS  ns1.cwo.com.

;; Query time: 348 msec
;; SERVER: 207.69.188.185#53(207.69.188.185)
;; WHEN: Sat Jul  3 16:25:10 2010
;; MSG SIZE  rcvd: 105
```

Let's take a took at the command-line options used above. The `@207.69.188.185` argument tells `dig` what DNS server to query. If it is not specified, `dig` will simply use whatever servers are listed in `/etc/resolv.conf` . The `a` argument at the end is the type of DNS record to lookup. In this case we looked for an "A" record which returned an IPv4 address.

16.1.9 finger

`finger` (1) isn't exactly a network diagnostic tool as much as it is a network-user diagnostic tool. Using `finger` , you can gather a handful of useful information about users on servers running the `fingerd` (8) daemon. Today very few servers still offer `fingerd` , but for those that do it can be a useful tool for keeping track of your friends and co-workers.

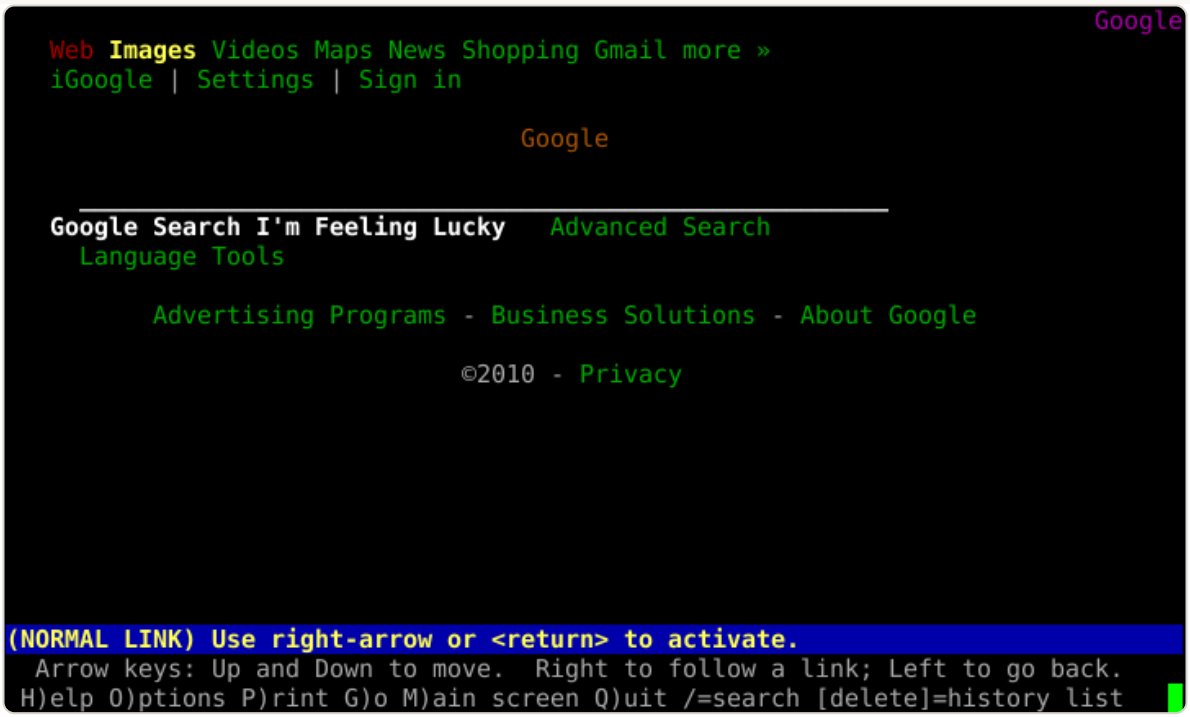
```
darkstar:~# finger alan@cardinal.lizella.net
[cardinal.lizella.net]
Login: alan                     Name: Alan Hicks
Directory: /home/alan                   Shell: /bin/bash
Office: 478 808 9919, 478 935 8133
On since Wed Apr 13 17:43 (UTC) on pts/9 from
75-150-12-113-atlanta.hfc.comcastbusiness.net
    32 minutes 24 seconds idle
     (messages off)
On since Wed Apr 13 17:45 (UTC) on pts/10 from :pts/9:S.0
   48 minutes 56 seconds idle
Mail forwarded to alan@lizella.net
No mail.
No Plan.
```

16.2 Web Browsers

Slackware includes a variety of web browsers. If you're using a graphical desktop, you'll find `Firefox` , `Seamonkey` , and others you may already be familiar with, but what about console access? Fortunately, there are a number of capable web browsers here as well.

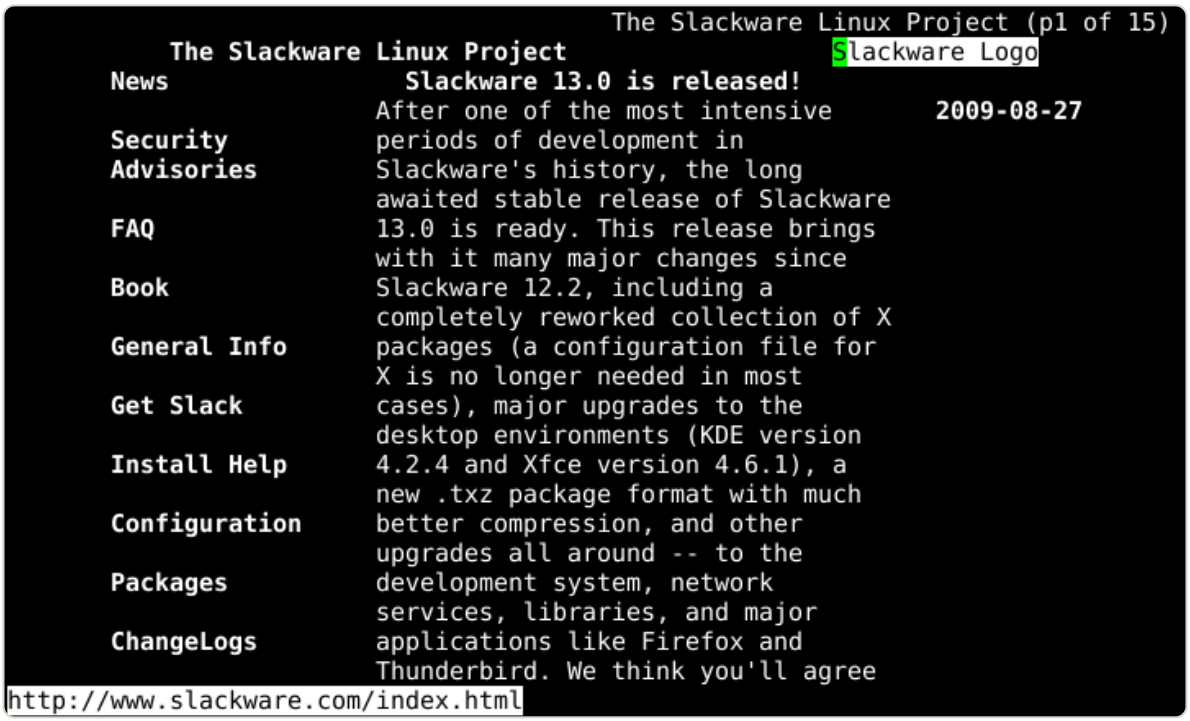
16.2.1 lynx

The oldest console-based web browser included with Slackware is definitely `lynx` (1), a very capable if somewhat limited web browser. `lynx` does not support frames, javascript, or pictures; it is strictly a text web browser. Navigation is performed using your keyboard's arrow keys and optionally, a mouse. While it lacks many features that other browsers support, `lynx` is one of the fastest web browsers you'll ever use for gathering information. For example, the `-dump` argument sends the formatted web page directly to the console, which can then be piped to other programs.



16.2.2 links

A more feature-rich alternative is the popular `links` (1), a console-based web browser that supports frames and has better table rendering than `lynx` . Like its predecessor, `links` is navigable with the arrow keys, and the use of a mouse is supported. Unlike `lynx` , it also includes a handy menu (simply click on the top line with your mouse to activate) and generally formats web pages better.



16.2.3 wget

Unlike the other browsers we've looked at, `wget` (1) is non-interactive. Rather than display HTTP content, `wget` downloads it. This takes the "browsing" out of the web browser. Unlike the dump modes of other browsers, `wget` does not format its downloads; rather it copies the content in its exact form on the web server with all tags and binary data in place. It also supports several recursive options that can effectively mirror online content to your local computer. `wget` need not operate exclusively on HTTP content; it also supports FTP and several other protocols.

```
darkstar:~# wget ftp://ftp.osuosl.org/pub/slackware/slackware-current/C
--2010-05-01 13:51:19--
ftp://ftp.osuosl.org/pub/slackware/slackware-current/ChangeLog.txt
           => `ChangeLog.txt'
Resolving ftp.osuosl.org... 64.50.236.52
Connecting to ftp.osuosl.org|64.50.236.52|:21... connected.
Logging in as anonymous ... Logged in!
==> SYST ... done.    ==> PWD ... done.
==> TYPE I ... done.  ==> CWD /pub/slackware/slackware-current ...  don
==> SIZE ChangeLog.txt ... 75306
==> PASV ... done.    ==> RETR ChangeLog.txt ... done.
Length: 75306 (74K)

100%[====================================>] 75,306       110K/s   in

2010-05-01 13:51:22 (110 KB/s) - `ChangeLog.txt' saved [75306]
```

16.3 Mail Clients

Slackware also includes a variety of email clients. If you're using a graphical desktop, you'll find `Thunderbird` , `Kmail` , `sylpheed` and others. As with web browsers, there are also applications that function within the shell. Once you

start using an email client in the console, you may find yourself not wanting to use anything else; the flexibility and configurability can be addicting.

16.3.1 pine

`pine` is one of the oldest command-line interface mail clients still in existance and remains one of the most user-friendly. `pine` was created by the University of Washington and carries with it both a trademark and a copyright license that are difficult to work with. Thankfully back in 2005, the university saw fit to re-write it without the trademark and with a more open license, so `alpine` (1), the pine-clone distributed with Slackware, was born.

To start using `alpine` , simply type `pine` at the command line. Using it is very simple due to its menu-driven system as well as the command reference neatly located at the bottom of the screen. See for yourself:

```
 ALPINE 2.02(1266)  MAIN MENU     Folder: INBOX   No Messages

     ?    HELP                    Get help using Alpine
     C    COMPOSE MESSAGE         Compose and send a message
     I    MESSAGE INDEX           View messages in current folder
     L    FOLDER LIST             Select a folder to view
     A    ADDRESS BOOK            Update address book
     S    SETUP                   Configure Alpine Options
     Q    QUIT                    Leave the Alpine program

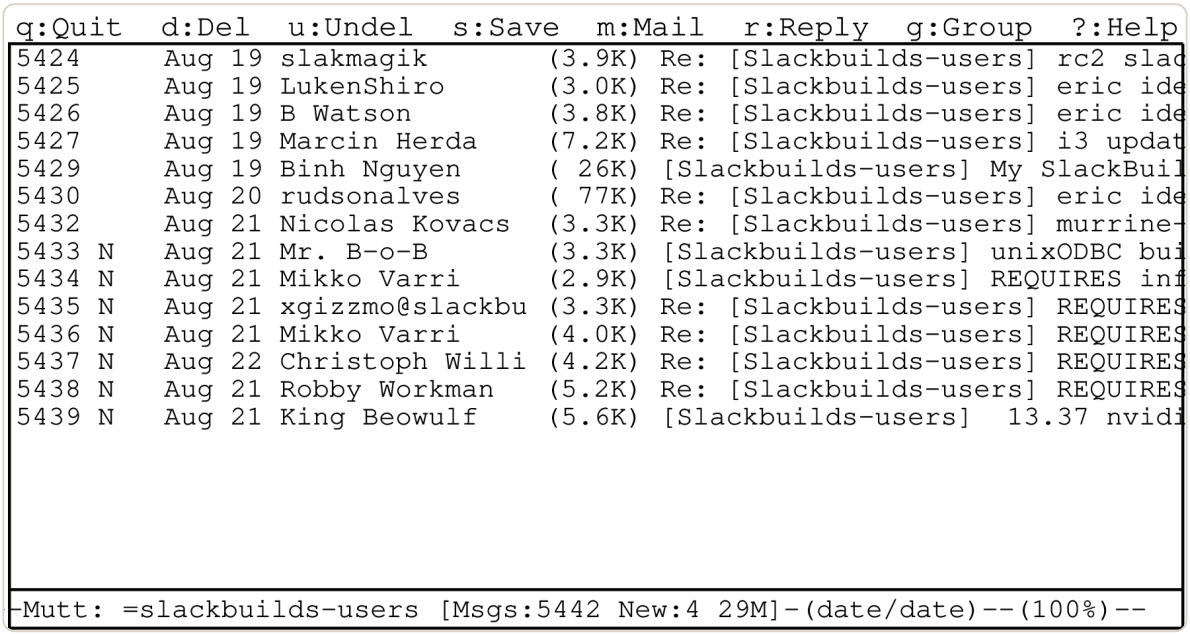
           Copyright 2006-2008 University of Washington
             Copyright 2009-2010 Re-Alpine Project
            [Folder "INBOX" opened with 0 messages]


 ? HELP                            P PrevCmd     R RelNotes
 O OTHER CMDS   > [ListFoldrs]     N NextCmd     K KBlock
```

Before configuring any mail client, you should check the documentation of your mail server to gather all of the pertinent information about what protocols and security measures your mail service uses. This will help you configure `pine` correctly. By default, `pine` will check for new e-mails delivered to a mail service running on your computer. Unless you're actually running such a mail service (many people do) this probably isn't what you want. Fortunately configuring `pine` is a straight forward process. Simply enter the [S]etup menu and chose the [C]onfig option. You'll be given an option to enter you name, mail path, SMTP server, and many other options.

16.3.2 mutt

Some people don't like `pine` . Some people want more control. Some people want a fully-configurable mail client with plugin support and a no-nonsense attitude. Those people use `mutt` (1). `mutt` isn't as user friendly as `pine` , but makes up for it with power. You won't find the user-friendly command reference at the bottom of the screen, `mutt` uses every last inch of real-estate for mail processing duty. It's feature support is extensive - threaded displays are no problem for the mighty mixed-breed! You can configure `mutt` with a `.muttrc` file in your home directory. With all the many different possible configuration options, there's even a man page for that, muttrc(5). You might want to read up on it.



Using `mutt` is unique because it is by nature a Mail User Agent (MUA), meaning its true purpose is to read and sort email. This was its only job originally, although some additional features such as retrieving mail via POP3 and even very basic transfering messages via SMTP have snuck into the application.

As is so often the case with robust console-based applications, the configuration options are myriad, and there is no "right" or "wrong" way of using `mutt` as long as it does what you want it to do. One thing to keep in mind if you are considering using `mutt` for mail handling is that its mail sending and receiving abilities are very limited. `mutt` focuses solely on sorting, reading, and composing mail messages in addition to other traditional Mail User Agent duties. This is keeping in focus with the UNIX philosophy of small tools that do one thing very well and which can be combined (or "chained") with other tools to complete whatever tasks are

required. With this in mind, you'll likely need to setup some external tool to receiving mail at a minimum.

The commands used to navigate around in `mutt` are highly customizable but the defaults can be listed by typing ? .

16.3.3 mailx

So those are great and everything, but what if you just want a mail client that isn't menu-driven? Thankfully `mailx` is here to save you.

`mailx` is based on the Berkeley Mail application, with a `mail` command appearing as early as Version 1 of AT&T's UNIX. It can be used either interactively or non-interactively.

`mailx` reads mail from your computer's mail spool and displays the usual combination of sender, subject, status, and size in a list, leaving the user at an interactive prompt. In fact, it might look familiar to you if you bothered checking your mail immediately after installing Slackware and read Pat Volkerding's greeting.

```
darkstar:~# mailx
Heirloom mailx version 12.4 7/29/08. Type ? for help.
"/var/spool/mail/root": 2 messages 2 new
>N 1 To root Thu Mar 10 23:33 52/1902 Register with the Linux counter p
>N 2 To root Thu Mar 10 23:35 321/15417 Welcome to Linux (Slackware 14
?;
```

To read a message, enter the number of the message at the prompt. This displays the message using `more` , so use the RETURN key to view the next page. Once the end of the message has been reached, press q to return to the list view, or RETURN to continue to the next message.

To see a list of available commands, enter ? at the `mail` prompt; using the commands provided, you can view the headers of mail in the spool, reply, delete, save, and many other common email tasks.

`mailx` is most powerful when used in scripting. For all of the options available for `mailx` , view its man page. A simple way to send an email to someone requires only the command itself and the destination address.

```
darkstar:~$ mailx bob@example.com
```

After the command has been issued, an interactive prompt appears for a subject line, the message body, and the end character (a single period on an otherwise empty line).

`mailx` can be used entirely without human intervention, however. Generally, it's safe to assume that any attribute you can define in the interactive shell for `mailx` can also be defined while scripting it or using it as one non-interactive command.

```
darkstar:~$ mailx -n -s "Test message" bob@example.com < ~/message.txt
```

In this example, the contents of the file `message.txt` would be sent as the message body to the specified recipient. No interaction from the user is required.

Within one's own computer (localhost) or one's own network, sending email in this manner is entirely possible. But over the internet a few more steps are usually required along the way. Of course, most notably there is usually an smtp server handling the delivery of your email. This, too, can be specified as part of your `mail` command:

```
darkstar:~$ env MAILRC=/dev/null
from="bob@example.com (Bob Dobbs)"
smtp=relay.example.com mail -n -s "Test message" connie@example.com < ~
```

In this case, the MAILRC variable is set to /dev/null in order to override any system defaults, and the smtp server as well as the `FROM:` line are defined. The rest of the command is the same as using `mailx` internally within one's own computer or network.

Over all, `mailx` is usually viewed as a mail client with the bare-minimum features; this is largely true, but when you need to be able to script sending notification emails or important update messages, it quickly becomes a lot more valuable than a fully interactive application like `pine` or `mutt` .

16.4 FTP Clients

Lots of data is stored on FTP servers the world over. In fact, Slackware Linux was first publically offered via FTP and continues to be distributed in this fashion today. Most open source software can be downloaded in source code or binary form via FTP, so knowing how to retrieve this information is a handy skill.

16.4.1 ftp

The simplest FTP client included with Slackware is named simply, `ftp` (1) and is a reliable if somewhat simple means of sending and retrieving data. `ftp` connects to an FTP server, asks for your username and password, and then allows you to put or get data to and from that server. `ftp` has fallen out of favor with more experienced users do to a lack of features, but remains a handy tool, and much of the documentation you see online will refer you to it.

Once an FTP session has been initialized, you'll be placed at a prompt somewhat like a shell. From here you can change and list directories using the "cd" and "ls" commands, just like a shell. Additionally, you may issue the "put" command to send a file to the server, or a "get" command to retrieve data from the server. If you're connecting to a public FTP server, you'll want to use the "anonymous" username and simply enter your e-mail address (or a fake one) for the password.

```
darkstar:~$ ftp ftp.osuosl.org
Name (ftp.osuosl.org:alan): anonymous
331 Please specify the password.
```

```
Password: secret
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> cd pub/slackware/slackware-current/
250 Directory successfully changed.
ftp> get ChangeLog.txt
local: ChangeLog.txt remote: ChangeLog.txt
200 PORT command successful. Consider using PASV.
150 Opening BINARY mode data connection for ChangeLog.txt (33967
bytes).
226 File send OK.
33967 bytes received in 0.351 secs (94 Kbytes/sec)
ftp> bye
221 Goodbye.
```

16.4.2 ncftp

`ncftp` (1) (pronounced nick-f-t-p), is a more feature rich successor to `ftp` , supporting tab completion and recursive retrieval. It automatically connects to a server as the anonymous user, unless you specify a different username on the commandline with the `-u` argument. The primary advantage over `ftp` is the ability to send and retrieve multiple files at once with the "mput" and "mget" commands. If you pass the `-R` argument to either of them, they will recursively put or get data from directories.

```
darkstar:~# ncftp ftp.osuosl.org
Logging in...
Login successful.
Logged in to ftp.osuosl.org.
ncftp / > cd pub/slackware/slackware-current
Directory successfully changed.
ncftp ...ware/slackware-current > mget -R isolinux
isolinux/README.TXT:                                    4.63 kB 16.77
```

```
isolinux/README_SPLIT.TXT:                                 788.00 B    5.43
isolinux/f2.txt:                                           793.00 B    5.68
isolinux/initrd.img:                                        13.75 MB 837.91
isolinux/iso.sort:                                          50.00 B  354.50
isolinux/isolinux.bin:                                      14.00 kB 33.99
isolinux/isolinux.cfg:                                     487.00 B    3.30
isolinux/message.txt:                                      760.00 B    5.32
isolinux/setpkg:
```

16.4.3 lftp

The last client we're going to look at is `lftp` (1). Like `ncftp` , it supports tab completion and recursive activity, but has a more friendly license. Rather than user "mget" and "mput", all recursive operations are handled with the "mirror" command. "mirror" has many different options available, so I'll have to refer you to the man page and the built-in "help" command for complete details.

```
darkstar:~# lftp ftp.osuosl.org
lftp ftp.osuosl.org:~> cd /pub/slackware/slackware-current
cd ok, cwd=/pub/slackware/slackware-current
lftp ftp.osuosl.org:/pub/slackware/slackware-current> mirror isolinux
Total: 2 directories, 16 files, 1 symlink
New: 16 files, 1 symlink
14636789 bytes transferred in 20 seconds (703.7K/s)
lftp ftp.osuosl.org:/pub/slackware/slackware-current> bye
```

16.5 rsync

Ready to see something cool? Have you ever found yourself needing just a handful of files from a large directory, but you're not entirely sure which files you already have and which ones you need? You can download the entire directory again, but that's duplicating a lot of work. You can pick and chose, manually check everything,

but that's very tedious. Perhaps you've downloaded a large file such as an ISO, but something went wrong with the download? It doesn't make sense that you should have to pull down the entire file again if only a few bits have been corrupted. Enter `rsync` (1), a fast and versatile copying tool for local and remote files.

`rsync` uses a handful of simple, but very effective techniques to determine what needs to be changed. By checking file size and time stamps, it can determine if two files are different. If something has changed, it can determine what bytes are different, and simply download that handfull of data rather than an entire file. It is truly a marvel of modern technology.

In its simplist form, `rsync` connects to an rsync protocol server and downloads a list of files and directories, along with their sizes, timestamps, and other information. It then compares this to the local files (if any) to determine what it needs to transfer. Only files that are different will be synced. Additionally, it breaks up large files into smaller chunks and compares those chunks using a quick and simple hash function. Any chunks that match are not transferred, so the amount of data that must be copied can be dramatically reduced. `rsync` also supports compression, verbose output, file deletion, permission handling, and many other options. For a complete list, you'll need to refer to the man page, but I've included a small table of some of the more common options.

-v	**Increased verbosity**
-c	Checksum all files rather than relying on file size and timestamp
-a	Archive mode (equivilant to -rlptgoD)
-e	Specify a remote shell to use
-r	Recursive mode
-u	Update - skip files that are newer on the receiving end
-p	Preserve permissions
-n	Dry-run - perform a trial run without making any changes
-z	Compress - handy for slow network connections

Due to the power and versatility of `rsync` , it can be invoked in a number of ways. The following two examples connect to an rsync protocol server to retrieve some information and to another server via ssh to encrypt the transmission.

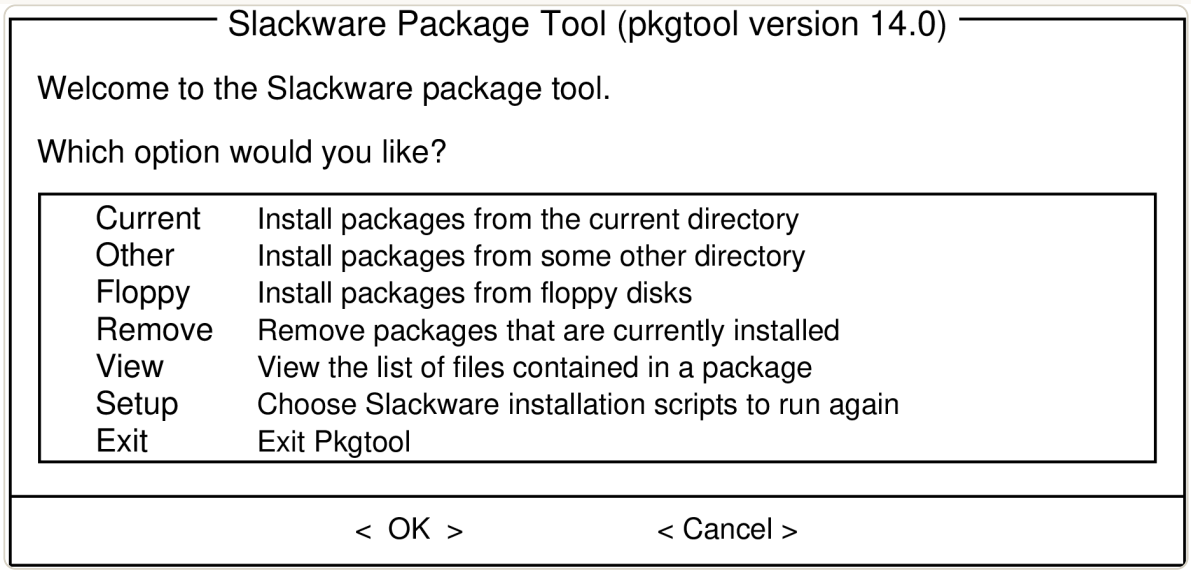
```
darkstar:~# rsync -avz rsync://ftp.osuosl.org/pub/slackware/slackware-c
/src/slackware-current/
darkstar:~# rsync -e ssh ftp.slackware.com:/home/alan/foo /tmp/foo
```

17. Package Management

Package management is an essential part of any Linux distribution. Every piece of software included by Slackware, along with many third-party tools are distributed as source code that can be compiled, but compiling all those thousands of different applications and libraries is tedious and time consuming. That's why many people prefer to install pre-compiled software packages. In fact, when you installed Slackware, the `setup` program primarily worked by running package management tools on a list of packages. Here we'll look at the various tools used for handling Slackware packages.

17.1 pkgtool

The simplest way to perform package maintenance tasks is to invoke `pkgtool` (8), a menu-driven interface to some of the other tools. `pkgtool` allows you to install or remove packages as well as view the contents of those packages and the list of currently installed packages in a user-friendly ncurses interface.



`pkgtool` is a convenient and easy way to perform the most basic tasks, but for more advanced work more flexible tools are needed.

17.2 Installing, Removing, and Upgrading Packages

While `pkgtool` scores points for convenience, `installpkg` (8) is much more capable of handling odd tasks, such as quickly installing a single package, installing an entire disk set of packages, or scripting an install. `installpkg` takes a list of packages to install, and simply installs them without asking any questions. Like all Slackware package management tools, it assumes that you know what you're doing and doesn't pretend to be smarter than you. In its simplest form, `installpkg` simply takes a list of packages to install, and does exactly what you would expect.

```
darkstar:~# installpkg blackbox-0.70.1-i486-2.txz
Verifying package blackbox-0.70.1-i486-2.txz.
Installing package blackbox-0.70.1-i486-2.txz:
PACKAGE DESCRIPTION:
#blackbox (Blackbox window manager)
#
#Blackbox is that fast, light window manager you have been looking for
#without all those annoying library dependencies.
#
#Also included in this package is the bbkeys utility for controlling
```

```
#keyboard shortcut commands from within Blackbox.
#
#The Blackbox home page is http://blackboxwm.sourceforge.net
#
Package blackbox-0.70.1-i486-2.txz installed.
```

You can of course install multiple packages at a time, and in fact use shell wild cards. The following installs all of the "N" series packages from a mounted CD-ROM:

```
darkstar:~# installpkg /mnt/cdrom/slackware/n/*.txz
```

At any given time, you can see what packages are installed on your system by listing the contents of /var/log/packages, which lists not only every application on your system but also the version number. Should you want to know what individual files were installed as a part of that package, `cat` the contents of the package:

```
darkstar:~# cat /var/log/packages/foo-1.0-x86_64.txz
```

This will return everything from the size of the package, a description of what it does, and the name and location of every file installed as a part of the package.

Removing a package is every bit as easy as installing one. As you might expect, the command to do this is `removepkg` (8). Simply tell it which packages to remove, and `removepkg` will check the contents of the package database and remove all the files and directories for that package with one caveat. If that file is included in multiple installed packages, it will be skipped and if a directory has new files in it, the directory will be left in place. Because of this, removing packages takes a good while longer than installing them.

```
darkstar:~# removepkg blackbox-0.70.1-i486-2.txz
```

Finally, upgrading is just as easy with (you guessed it), `upgradepkg` (8) which first installs a new package, then removes whatever files and directories are left-over from the old package. One important thing to remember is that `upgradepkg` doesn't check to see if the previously installed package has a higher version number than the "new" package, so it can also be used to downgrade to older versions.

```
darkstar:~# upgradepkg blackbox-0.70.1-i486-2.txz
+==============================================================================
| Upgrading blackbox-0.65.0-x86_64-4 package using
./blackbox-0.70.1-i486-2.txz
+==============================================================================

Pre-installing package blackbox-0.70.1-i486-2...

Removing package
/var/log/packages/blackbox-0.65.0-x86_64-4-upgraded-2010-02-23,16:50:51
    --> Deleting symlink /usr/share/blackbox/nls/POSIX
    --> Deleting symlink /usr/share/blackbox/nls/US_ASCII
    --> Deleting symlink /usr/share/blackbox/nls/de
    --> Deleting symlink /usr/share/blackbox/nls/en
    --> Deleting symlink /usr/share/blackbox/nls/en_GB
...
Package blackbox-0.65.0-x86_64-4 upgraded with new package
./blackbox-0.70.1-i486-2.txz.
```

All of these tools have useful arguments. For example, the `--root` to `installpkg` will install packages into an arbitrary directory. The `--dry-run` argument will instruct `upgradepkg` to simply tell you what it would attempt

without actually making any changes to the system. For complete details, you should (as always) refer to the man pages.

17.3 Package Compression Formats

In the past, all Slackware packages were compressed with the `gzip` (1) compression utility, which was a good compromise between compression speed and size. Recently, new compression schemes have been added and the package management tools have been upgraded to handle these. Today, official Slackware packages are compressed with the `xz` utility and end with .txz extensions. Older packages (and many third party packages) still use the .tgz extension.

It's worth emphasizing that .tgz and .txz (or, more succinctly, .t?z files) are very standard, non-unique extensions for compressed .tar files. This has many advantages; they're easy to build on nearly any UNIX system (many other package formats require special toolchains), and they're just as simple to de-construct.

However, it is also important to realize that just because all Slackware packages *are* .t?z files, not all .t?z files are Slackware packages. `Installpkg` won't magically install just any .t?z file, only Slackware packages.

17.3.1 slackpkg

`Slackpkg` is an automated tool for management of Slackware Linux Packages. It originally appeared in /extra for the release of slackware-12.1, and since the release of slackware-12.2 it has been included in the ap/ series of a base installation.

Just as you are able to use `installpkg` to install Slackware packages from the /extra directory included on the install media, you can use `slackpkg` to pull packages from the Internet and install them. This is particularly useful for security updates or significant application upgrades that are posted to the Slackware servers, some of which you may want to start using on your own system.

Without `slackpkg` , the process would be:

1. Notice in the Slackware changelog that an update has been released.
2. Look on your local Slackware mirror to find a download link of the package.

3. Download the package from a Slackware mirror to your hard drive.
4. `installpkg` `pkgtool`

With `slackpkg` , this is reduced to:

1. Notice in the Slackware changelog that an update for `foo` has been released.
2. `slackpkg` `install` foo

Clearly, this streamlines a fairly common task.

To use `slackpkg` , configure your system with a Slackware mirror by editing `/etc/slackpkg/mirrors` as root. Find the mirror that is associated with your Slackware version and architecture, and uncomment it. This list of mirrors offers ftp and http access, but you must uncomment *only one* mirror.

Once a mirror has been selected, update the list of remote files by issuing the initial command `slackpkg update` . This should be done any time you notice that a new package has been posted (regularly checking in with the Slackware changelog is recommended; see for more information).

To search for a package, use `slackpkg search foo` , and to install use `slackpkg install foo` .

Once a package has been installed with `slackpkg` , it can be removed or upgraded using `pkgtool` and the other package management commands as detailed in the section called “Installing, Removing, and Upgrading Packages”.

For more information see the `man` pages for slackpkg(8) and slackpkg.conf(5), and see its website at http://www.slackpkg.org/

17.3.2 rpm2tgz

One of the most ubiquitous package formats for Linux software is RPM; it's not uncommon to find a developer offering their application for download as either source code or an RPM file, and no more. In this case, you would have three options:

1. Build your own Slackware package.

2. Compile and install directly from source code.

3. Convert and install from RPM.

Building from source code or creating your own Slackware package is usually not as complex as you might think but installing directly from source code is generally discouraged because there is no easy way to track what has been installed on your system after issuing the `make install` command. Building your own Slackware packages is outside the scope of this chapter. So this leaves us with the helpful tool `rpm2tgz` .

`rpm2tgz` converts RPM packages into a Slackware package that can then be installed via `pkgtool` or `installpkg` . This circumvents the need to create your own Slackware package but grants you the benefit of being able to remove, update, and track what you've installed.

While a Slackware package is just a shell script and source code, an RPM package can by comparison be a maze of dependency listings and special instructions. Therefore, `rpm2tgz` will not always work, especially on very complex applications, and it will never magically resolve dependencies.

To try `rpm2tgz` , download an RPM file from a trusted source and convert it:

```
rpm2tgz foo-x.x.xx.rpm
```

The result is a .tgz file, so after the conversion is finished, the original RPM can safely be discarded. Use `installpkg` to install the Slackware package you've just created, provided that you've installed all dependency code for the application to actually function.

18. Keeping Track of Updates

18.1 The -stable Branch

After a new version of Slackware is released, the Slackware team will, as needed, release updated packages to fix serious security vulnerabilities and particularly nasty bugs. Therefore, it's important to keep up with all of the patches for your version of Slackware, which is referred to as the *-stable* branch. There is also a *-current* branch, which is where we do our development work toward the next stable release (and as such, there are often intrusive changes there), but unless you're willing to work with a possibly broken system and are able to fix things on your own, we strongly recommend that you stick with the -stable branch.

Since -stable updates aren't distributed on the disks, you'll need to obtain them from the Internet. Many people and organizations offer mirrors from which you can download the entire Slackware tree (or only the `patches/` directory) in any number of ways. While some mirrors offer web access, the most common ways of obtaining updates are via ftp and/or rsync servers. The Slackware project maintains a small list (organized by country) of known mirrors. If you're unsure which mirror to use, simply consult http://www.slackware.com/getslack/ for suggestions. If you have a major university near you, there's a good chance that they offer a mirror of numerous open source projects, and Slackware may be among them. The only real requirement for a mirror is that it be complete; usually it's best to use a mirror near where you live in order to achieve the fastest transfer times and use the least amount of Internet resources.

So how do you know when there are updates? The best way is to consult the `ChangeLog.txt` on any up-to-date mirror. You can always find the latest changelogs for the -current and most recent -stable branch on the Slackware Project's web page, but if you're running an older version of Slackware, you'll need to check a mirror.

```
darkstar:~# wget -O -
```

```
\ ftp://slackware.osuosl.org/pub/slackware/slackware64-current/ChangeLo
| less
Thu Aug 16 04:01:31 UTC 2012
Getting close! Hopefully we've cleared out most of the remaining issues
and are nearly ready here. We'll call this release candidate 2.
Unless there's a very good rationale, versions are frozen.
Any reports of remaining bugs will be gladly taken, though.
    #include <more/cowbell.h>
a/aaa_base-14.0-x86_64-4.txz: Rebuilt.
    Remove mention of HAL in the initial welcome email (mention udisks2
    instead). Thanks to Dave Margell.
a/bash-4.2.037-x86_64-1.txz: Upgraded.
```

18.2 Security Update Mailing List

While the Slackware team does release updated bugfix-only packages (i.e. not security fixes) occasionally, you're probably most interested in security fixes for vulnerabilities discovered after the -stable release. The Slackware Project maintains a mailing list that will notify you of any updated packages for such serious issues. In order to subscribe to the mailing list, send an e-mail to majordomo@slackware.com with the words 'subscribe slackware-security' in the body of the message. The majordomo will be happy to add your name to the list, and when new packages are released, it will mail an advisory to you.

18.3 Upgrading Slackware Versions

Now that we've gone this far, you should feel reasonably competent in your ability to manage your Slackware system. But what do we do with it when there's a new release? Updating from one release of Slackware to another is a lot more complicated than simply updating a few packages. Each release changes a lot of things, and while many of these changes are small, some of them can completely break your system if you haven't prepared for them and/or don't understand what is changing and why. While some Linux distributions provide highly automated tools

that attempt to handle every tiny detail for you, Slackware takes a much more hands-on approach to things.

The very first thing you should do before attempting an upgrade is the one that many people neglect: decide if it's really necessary to upgrade. If the old system is stable and doing everything you want it to do, there may be no need to do an operating system upgrade at all.

Assuming you decide to do the upgrade, then the second thing you should do is read the `CHANGES_AND_HINTS.TXT` file on your upgrade discs or a mirror. This file is updated during the development period before every release, and it lists a lot of helpful hints and tips to aid you in dealing with the changes.

Finally, read the `UPGRADE.TXT` file before proceeding. After doing these things, you may decide that it's less trouble and potential for problems to backup your configuration files and data and do a fresh installation of the new Slackware release rather than attempt a possibly tricky upgrade. However, if you still wish to continue, make backups of your data and configuration files first. At a minimum, it's good practice to backup the `/etc` and `/home` directories. This will give you a chance to perform a reinstall if something goes wrong with the upgrade.

Since every new version of Slackware has a few differences, giving complete instructions here is not only futile but potentially misleading. You should always consult the documentation included on your Slackware disks or your favorite mirror.

19. The Linux Kernel

19.1 What Does the Kernel Do?

You've probably heard people talking about compiling the kernel or building a kernel, but what exactly is the kernel and what does it do? The kernel is the center of your computer. It is the foundation for the entire operating system. The kernel acts as a bridge between the hardware and the applications. This means that the

kernel is (usually) the sole piece of software responsible for ordering around the hardware components of your computer. It is the kernel that instructs the hard drive to search for a certain data stream. It is the kernel that instructs your network card to transmit rapid changes in voltage. The kernel also listens to hardware as well. When the network card detects a remote computer sending information, it forwards that information to the kernel. This makes the kernel both the single most important piece of software on your computer and the most complex.

19.2 Working with Modules

The complexity of a modern linux kernel is staggering. The source code for the kernel weighs in at nearly 400MB uncompressed. There are thousands of developers, hundreds of options, and if everything were built together, the kernel would soon pass 100MB in size itself. In order to keep the size of the kernel down (as well as the amount of RAM needed for the kernel), most of the kernel options are built as modules. You can think of these modules as device drivers which can be inserted or removed from a running kernel at will. In truth, many of them aren't device drivers at all, but contain support for things such as network protocols, security measures, and even filesystems. In short, nearly any piece of the linux kernel can be built as a loadable module.

It's important to realize that Slackware will automatically handle loading most modules for you. When your system boots, `udevd` (8) is started and begins to probe your system's hardware. For each device it finds, it loads the proper module and created a device node in `/dev` . This usually means that you will not need to load any modules in order to use your computer, but occasionally this is necessary.

So what modules are currently loaded on your computer and how do we load and unload them? Fortunately we have a full suite of tools for handling this. As you might have guessed, the tool for listing modules is `lsmod` (8).

```
darkstar:~# lsmod
Module          Size        Used by
nls_utf8        1952        1
cifs            240600      2
i915            168584      2
```

```
drm             168128      3 i915
i2c_algo_bit    6468        1 i915
tun             12740       1
... many more lines ommitted ...
```

In addition to showing you what modules are loaded, it displays the size of each module and tells you what other modules are using it.

There are two applications for loading modules: `insmod` (8) and `modprobe` (8). Both will load modules and report any errors (such as loading a module for a device that isn't present in your system), but `modprobe` is preferred because it can load any module dependencies. Using either is straight-forward.

```
darkstar:~# insmod ext3
darkstar:~# modprobe ext4
darkstar:~# lsmod | grep ext
ext4        239928 1
jbd2         59088 1 ext4
crc16         1984 1 ext4
ext3        139408 0
jbd          48520 1 ext3
mbcache       8068 2 ext4,ext3
```

Removing modules can be a tricky process, and once again we have two programs for removing them: `rmmod` (8) and `modprobe` . In order to remove a module with modprobe, you'll need to use the `-r` argument.

```
darkstar:~# rmmod ext3
darkstar:~# modprobe -r ext4
darkstar:~# lsmod | grep ext
```

19.3 Compiling A Kernel and Why to do So

Most Slackware users will never need to compile a kernel. The huge and generic kernels contain virtually all the support you will need.

However, some users may need to compile a kernel. If your computer contains bleeding edge hardware, a newer kernel may offer improved support. Sometimes a kernel patch my be available that corrects a problem you are experiencing. In these cases a kernel compile is probably warranted. Users who simply want the latest and greatest version or who believe using a custom compiled kernel will give them greater performance can certainly upgrade, but are unlikely to actually notice any major changes.

If you still think compiling your own kernel is something you want or need to do, this section should walk you through the many steps. Compiling and installing a kernel is not that difficult, but there are a number of mistakes that can be made along the way, many of which can prevent your computer from booting and cause major frustration.

The first step is ensuring you have the kernel source code installed on your system. The kernel source package is included in the "k" disk set in the Slackware installer, or you can download another version from http://www.kernel.org/ . Traditionally, the kernel source is located in `/usr/src/linux` , a symbolic link that points to the specific kernel release used, but this is by no means set in stone. You can place the kernel source code virtually anywhere without encountering any problems.

```
darkstar:~# ls -l /usr/src
lrwxrwxrwx 1 root root 14 2009-07-22 19:59 linux -> linux-2.6.29.6/
drwxr-xr-x 23 root root 4096 2010-03-17 19:00 linux-2.6.29.6/
```

The most difficult part of any kernel compile is the kernel configuration. There are hundreds of options, many of which can optionally be compiled into modules. This means there are thousands of ways to configure a kernel. Fortunately, there are a few handy tricks that can keep you from running into too much trouble. The kernel

configuration file is `.config` . If you are very brave, you can manually edit this file with a text editor, but I highly recommend you use the kernel's built-in tools for manipulating `.config` .

Unless you are very familiar with configuring kernels, you should always start with a solid base configuration and modify it. This prevents you from skipping an important option that might force you to compile the kernel again and again until you get it right. The best kernel `.config` files to start with are those used by Slackware's default kernels. You can find them on your Slackware install disks or at your favorite mirror in the `kernels/` directory.

```
darkstar:~# mount /mnt/cdrom
darkstar:~# cd /mnt/cdrom/kernels
darkstar:/mnt/cdrom/kernels# ls
VERSIONS.TXT huge.s/ generic.s/ speakup.s/
darkstar:/mnt/cdrom/kernels# ls genric.s System.map.gz bzImage config
```

You can replace the default `.config` file easily by copying or downloading the `config` file for the kernel you wish to use as a base. Here I am using Slackware's recommended generic.s kernel for a base, but you may wish to use the huge.s config file. The generic kernel builds more things as modules and thus creates a smaller kernel image, but it usually requires the use of an initrd.

```
darkstar:/mnt/cdrom/kernels# cp generic.s/config /usr/src/linux/.config
```

Important

The Slackware kernel file lacks the "dot" while the kernel file includes it. If you forget, or simply copy the `config` to `/usr/src` whatever `.config` file was already present will be used instead.

If you want to use the configuration for the currently running kernel as your base, you may be able to locate it at `/proc/config.gz` . This is a special kernel-

related file that includes the entire kernel configuration in a compressed format and requires that your kernel was built to support it.

```
darkstar:~# zcat /proc/config.gz > /usr/src/linux/.config
```

Now that we've created a solid base configuration, it's time to make any configuration changes we want. The entire kernel build process from configuration to compilation is performed with the `make` (1) command and special arguments to it. Each argument performs a different function.

If you are upgrading to a newer kernel release, you will definitely want to use the `oldconfig` argument. This will step through your base `.config` and look for missing elements that usually indicates that the new kernel release contains additional options. Since options are added at virtually every kernel release, this is generally a good thing to do.

```
darkstar:/usr/src/linux# make oldconfig
scripts/kconfig/conf -o arch/x86/Kconfig
*
* Restart config...
*
*
* File systems
*
Second extended fs support (EXT2_FS) [M/n/y/?] m
    Ext2 extended attributes (EXT2_FS_XATTR) [N/y/?] n
    Ext2 execute in place support (EXT2_FS_XIP) [N/y/?] n
Ext3 journalling file system support (EXT3_FS) [M/n/y/?] m
    Ext3 extended attributes (EXT3_FS_XATTR) [Y/n/?] y
        Ext3 POSIX Access Control Lists (EXT3_FS_POSIX_ACL) [Y/n/?] y
        Ext3 Security Labels (EXT3_FS_SECURITY) [Y/n/?] y
```

```
The Extended 4 (ext4) filesystem (EXT4_FS) [N/m/y/?] (NEW) m
```

Here you can see that I the new kernel I am compiling has added support for a new filesystem: ext4. `oldconfig` has gone through my original configuration, kept all the old options exactly as they were set, and prompted me on what to do with new options. Typically it is save to choose the default option, but you may wish change this. `oldconfig` is a very handy tool for presenting you with only new configuration options, making it ideal for users who simply have to try out the latest kernel release.

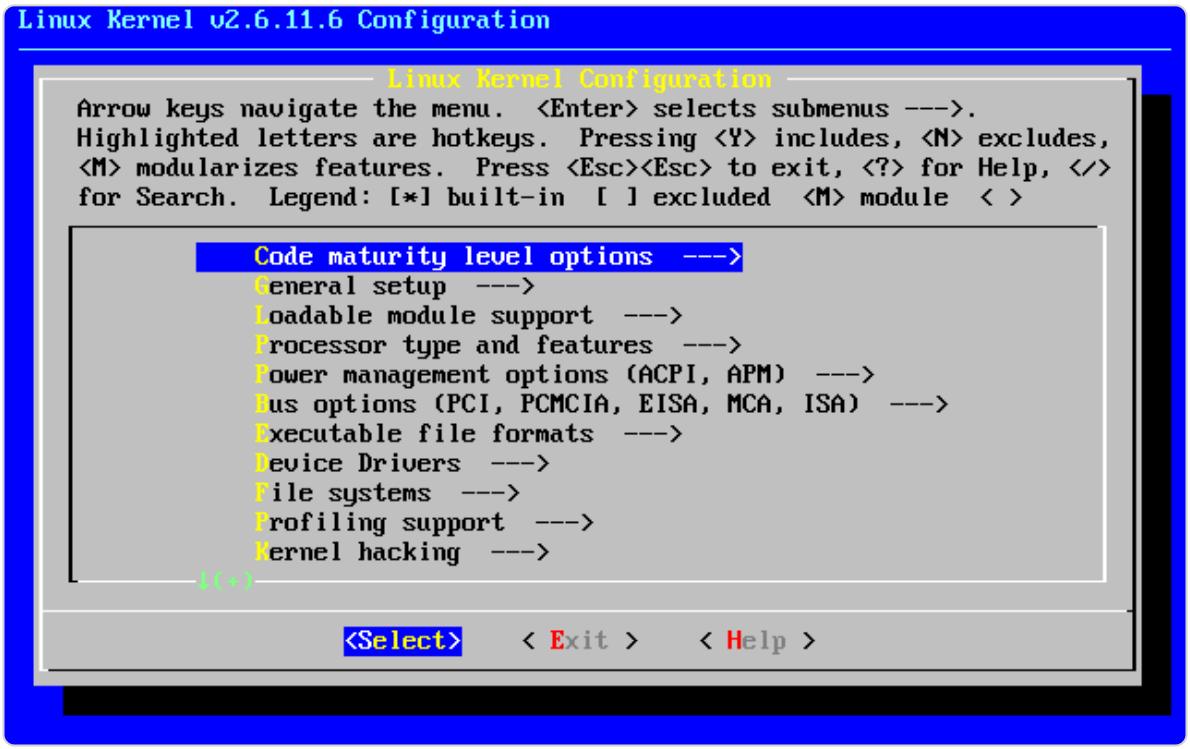
For more serious configuration tasks, there are a multitude of options. The linux kernel can be configured in three primary ways. The first is `config` , which will step through each and every option one by one and ask what you would like to do. This is so tedious that hardly anyone ever uses it anymore.

```
darkstar:/usr/src/linux# make config
scripts/kconfig/conf arch/x86/Kconfig
*
* Linux Kernel Configuration
*
*
* General setup
*
Prompt for development and/or incomplete code/drivers (EXPERIMENTAL) [Y
Local version - append to kernel release (LOCALVERSION) [] -test
Automatically append version information to the version string (LOCALVE
Support for paging of anonymous memory (swap) (SWAP) [Y/n/?]
```

Fortunately, there are two much easier ways to configure your kernel, `menuconfig` and `xconfig` . Both of these create a menu-driven program that lets you select and de-select options without having to step through each one. `menuconfig` is the most commonly used method, and the one I recommend. `xconfig` is only useful if you are attempting to compile the kernel from a

graphical user interface within `X` . Both are so similar however, that we are only going to document `menuconfig` .

Running `make menuconfig` from a terminal will present you with the friendly curses-driven interface you see below. Each kernel section is given its own submenu, and you can navigate with the arrow keys.



Warning

If you are compiling a kernel that is the same release as the stock Slackware kernel, you must set the "Local version" option. This is found on the "General setup" submenu. Failure to set this will result in your kernel compile over-writing all the modules used by the stock kernels. This can quickly render your system unbootable.

Once you've finished configuring the kernel, it's time to begin compiling it. There are many different methods for this, but the most reliable is to use `bzImage` . When you pass this argument to `make` , the kernel compilation will begin and you will see lots of data scroll through the terminal until either the compile process is complete or a fatal error is encountered.

```
darkstar:/usr/src/linux# make bzImage
scripts/kconfig/conf -s arch/x86/Kconfig
```

```
    CHK include/linux/version.h
    CHK include/linux/utsrelease.h
    SYMLINK include/asm -> include/asm-x86
    CALL scripts/checksyscalls.sh
    CC scripts/mod/empty.o
    HOSTCC scripts/mod/mk_elfconfig
    MKELF scripts/mod/elfconfig.h
    HOSTCC scripts/mod/file2alias.o
... many hundreds of lines ommitted ...
```

If the process ends in an error, you should check your kernel configuration first. Compile errors are usually caused by a fault `.config` file. Assuming everything went alright, we're still not entirely finished, as we need to build the modules.

```
darkstar:/usr/src/linux# make modules
    CHK include/linux/version.h
    CHK include/linux/utsrelease.h
    SYMLINK include/asm -> include/asm-x86
    CALL scripts/checksyscalls.sh
    HOSTCC scripts/mod/file2alias.o
... many thousands of lines omitted ...
```

If both the kernel and the modules compiles finished sucessfully, we're ready to install them. The kernel image needs to be copied into a safe location, typically the `/boot` directory, and you should give it a unique name to avoid overwriting any other kernel images located there. Traditionaly kernel images are named `vmlinuz` with the kernel release and local version appended.

```
darkstar:/usr/src/linux# cat arch/x86/boot/bzImage > /boot/vmlinuz-rele
```

```
darkstar:/usr/src/linux# make modules_install
```

Once these steps have been completed, you will have a new kernel image located under `/boot` and a new kernel modules directory under `/lib/modules` . In order to use this new kernel, you will need to edit `lilo.conf` , create an initrd for it (only if you need to load one or more of this kernel's modules to boot), and run `lilo` to update the boot loader. When you reboot, if all went according to plan, you should have an option to boot with your newly compiled kernel. If something went wrong, you may be spending some time fixing the problem.